

Fundamentos de Inteligencia Artificial para Ingenieros de
Software

Tema 1. Introducción al aprendizaje automático

Índice

Esquema

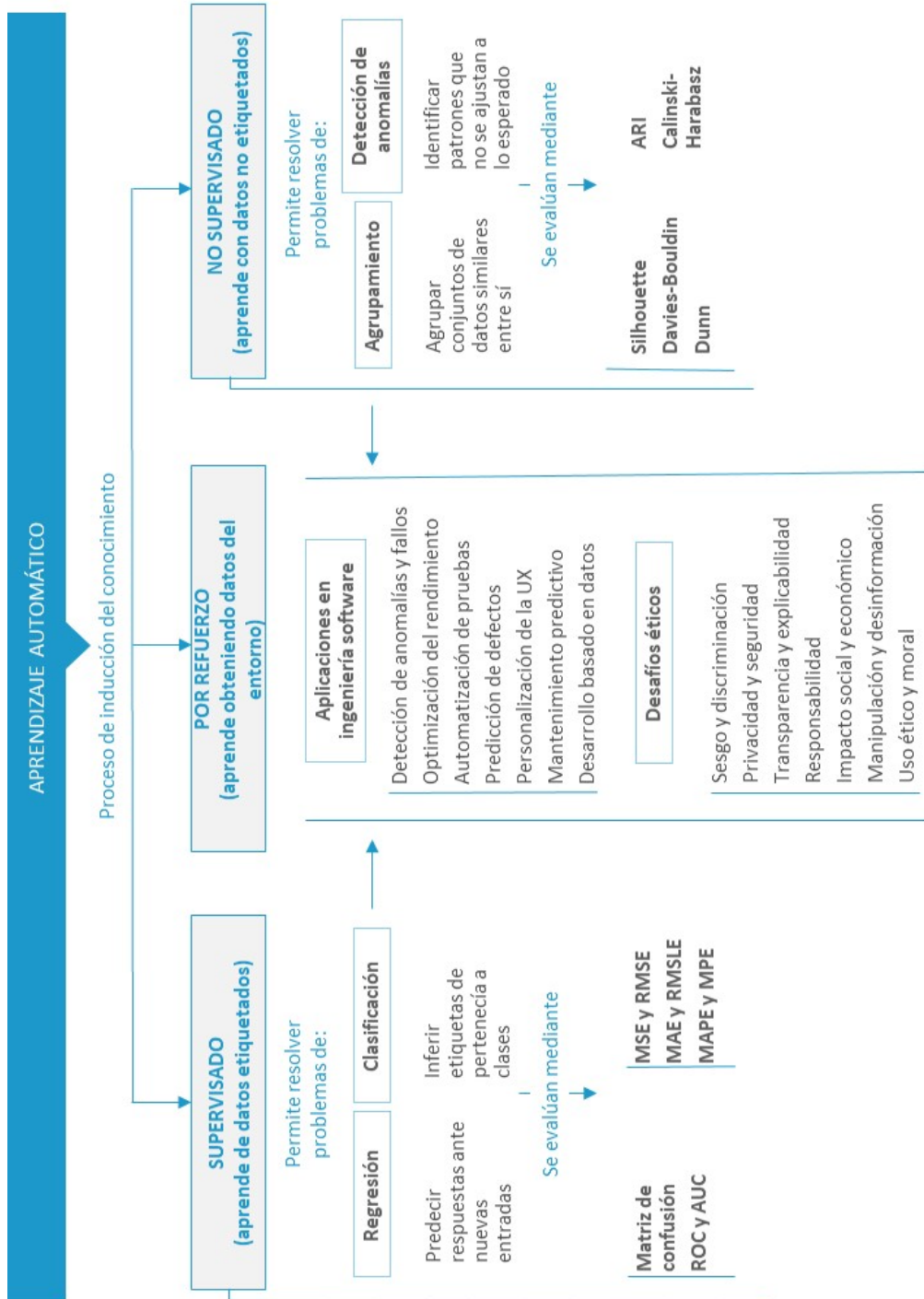
Ideas clave

- 1.1. Introducción y objetivos
- 1.2. Definición y tipos de aprendizaje automático
- 1.3. Supervisado vs. no supervisado
- 1.4. Métodos de evaluación de modelos
- 1.5. Aplicaciones prácticas en ingeniería de software
- 1.6. Desafíos éticos en el aprendizaje automático
- 1.7. Referencias bibliográficas

A fondo

- Una introducción al aprendizaje automático con Scikit-Learn
- Algoritmos de aprendizaje supervisado
- Interpretación de resultados de aprendizaje automático

Test



1.1. Introducción y objetivos

En este tema introduciremos los conceptos básicos del aprendizaje automático. Se entiende por aprendizaje automático a los algoritmos que se ejecutan en los ordenadores para aprender automáticamente en base a los datos proporcionados. Se trata de crear programas capaces de generalizar comportamientos a partir de los datos suministrados en forma de ejemplos.

Abordaremos en detalle los dos tipos de aprendizaje automático principales: supervisado y no supervisado, así como los distintos modelos y algoritmos empleados para llevarlos a cabo. A continuación, veremos cómo se evalúan dichos modelos y las métricas que se emplean para analizar el rendimiento de estos. Seguidamente, exploraremos las aplicaciones prácticas de estos tipos de aprendizaje y finalizaremos analizando las implicaciones éticas en el desarrollo de aplicaciones de inteligencia artificial.

Los objetivos del tema son:

- ▶ Conocer los tipos y características del aprendizaje supervisado.
- ▶ Diferenciar entre problemas de regresión y problemas de clasificación.
- ▶ Conocer los tipos y características del aprendizaje no supervisado.
- ▶ Diferenciar entre problemas de agrupamiento y problemas de detección de anomalías.
- ▶ Aplicar distintas métricas para evaluar modelos de aprendizaje automático.
- ▶ Explorar las distintas aplicaciones del aprendizaje automático en la ingeniería del *software*.

- ▶ Analizar las implicaciones éticas asociadas al desarrollo de aplicaciones con aprendizaje automático.

1.2. Definición y tipos de aprendizaje automático

El aprendizaje automático surge a mediados de los años 80 con la aplicación de las **redes de neuronas** y los **árboles de decisión**. El aprendizaje automático se empezó a utilizar en problemas de predicción complejos donde los modelos estadísticos clásicos no eran muy buenos. Por ejemplo, el reconocimiento de voz e imágenes, la predicción de series temporales no lineales, la predicción de los mercados financieros, el reconocimiento de texto escrito, etc.

Se puede definir el aprendizaje automático como un proceso de inducción del conocimiento fundamental para la clasificación de imágenes, el procesamiento de lenguaje natural y la toma de decisiones automatizadas.

El aprendizaje automático, también conocido como *machine learning*, se divide en tres amplias categorías que se corresponden con los paradigmas de aprendizaje según la **naturaleza** de los datos y de la **disponibilidad** de estos. Estas tres categorías son: el **aprendizaje supervisado**, el **aprendizaje no supervisado** y el **aprendizaje por refuerzo**.

El aprendizaje supervisado utiliza **ejemplos conocidos** para obtener las inferencias, mientras que el aprendizaje no supervisado no dispone de ejemplos con un objetivo o etiqueta conocido. Finalmente, en el aprendizaje por refuerzo es un agente de *software* que aprende a medida que **interactúa** con el entorno; de esta manera, obtiene los datos a partir de la **respuesta** del entorno a las acciones que el agente realiza.

A su vez, los **problemas** se pueden dividir en los siguientes **subtipos**:

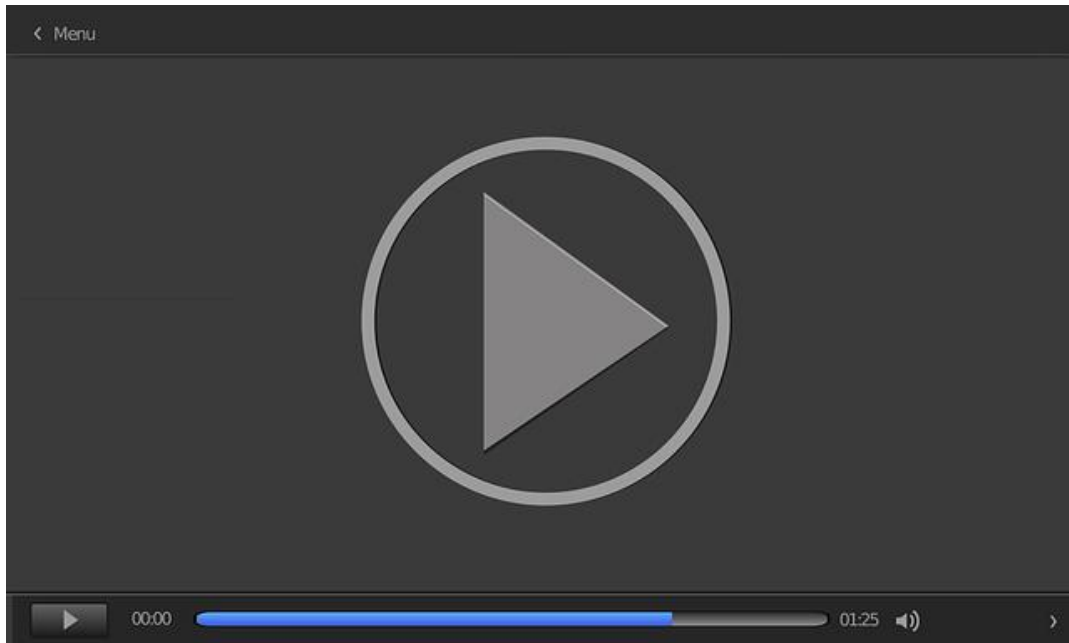
► **Aprendizaje supervisado:**

- Los problemas de **regresión** tienen como objetivo predecir un **valor numérico continuo**.
- Los problemas de **clasificación** tienen como objetivo predecir el **valor** de una **etiqueta o categoría**.

► **Aprendizaje no supervisado:**

- Los problemas de **agrupamiento** buscan encontrar **patrones** en los datos mediante la relación de grupos de datos similares.
- Los problemas de **detección de anomalías** buscan encontrar datos que no siguen **patrones establecidos** en el resto del conjunto de datos.

En el vídeo *Introducción al aprendizaje por refuerzo* se presentan los conceptos del aprendizaje por refuerzo, cómo funciona esta técnica, sus casos de uso y ejemplos de aplicación.



Introducción al aprendizaje por refuerzo

Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=9737130c-4465-4dc2-ab57-b1e300e80094>

1.3. Supervisado vs. no supervisado

Los principales **métodos** de aprendizaje automático que pueden beneficiar al desarrollo de *software* tanto en la integración en el proceso de **desarrollo** de un proyecto como en formar parte de la **solución diseñada** son los modelos de aprendizaje automático supervisado y no supervisado.

Aprendizaje supervisado

Recordemos que el aprendizaje supervisado es un tipo de aprendizaje automático donde se realizan inferencias por medio de una función que establece una **correspondencia** entre las **entradas** y las **salidas** del sistema. En el aprendizaje supervisado tenemos datos que son generados por una «caja negra», donde un vector de variables de entrada x (llamadas variables independientes) entran por un lado y, por otro lado, las variables de respuesta y son obtenidas.

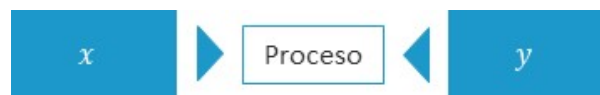


Figura 1. El aprendizaje automático busca el proceso que relaciona las variables de entrada x con las variables de respuesta y . Fuente: elaboración propia.

En el aprendizaje supervisado, para cada **observación** de las variables predictoras x existe una **medida** de la variable respuesta y .

En el caso concreto de los problemas de regresión, la variable respuesta y del sistema que se desea inferir o generalizar es una variable **cuantitativa**, es decir, una variable **numérica continua**. De esta manera, el objetivo del aprendizaje supervisado es **predecir** las respuestas que habrá en el futuro con nuevas variables de entrada.

Para ello, se utilizan los **algoritmos** y se trata al mecanismo de generación de los datos como algo desconocido. Es decir, se considera el interior de la caja como algo **complejo y desconocido**. Por tanto, el enfoque es buscar una función $f(x)$ que opere con los datos x para producir las respuestas y . De esta manera, la evaluación de este modelo se lleva a cabo por medio de su capacidad predictiva.



Figura 2. En el aprendizaje supervisado por medio de algoritmos se busca la función $f(x) = y$ para relacionar la entrada con la salida. Fuente: elaboración propia.

El otro gran grupo de algoritmos de aprendizaje supervisado son los que están enfocados a los **problemas de clasificación**. De esta manera, el aprendizaje supervisado utiliza **ejemplos conocidos** para inferir la etiqueta (clasificar) de los vectores de entrada x eligiendo una de entre varias categorías o clases.

En un problema de clasificación, las categorías o clases son las *etiquetas* que se les intenta asignar a los datos. Estos algoritmos, al igual que en los problemas de regresión, utilizan ejemplos etiquetados previamente para aprender los **patrones** para llevar a cabo una clasificación. En este tipo de problemas la variable respuesta y es una variable con **dos o más categorías**.

Un ejemplo sería asignar a un correo electrónico la categoría de *spam* o no *spam* en función de los correos recibidos previamente. Otro ejemplo es realizar un diagnóstico a un paciente en función de sus características (sexo, presión sanguínea, colesterol, etc.).

La etiqueta de clase es, por tanto, una **variable discreta**. La variable de clase representa las diferentes **categorías** o clases a las que se asigna cada instancia de

datos. Estas categorías son **finitas** y **distintas**, lo que significa que son valores discretos.

En los problemas de clasificación el objetivo es **identificar** a qué categoría o clase pertenece la nueva observación utilizando, para ello, una serie de observaciones y categorías conocidas previamente. Los problemas de clasificación se dividen en dos grandes grupos: **clasificación binaria** y clasificación **multiclase**.

Por ejemplo, en un problema de clasificación binaria la variable de clase podría tener **dos valores discretos**, como 0 y 1, donde cada valor representa una clase diferente. En cambio, en un problema de clasificación multiclase la variable de clase podría tener **varios valores discretos** cada uno correspondiente a una clase específica.

El objetivo del aprendizaje supervisado es **generalizar** utilizando, para ello, ejemplos conocidos:

- ▶ En el caso de los problemas de **regresión**, la variable respuesta y es una variable **numérica continua**.
- ▶ En el caso de los problemas de **clasificación**, la variable respuesta y es una variable con dos o **más categorías** o clases.

Aprendizaje no supervisado

El aprendizaje no supervisado es un tipo de aprendizaje del que se disponen datos, pero no tenemos una etiqueta o un objetivo establecido que queremos extraer de dichos datos. El aprendizaje no supervisado se divide en dos categorías: algoritmos de **agrupamiento** o *clustering* y algoritmos de **detección de anomalías** o *outliers*.

Los algoritmos de agrupamiento se utilizan para **organizar** un conjunto de objetos (datos) en grupos llamados **clústeres** de manera tal que los objetos dentro de un mismo clúster sean más **similares** entre sí que a los objetos de otros clústeres. Esta

técnica es muy útil en el análisis exploratorio de datos donde se busca descubrir **patrones o estructuras ocultas** en los datos.

El **agrupamiento** es muy útil para el análisis de datos, ya que permite a los investigadores y desarrolladores entender y organizar **grandes volúmenes** de datos de manera intuitiva y efectiva.

Por otra parte, los algoritmos de detección de anomalías se utilizan para identificar patrones en los datos que no se ajustan a un **comportamiento esperado** o normal. Estos patrones anómalos pueden indicar **problemas**, como fraudes, fallos en el sistema, errores en los datos o condiciones médicas inusuales. La detección de anomalías tiene como objetivo encontrar aquellos puntos de datos que se desvían significativamente del **comportamiento esperado**. Estos puntos pueden representar eventos raros o no deseados.

La detección de anomalías es una técnica crucial en el análisis de datos, ya que permite identificar y responder a **eventos inusuales** que podrían indicar problemas significativos en un sistema o proceso.

Cabe destacar que los problemas de detección de anomalías pueden resolverse también mediante **aprendizaje supervisado** si se dispusiera de un **volumen suficiente** de datos etiquetados. Sin embargo, los modelos supervisados estarían limitados a los casos anómalos ya conocidos y su capacidad para detectar **nuevas situaciones** quedaría en entredicho. Es por ello por lo que cuando se busca solucionar este tipo de problemas se opta por algoritmos de aprendizaje no supervisado.

Diferencias

En el aprendizaje supervisado se **entrena** el modelo con un conjunto de datos etiquetados donde cada entrada tiene una **salida conocida**. El objetivo es **predecir** la salida correcta para nuevas entradas basándose en el conocimiento adquirido durante el entrenamiento.

En el aprendizaje no supervisado se **entrena** el modelo con un conjunto de datos no etiquetados donde las **salidas no son conocidas**. El objetivo es agrupar o **segmentar** los datos en diferentes **categorías** basándose en sus características intrínsecas.

1.4. Métodos de evaluación de modelos

Uno de los aspectos clave en el desarrollo de modelos de aprendizaje automático es el **rendimiento** de estos. Esto es, ¿cómo podemos decirle a un modelo que el entrenamiento progresa correctamente? Para llevar a cabo esta tarea existe una serie de **métricas cuantitativas** que nos permiten medir el rendimiento de los modelos de aprendizaje automático.

A esta métrica empleada se la conoce también como **función de pérdida** (*loss-function*). Durante la fase de entrenamiento, los modelos de ML buscan **minimizar** o **maximizar** esta función de pérdida, de manera que los resultados obtenidos sean lo más correctos posibles.

Según el tipo de problema al que nos enfrentemos (regresión, clasificación, agrupamiento o detección de anomalías) emplearemos unas técnicas u otras.

Métricas para los problemas de clasificación

Matriz de confusión

La mejor métrica de rendimiento de un algoritmo de clasificación es saber si el clasificador tiene éxito para su propósito. Por tanto, para evaluar un clasificador se pueden utilizar los **valores predichos** de las clases y los valores **reales** de las clases o la probabilidad estimada de la predicción.

Un método habitual para describir el rendimiento de un modelo de clasificación es la **matriz de confusión**: una matriz simple de las clases observadas y predichas para los datos.

Las celdas en la diagonal indican los casos en los que las clases se predicen correctamente, mientras que las celdas fuera de la diagonal ilustran el número de errores para cada caso posible. Las celdas de la tabla indican el número de

verdaderos positivos (TP), falsos positivos (FP), verdaderos negativos (TN) y falsos negativos (FN).

	Reales	
	Positiva	Negativa
Predichos	Positiva	Negativa
Positiva	<i>True positive (TP)</i>	<i>False positive (FP)</i>
Negativa	<i>False negative (FN)</i>	<i>True negative (TN)</i>

Tabla 1. La matriz de confusión para dos posibles clases. Fuente: elaboración propia.

Los valores se pueden **definir** de la siguiente forma:

- ▶ La tasa de **verdaderos positivos** (TP o *true positives*) se trata de las clasificaciones correctas de las instancias que corresponden a la clase positiva.
- ▶ La tasa de **falsos positivos** (FP o *false positives*) se trata de las clasificaciones de la clase negativa que han sido incorrectamente clasificadas como clase positiva.
- ▶ La tasa de **verdaderos negativos** (TN o *true negatives*) se trata de las clasificaciones correctas de las instancias que corresponden a la clase negativa.
- ▶ La tasa de **falsos negativos** (FN o *false negatives*) se trata de las clasificaciones de la clase positiva que han sido incorrectamente clasificadas como clase negativa.

A la hora de evaluar los algoritmos de clasificación es importante diferenciar entre una **clasificación binaria** o **multiclase**. De esta manera, una de las métricas más comunes es la matriz de confusión que nos proporciona información acerca de los **aciertos** y **errores** de cada una de las clases y, a partir de esta, se pueden obtener métricas como *accuracy*, *sensitivity*, *specificity*, *precision*, *recall* y *f-measure*.

Para poder evaluar correctamente el rendimiento de un clasificador y evaluar la cantidad y calidad de los errores, hay que volver a la matriz de confusión y ver todas las métricas posibles derivadas de esta.

Una matriz de confusión es una tabla que organiza las **predicciones** en función de los **valores reales** de los datos. Una de las dimensiones de la tabla hace referencia a la categoría de los valores predichos y la otra hace referencia a las categorías reales. De esta manera, las instancias clasificadas correctamente caen en la **diagonal** de la matriz y los valores fuera de la diagonal indican las instancias clasificadas incorrectamente (predicciones incorrectas).

Las métricas de rendimiento obtenidas con la matriz de confusión se basan en cuantas **instancias** caen dentro y fuera de la diagonal. La mayoría de las métricas de rendimiento consideran la capacidad que tiene un clasificador de discernir una categoría respecto de las demás. La **categoría de interés** se conoce como clase positiva, mientras que las otras categorías se conocen como clase negativa.

La exactitud o *accuracy rate* es la **proporción** del número de **predicciones correctas** entre el número total de predicciones. Esta métrica se obtiene de la siguiente manera:

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

La métrica de **precisión** (*precision*), que también se conoce como *positive predictive value* (PPV), indica la proporción de ejemplos que son **verdaderamente positivos**, es decir, cuántas veces está en lo cierto cuando el modelo predice la clase positiva.

$$precision = \frac{TP}{TP + FP}$$

Por otro lado, la métrica de **recuperación** o *recall* indica si los resultados están **completos** y equivale a la métrica de sensibilidad. Un modelo con gran *recall* captura

un gran porcentaje de ejemplos positivos. Esta métrica también es conocida como **sensibilidad** (*sensitivity*) y muestra la *ratio* de los ejemplos positivos correctamente clasificados.

$$recall = \frac{TP}{TP + FN}$$

La *F-measure*, también conocida como F1, es una métrica que combina *precision* y *recall* utilizando la media **armónica**. Esta se utiliza porque los valores indican proporciones entre cero y uno. Se utiliza con mucha frecuencia puesto que **simplifica** el rendimiento de un algoritmo de clasificación a una **única métrica**. De forma matemática se expresa como:

$$F1 = \frac{2 * precision * recall}{recall + precision} = \frac{2 * TP}{2 * TP + FP + FN}$$

Por último, la **especificidad** (*specifity*) de un modelo indica la proporción de los **ejemplos negativos** correctamente clasificados. Estas métricas tienen valores de cero a uno, de manera que uno es lo más deseable. Para obtener estos valores se debe usar la siguiente fórmula:

$$Especificidad = \frac{TN}{TN + FP}$$

Todas las métricas para la clasificación multiclase se derivan de las métricas de la clasificación binaria, pero, en este caso, se **promedian** para todas las clases. Por tanto, la tasa de **éxito** para esta clasificación se define como la fracción de ejemplos correctamente clasificados. En general, los resultados de la clasificación **multiclase** son más **difíciles** de entender que los resultados de la clasificación binaria.

Además de la precisión, las herramientas comunes vuelven a ser las métricas derivadas de la matriz de confusión, así como el **informe de clasificación** que vimos en el caso binario, pero, esta vez, para cada una de las n posibles clases.

Curvas ROC y AUC

Las métricas vistas anteriormente, que se obtienen a partir de la matriz de confusión, establecen un **punto de corte** determinado sobre la distribución de probabilidad para estipular si una observación es clasificada como una **clase determinada**.

Por ejemplo, a los valores por encima de 0,5 se les asigna la clase positiva (1) y a los valores iguales o por debajo de 0,5 se les asigna la clase negativa (0). Con el fin de tener una mayor **visibilidad** de las decisiones que toma el clasificador en esos límites, se puede utilizar la **curva ROC** (*receiver operating characteristics*) (Altman y Bland, 1994; Brown y Davis, 2006; Fawcett, 2006).

La curva ROC fue diseñada como un método general que, dada una colección de puntos de datos continuos, determina un **umbral efectivo** de modo que los valores por encima del umbral son indicadores de un **evento específico**.

La curva ROC se utiliza para realizar una **evaluación cuantitativa** del modelo. En los modelos de clasificación binaria esta curva representa la relación entre la tasa de **verdaderos positivos** (TPR) y la tasa de **falsos positivos** (FPR) para los diferentes umbrales de decisión.

La **TPR** se calcula con el número de verdaderos positivos dividido por el número total de verdaderos positivos y falsos negativos; es decir, el *recall*. La **FPR** se calcula con el número de falsos positivos dividido por el número total de verdaderos negativos y falsos positivos.

$$FPR = \frac{FP}{FP + TN}$$

La curva ROC es útil porque muestra cómo el modelo de clasificación realiza la **distinción** entre las clases (positiva y negativa) en una **variedad de umbrales** de decisión y sin tener en cuenta un umbral específico. De esta manera, cuanto más alejada esté la curva ROC del punto de **referencia diagonal** (que indica una clasificación aleatoria), mejor será el **rendimiento** del modelo.

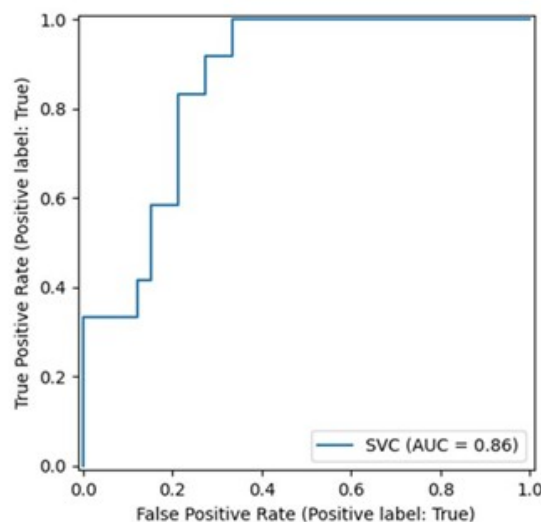


Figura 3. Gráfico de una curva ROC. Fuente: Pedregosa, et. al., 2011.

En la Figura 3 se muestra un ejemplo de curva ROC con diferentes umbrales de decisión, de manera que se presenta el comportamiento del **clasificador**. Asimismo, se puede observar que la curva ROC se acerca bastante a la esquina superior izquierda del diagrama. Por tanto, cuanto más se acerque a la **esquina superior izquierda** del gráfico, **mejor clasificará** el modelo de los datos en las categorías.

Un modelo perfecto que separe completamente a las dos clases tendría una TPR y FPR de 100 %.

Gráficamente, la curva ROC sería un solo paso entre $(0,0)$ y $(0,1)$, de manera que permanecerá **constante** desde $(0,1)$ hasta $(1,1)$. El área bajo la curva ROC (AUC) para tal modelo sería uno. Sin embargo, un modelo completamente **ineficaz** daría como resultado una curva ROC que sigue de cerca a la línea diagonal de 45° , de manera que tendría un área bajo la curva ROC de, aproximadamente, 0,50.

Para comparar visualmente los diferentes modelos se puede superponer en el mismo gráfico las curvas ROC. De esta manera, comparar las curvas ROC puede ser útil para contrastar dos o más modelos con **diferentes conjuntos de predictores** (para el mismo modelo), con diferentes **parámetros de ajuste** (dentro de las comparaciones de los modelos) y con **clasificadores** diferentes (entre los modelos).

Métricas para problemas de regresión

SE y RMSE

Cuando el resultado es un número, el método más común para caracterizar las **capacidades predictivas** de un modelo es utilizar la **raíz del error cuadrático medio** (RMSE). Esta métrica es una función de los residuos del modelo, es decir, de los valores observados menos las predicciones del modelo. El error cuadrático medio (MSE) se calcula al elevar al cuadrado los **residuos** y sumarlos. Luego, el RMSE se calcula al tomar la raíz cuadrada del MSE para que tenga las mismas **unidades** que los datos originales. Generalmente, el valor se interpreta según la **lejanía** (en promedio) de los residuos de cero o según la **distancia promedio** entre los valores observados y las predicciones del modelo.

Las **definiciones matemáticas** de estas métricas son:

El **error cuadrático medio** o *mean square error* (MSE) es la media de la diferencia entre el valor real y el valor predicho o estimado al cuadrado.

$$MSE = 1/n \sum_{i=1}^n (y_i - \widehat{y}_i)^2$$

La **raíz del error cuadrático medio** o *root mean square error* (RMSE) es la raíz cuadrada de la media de la diferencia entre el valor real y el valor predicho o estimado al cuadrado.

$$RMSE = \sqrt{1/n \sum_{i=1}^n (y_i - \widehat{y}_i)^2}$$

El MSE **penaliza** fuertemente los **errores grandes** debido a su naturaleza cuadrática. Por su parte, la RMSE también opera de esa manera, pero tiene una interpretación más **intuitiva**, ya que se devuelve a la escala original de la variable objetivo.

MAE y RMSLE

El error absoluto medio o *mean absolute error* (MAE) se define como la **diferencia** en el valor absoluto entre el **valor real** y el **predicho**. Esta métrica es menos sensible a los valores atípicos en comparación con otras métricas. Se calcula de la siguiente manera:

$$MAE = 1/n \sum_{i=1}^n |y_i - \widehat{y}_i|$$

Es simple y efectiva porque calcula la **diferencia absoluta promedio** entre las predicciones del modelo y los valores reales. El MAE es la medida de la magnitud promedio de los errores en las predicciones y no considera su dirección, lo que significa que tanto las sobreestimaciones como las subestimaciones contribuyen por igual al error total.

La raíz del logaritmo cuadrático del error medio o *root mean squared logarithmic error* (RMSLE) es una métrica comúnmente utilizada en los problemas de **regresión** cuando las variables objetivo tienen una escala muy **amplia** y las **diferencias relativas** son más importantes que las diferencias absolutas:

$$RMSLE = \sqrt{\frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{y_i} \right)^2}$$

Esta métrica es especialmente útil cuando se trabaja con variables que tienen valores muy **dispersos** o siguen distribuciones de **cola larga** (como los precios de las acciones, las ventas minoristas, las métricas de rendimiento en publicidad, etc.).

MAPE y MPE

Finalmente, se presentan las dos últimas métricas que se utilizan para medir el **porcentaje de error**: MAPE y MPE.

El error porcentual absoluto medio (MAPE) es el **porcentaje equivalente** de MAE.

La ecuación se parece a la del MAE, pero posee ajustes para convertir todo en **porcentajes**:

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Así, como **MAE** es la **magnitud** promedio del error producido por su modelo, **MAPE** proporciona información sobre el **rendimiento** del modelo (en términos de porcentaje) del error promedio de las predicciones en comparación con los valores

observados. Al igual que MAE, MAPE también tiene una interpretación clara, ya que los porcentajes son más fáciles de **conceptualizar** para las personas. Gracias al uso del valor absoluto, tanto MAPE como MAE son robustos a los efectos de los **valores atípicos**.

Por su parte, el **error porcentual medio** (MPE) es exactamente igual a MAPE. La única diferencia es que carece de la operación de valor absoluto:

$$MPE = \frac{100}{n} \sum \left(\frac{y - \hat{y}}{y} \right)$$

Aunque el MPE carece de la operación de **valor absoluto**, su ausencia hace que sea útil. Dado que los errores positivos y negativos se cancelarían, no se puede hacer ninguna afirmación sobre el desempeño de las predicciones del modelo en general. Sin embargo, si hay más errores negativos o positivos, este sesgo aparecerá en el MPE.

A diferencia de MAE y MAPE, el MPE nos resulta útil porque nos permite ver si nuestro modelo **subestima sistemáticamente** (más error negativo) o **sobreestima** (error positivo).

Es importante recordar que todas las métricas presentadas se obtienen a partir del cálculo de los residuos producidos por los modelos; es decir, de la **diferencia** entre el **valor real** y el **estimado**. Para cada una se utiliza la magnitud de la métrica para decidir si el modelo está funcionando bien o no. De manera general, los valores de la métrica con errores pequeños indican una buena **capacidad predictiva**, mientras que los valores grandes sugieren lo contrario.

A la hora de decidir qué métrica utilizar es importante tener en cuenta la **naturaleza** de los **datos**. Los valores atípicos pueden ser claves a la hora de tomar esta decisión, ya que algunos dominios de datos pueden ser más propensos a tener **valores atípicos**, mientras que otros pueden no verlos con tanta frecuencia.

Métricas para problemas de agrupamiento y detección de anomalías

Los algoritmos de agrupamiento se utilizan para **agrupar** un conjunto de objetos en subconjuntos (clústeres) de manera que los objetos dentro de cada grupo sean más **similares** entre sí que con los objetos de otros grupos. A continuación, se presentan las principales **métricas** utilizadas para evaluar estos algoritmos:

El índice de silueta o *silhouette score* mide la **calidad** de un agrupamiento. Se basa en la **distancia media** entre cada **punto** y los puntos de su propio **clúster** y la distancia media entre cada punto y los puntos del clúster más cercano.

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Donde $a(i)$ es la distancia media entre el punto i y todos los demás puntos en el **mismo clúster**; $b(i)$ es la distancia media entre el punto i y todos los puntos en el **clúster más cercano** al clúster de i . Un valor $s(i)$ cercano a 1 indica que el punto i está **bien agrupado**, mientras que un valor cercano a 0 indica que está en el límite entre dos clústeres. Un valor negativo quiere decir que el punto está **mal agrupado**.

El índice de Davies-Bouldin (*Davies-Bouldin Index*) mide la **calidad** de un **agrupamiento** basándose en la razón entre la dispersión dentro del clúster y la separación entre los clústeres.

$$DB = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \frac{\sigma_i + \sigma_j}{d_{ij}}$$

Donde σ_i es la dispersión del clúster i ; d_{ij} es la distancia entre los centroides de los clústeres i y j , y k es el número de clústeres. Los valores más bajos de DB indican una **mejor separación** entre los clústeres y **menor dispersión** dentro de los clústeres.

El índice de Dunn (*Dunn index*) evalúa la **calidad** de un agrupamiento al considerar la **distancia mínima** entre puntos de diferentes clústeres (separación) y la distancia **máxima** dentro de un clúster (compacidad).

$$D = \min_{1 \leq i < j \leq k} \left(\frac{\delta(C_i, C_j)}{\max_{1 \leq l \leq k} \Delta(C_l)} \right)$$

Donde $\delta(C_i, C_j)$ es la distancia entre los clústeres C_i y C_j y $\Delta(C_l)$ es la distancia máxima dentro del clúster C_l . Los valores más altos de Dunn indican un buen agrupamiento, con clústeres bien **separados** y **compactos**.

El coeficiente de Rand ajustado o *adjusted rand index* (ARI) mide la **similitud** entre dos agrupaciones (por ejemplo, la agrupación predicha y una agrupación verdadera) y ajusta por el azar.

$$ARI = \frac{RI - E[RI]}{\max(RI) - E[RI]}$$

Donde RI es el índice de Rand. ARI toma valores entre -1 y 1, donde 1 indica un **acuerdo perfecto** entre las dos agrupaciones y 0 y los valores negativos indican un **acuerdo esperado por azar**.

El índice de Calinski-Harabasz o *Calinski-Harabasz index*, también conocido como el índice de razón de varianza, mide la relación entre la **suma** de la **dispersión** dentro de los clústeres y la dispersión entre los clústeres.

$$CH = \frac{\text{trace}(B_k) / (k - 1)}{\text{trace}(W_k) / (N - k)}$$

Donde $\text{trace}(B_k)$ es la traza de la **matriz de dispersión** entre los clústeres; $\text{trace}(W_k)$ es la traza de la matriz de dispersión dentro de los clústeres; N es el número total de puntos y k es el número de clústeres. Los valores más altos indican agrupaciones más **definidas** y **compactas**.

1.5. Aplicaciones prácticas en ingeniería de software

Dentro de la rama de la ingeniería del *software*, el aprendizaje automático (ML) ha proporcionado herramientas avanzadas para mejorar la **eficiencia**, **precisión** y **capacidad predictiva** en diversas etapas del ciclo de vida del desarrollo de *software*. A continuación, se detallan las **aplicaciones prácticas** más destacadas del aprendizaje automático en la ingeniería del *software*:

Detección de anomalías y fallos

El aprendizaje automático se utiliza para identificar **patrones inusuales** en los sistemas de *software* que pueden indicar fallos o comportamientos anómalos.

- ▶ **Monitoreo en tiempo real:** los algoritmos de ML supervisan el rendimiento del *software* en tiempo real y alertan sobre anomalías antes de que causen problemas graves.
- ▶ **Análisis de registros:** las herramientas basadas en ML analizan grandes volúmenes de registros para detectar patrones que preceden a fallos, lo que permite un mantenimiento predictivo.

Optimización del rendimiento del software

El ML ayuda a mejorar el rendimiento del *software* mediante la **optimización** de **recursos** y la identificación de **cuellos de botella**.

- ▶ **Gestión de recursos:** los algoritmos de ML pueden ajustar dinámicamente la asignación de recursos (CPU, memoria) basándose en patrones de uso y demanda.
- ▶ **Optimización de código:** las herramientas de análisis estático de código basadas en ML pueden identificar las secciones de código ineficientes y sugerir optimizaciones.

Desarrollo y pruebas automatizadas

El aprendizaje automático facilita la automatización de pruebas de *software* mejorando la cobertura y eficiencia de las pruebas.

- ▶ **Generación de casos de prueba:** los algoritmos de ML generan automáticamente casos de prueba efectivos y cubren un amplio espectro de escenarios posibles.
- ▶ **Pruebas de regresión:** las herramientas basadas en ML detectan automáticamente regresiones en nuevas versiones del *software* y comparan el comportamiento con versiones anteriores.

Predicción de defectos

El ML se utiliza para predecir la probabilidad de **defectos** en el *software*, lo que permite a los desarrolladores centrarse en las áreas más **críticas**.

- ▶ **Modelos predictivos:** los algoritmos de ML entrenados con datos históricos de defectos pueden predecir la probabilidad de fallos en nuevas versiones del *software*.
- ▶ **Priorización de corrección de defectos:** basándose en las predicciones, se pueden priorizar los esfuerzos de corrección en las áreas más propensas a defectos.

Personalización de la experiencia del usuario

El ML permite personalizar la experiencia del usuario en aplicaciones de *software*, adaptándose a las preferencias y comportamientos individuales.

- ▶ **Recomendaciones personalizadas:** los sistemas de recomendación basados en ML sugieren contenido, productos o funciones relevantes para cada usuario.
- ▶ **Interfaces adaptativas:** los algoritmos de ML ajustan dinámicamente la interfaz de usuario para mejorar la usabilidad y satisfacción del usuario.

Mantenimiento predictivo

El aprendizaje automático permite **predecir** cuándo un componente de *software* puede fallar, lo que facilita el **mantenimiento proactivo**.

- ▶ **Análisis predictivo:** utilizando datos históricos y patrones de uso, los algoritmos de ML pueden predecir el tiempo de vida útil de componentes específicos del *software*.
- ▶ **Planificación de mantenimiento:** basándose en las predicciones, se pueden planificar intervenciones de mantenimiento antes de que ocurran fallos críticos.

Desarrollo basado en datos (data-driven development)

El ML facilita el desarrollo basado en datos, lo que permite a los desarrolladores tomar **decisiones informadas** basadas en **análisis de datos**.

- ▶ **Análisis de uso del *software*:** las herramientas de ML analizan cómo los usuarios interactúan con el *software*. Esto proporciona pistas para mejoras y nuevas funcionalidades.
- ▶ **A/B testing automatizado:** los algoritmos de ML pueden automatizar y optimizar experimentos A/B para determinar qué versiones del *software* proporcionan mejor rendimiento o satisfacción del usuario.

Ejemplo guiado

Podemos aplicar el aprendizaje automático para detectar **anomalías** en los logs de un servidor web, lo que a su vez permite identificar posibles fallos antes de que afecten la **operatividad** del sistema. En este caso práctico utilizaremos técnicas de **aprendizaje no supervisado**, dado que en muchos casos no se dispone de etiquetas que clasifiquen explícitamente los eventos como normales o anómalos.

Los datos que emplearemos provienen de los logs generados por un sistema de *software* que monitorea la actividad de un **servidor web**. Cada entrada del log contiene información como el **tiempo** de la **solicitud**, el **tipo de operación** realizada (GET, POST, etc.), el estado de la **respuesta** (código HTTP), el **tiempo de respuesta** del servidor y el uso de **recursos** (CPU, memoria, etc.). La Tabla 2 nos muestra un ejemplo del tipo de datos que emplearemos.

<i>Timestamp</i>	<i>Operation</i>	<i>HTTP status</i>	<i>Response time (ms)</i>	<i>CPU usage (%)</i>	<i>Memory usage (MB)</i>
2024-07-01 12:01:45	GET	200	130	8	80
2024-07-01 12:01:45	GET	200	200	13	135
2024-07-01 12:01:46	POST	500	675	81	124

Tabla 2. Ejemplo de datos del log. Fuente: elaboración propia.

El primer paso será realizar la **preparación** de los **datos**. Para ello, se debe realizar:

► **Preprocesamiento:**

- **Limpieza de datos:** se eliminan los registros incompletos o corruptos.
- **Feature engineering:** se extraen características relevantes, como la tasa de error (proporción de respuestas con códigos HTTP 4xx y 5xx) y la tasa de solicitudes por segundo.
- **Normalización:** las características numéricas, como el tiempo de respuesta, uso de CPU y memoria, son normalizadas para asegurar que cada una contribuye de manera comparable al análisis.

- **Selección de características:** selección de variables que podrían indicar anomalías, como tiempos de respuesta elevados, uso de CPU inusual o un incremento en los errores HTTP.

El siguiente paso es escoger el **modelo a entrenar**. Puesto que no disponemos de etiquetas que nos indiquen las anomalías, estamos ante un caso de **aprendizaje no supervisado**. Uno de los algoritmos más comunes para este tipo de tareas es el Isolation Forest. Este algoritmo detecta anomalías identificando puntos que son más fáciles de **aislar** en los datos.

El siguiente código muestra cómo se podrían emplear dicho algoritmo mediante la librería SciKit-Learn de Python:

```
from sklearn.ensemble import IsolationForest

# Selección de características

features = ["Response Time", "CPU Usage", "Memory Usage", "Error Rate"]

# Creación del modelo

model = IsolationForest(contamination=0.01, random_state=42)

# Entrenamiento del modelo

model.fit(data[features])
```

Con el modelo ya entrenado mediante el método `fit`, podemos aplicarlo sobre nuevos logs y así poder detectar **puntos anómalos**. Seguidamente, se deben analizar las **características** de dichos puntos para verificar si efectivamente se trata de fallos o anomalías en el sistema. El siguiente código Python muestra este proceso. Primero,

se aplica el modelo entrenado sobre **nuevos datos** mediante el método `predict`, que nos indica si todos los datos están dentro del modelo entrenado (+1) o si son anómalos (-1). Seguidamente, mostramos dichos datos para su análisis.

```
# Predicción de anomalías

anomalies = model.predict(data[features])

# Visualización de anomalías detectadas

data["Anomaly"] = anomalies

anomalous_data = data[data["Anomaly"] == -1]

print(anomalous_data)
```

El modelo entrenado se puede integrar en el sistema de monitorización en **tiempo real** del servidor de modo que ante cada nuevo log se verifique si se trata de alguna anomalía. Además, se debería configurar un sistema de **alertas** que notifique a los administradores cuando se detecte una **anomalía**.

Por último, se debe realizar un **monitoreo continuo** del modelo para revisar el **rendimiento** de este, ajustar **parámetros** y **reentrenarlo** cuando sea necesario para mantener la precisión en la detección. Por ejemplo, se pueden recopilar más datos para mejorar el algoritmo de detección o disponer de diversos modelos entrenados con datos que hayamos filtrado previamente de modo que tengamos un modelo con datos normales o esperables y otro solo con datos anómalos. Con esto último, podemos realizar una **doble verificación** de logs, puesto que cada nueva entrada del log debería aparecer «dentro» en un modelo y «fuera» en el otro.

Las **ventajas** de la implementación de este sistema serían:

- ▶ **Reducción del tiempo de respuesta:** se pueden detectar fallos antes de que impacten en los usuarios finales o responder ante ellos con celeridad.
- ▶ **Minimizar el *downtime*:** permite intervenciones proactivas para corregir fallos potenciales.
- ▶ **Mejora de la calidad del *software*:** aumenta la confiabilidad del sistema a través de un monitoreo continuo y una detección de fallos más eficiente.

1.6. Desafíos éticos en el aprendizaje automático

Al emplear técnicas de aprendizaje automático en el desarrollo del *software* nos enfrentamos también a nuevos **desafíos éticos** y **legales** que debemos tener en cuenta.

- ▶ **Sesgo y discriminación:** los modelos de ML pueden perpetuar o amplificar sesgos presentes en los datos de entrenamiento. Esto puede llevar a decisiones discriminatorias que afectan a determinados grupos sociales, étnicos o de género.
- ▶ **Privacidad y seguridad de los datos:** el aprendizaje automático a menudo requiere grandes cantidades de datos personales, lo que plantea riesgos relacionados con la privacidad y la seguridad.
- ▶ **Transparencia y explicabilidad:** los modelos de ML, especialmente los de tipo caja negra, como las redes neuronales profundas, pueden ser difíciles de interpretar y explicar.
- ▶ **Responsabilidad y rendición de cuentas:** es importante definir quién es responsable de las decisiones y acciones tomadas por los sistemas de ML.
- ▶ **Impacto social y económico:** el uso extensivo de ML puede tener impactos significativos en la sociedad y la economía, como la pérdida de empleos debido a la automatización.
- ▶ **Manipulación y desinformación:** los sistemas de ML pueden ser utilizados para manipular a las personas o difundir desinformación.
- ▶ **Uso ético y moral:** los desarrolladores deben considerar el uso ético y moral de los sistemas de ML y evitar aplicaciones que puedan causar daño o ser utilizadas de manera inapropiada.

1.7. Referencias bibliográficas

Alpaydin, E. (2020). *Introduction to machine learning* (4ª ed.). MIT Press.
<https://mitpress.mit.edu/books/introduction-machine-learning-fourth-edition>

Altman, D. G. y Bland, J. M. (1994). Statistics notes: diagnostic tests 1: sensitivity and specificity. *British Medical Journal*, 308(1552).

Binns, R. (2018). Fairness in machine learning: lessons from political philosophy. *Proceedings of Machine Learning Research*, 81, 1- 11. <https://proceedings.mlr.press/v81/binns18a/binns18a.pdf>

Brown, C. D. y Davis, H. T. (2006). Receiver operating characteristics curves and related decision measures: a tutorial. *Chemometrics and Intelligent Laboratory Systems*, 80(1), 24-38.

Chandola, V., Banerjee, A. y Kumar, V. (2009). Anomaly detection: a survey. *ACM Computing Surveys (CSUR)*, 41(3), 1-58.

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861-874.

Liu, F. T., Ting, K. M. y Zhou, Z. H. (2008). *Isolation forest* (pp. 413-422). IEEE International Conference on Data Mining.

Marsland, S. (2015). *Machine learning: an algorithmic perspective* (2ª ed.). CRC Press. <https://www.crcpress.com/Machine-Learning-An-Algorithmic-Perspective-Second-Edition/Marsland/p/book/9781466583283>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. y Duchesnay, E. (2011). Scikit-learn: machine learning in Python. *JMLR*, 12, 2825-2830. <https://qu4nt.github.io/sklearn-doc-es/visualizations.html>

Una introducción al aprendizaje automático con Scikit-Learn

An introduction to machine learning with scikit-learn. (S. f.). <https://scikit-learn.org/1.4/tutorial/basic/tutorial.html>

Scikit-learn es una de las bibliotecas más populares de código abierto para el desarrollo de modelos de aprendizaje automático. Entre sus principales ventajas encontramos: la facilidad de instalación y uso, una amplia y completa documentación con multitud de ejemplos y una gran variedad de algoritmos y técnicas de aprendizaje supervisado y no supervisado. En este sencillo tutorial nos mostrarán en unos pocos pasos cómo podemos cargar un conjunto de datos, entrenar un modelo y comenzar a predecir con él.

Algoritmos de aprendizaje supervisado

Nasteski, V. (2017). An overview of the supervised machine learning methods. *Horizons B*, 4(51-62), 56. https://www.researchgate.net/profile/Vladimir-Nasteski/publication/328146111_An_overview_of_the_supervised_machine_learning_methods

Hasta ahora hemos visto la teoría de cómo funciona el aprendizaje supervisado y sus dos vertientes, clasificación y predicción, pero ¿qué modelos y técnicas existen para ello? En este artículo se realiza una revisión de las principales técnicas de aprendizaje supervisado y cómo funcionan los árboles de decisión, la regresión lineal, la regresión logística y la clasificación Bayesiana.

Interpretación de resultados de aprendizaje automático

Escobar, J. A. (2023). *Interpretación correcta de los resultados en machine learning*. Iartificial. <https://iartificial.blog/aprendizaje/interpretacion-correcta-de-los-resultados-en-machine-learning/>

Hemos visto en este tema que disponemos de diversas métricas para evaluar los algoritmos de aprendizaje automático, sin embargo, ¿que transmiten estos valores? Uno de los puntos clave en el aprendizaje es entender e interpretar correctamente los resultados, pues nos dicen mucho sobre los datos de los que disponemos, cómo podemos mejorar los modelos y qué conclusiones podemos extraer de los resultados obtenidos. Este artículo hace un repaso de los puntos clave que se deben tomar en consideración cuando se analizan resultados de aprendizaje automático.

1. ¿Qué impulsó la investigación y avance en el aprendizaje automático?
 - A. La búsqueda de inteligencia avanzada en computación.
 - B. La resolución de problemas de predicción complejos.
 - C. El diseño de asistentes virtuales.
 - D. Las obras de ciencia ficción.

2. ¿El aprendizaje automático es?
 - A. Un algoritmo matemático que resuelve un problema.
 - B. Un algoritmo computacional inteligente.
 - C. Un proceso de inducción del conocimiento.
 - D. Un mecanismo mediante el que dotar de inteligencia a una aplicación.

3. Las categorías del aprendizaje automático se dividen según:
 - A. La naturaleza de los datos y la disponibilidad de estos.
 - B. La capacidad para resolver problemas complejos.
 - C. La complejidad computacional del problema formulado.
 - D. La existencia o no de una resolución humana al problema dado.

4. El aprendizaje automático se divide en las siguientes categorías:
 - A. Redes neuronales y aprendizaje profundo.
 - B. Supervisado, no supervisado y por refuerzo.
 - C. Supervisado y no supervisado.
 - D. Aprendizaje profundo y procesamiento del lenguaje natural.

5. El aprendizaje supervisado:

- A. Se basa en inferir una respuesta con base en los datos etiquetados previamente.
- B. Se basa en inferir una respuesta tras una supervisión humana.
- C. Se basa en inferir una respuesta a partir de los datos obtenidos del entorno.
- D. Ninguna de las anteriores es correcta.

6. El aprendizaje no supervisado:

- A. Se basa en inferir una respuesta con base en los datos etiquetados y conocidos previamente.
- B. Se basa en inferir una respuesta tras una supervisión humana.
- C. Se basa en inferir una respuesta a partir de los datos obtenidos del entorno.
- D. Se basa en inferir una respuesta con base en un conjunto de datos cuyas etiquetas no son conocidas.

7. El aprendizaje por refuerzo:

- A. Se basa en inferir una respuesta con base en los datos etiquetados y conocidos previamente.
- B. Se basa en inferir una respuesta tras una supervisión humana.
- C. Se basa en inferir una respuesta a partir de los datos obtenidos del entorno y de las acciones tomadas.
- D. Se basa en inferir una respuesta con base en un conjunto de datos cuyas etiquetas no son conocidas.

8. ¿Qué dos tipos de problemas resuelve el aprendizaje supervisado?
 - A. Regresión y clasificación.
 - B. Agrupamiento y clasificación.
 - C. Agrupamiento y detección de anomalías.
 - D. Detección de anomalías y regresión.

9. ¿Qué nos muestra una matriz de confusión?
 - A. La cantidad de confusión que hay en un algoritmo.
 - B. Una relación entre las clases observadas y predichas.
 - C. Un conjunto de datos no etiquetados listos para el entrenamiento.
 - D. La capacidad de un modelo de predecir un resultado.

10. Si entrenamos un clasificador para detectar el SPAM y queremos un modelo que maximice la detección de este, entonces debemos optimizar: (spam-positive, no spam-negative)
 - A. La tasa de TP (*true positive*).
 - B. La tasa de FP (*false positive*).
 - C. La tasa de TN (*true negative*).
 - D. La tasa de FN (*false negative*).

11. Si entrenamos un clasificador para detectar el SPAM y queremos un modelo que minimice la clasificación de correos legítimos como SPAM debemos: (spam-negative, no spam-positive)
 - A. Maximizar la tasa de TP (*true positive*).
 - B. Minimizar la tasa de FP (*false positive*).
 - C. Maximizar la tasa de TN (*true negative*).
 - D. Minimizar la tasa de FN (*false negative*).

12. La precisión de un modelo nos indica:
- A. Cuántas veces está en lo cierto cuando el modelo predice la clase positiva.
 - B. La proporción entre el número de predicciones correctas y el número total de predicciones.
 - C. La *ratio* de los ejemplos positivos correctamente clasificados.
 - D. La proporción de los ejemplos negativos correctamente clasificados.
13. La especificidad de un modelo nos indica:
- A. Cuántas veces está en lo cierto cuando el modelo predice la clase positiva.
 - B. La proporción entre el número de predicciones correctas y el número total de predicciones.
 - C. La *ratio* de los ejemplos positivos correctamente clasificados.
 - D. La proporción de los ejemplos negativos correctamente clasificados.
14. Una curva ROC nos permite:
- A. Hallar el mejor umbral para determinar cuándo un dato pertenece a una u otra categoría.
 - B. Medir la precisión de un modelo de clasificación multiclase.
 - C. Hallar el coeficiente AUC de un modelo de regresión.
 - D. Clasificar los resultados del modelo con base en la matriz de confusión.

15. ¿Cuál es una característica del MSE?
- A. Es especialmente útil cuando se trabaja con variables que tienen valores muy dispersos.
 - B. Nos permite ver si nuestro modelo subestima sistemáticamente (más error negativo) o sobreestima.
 - C. Penaliza fuertemente los errores grandes debido a su naturaleza cuadrática.
 - D. Ninguna de las anteriores es correcta.
16. Las métricas para los algoritmos de agrupamiento:
- A. Se basan en medir la distancia entre los puntos de un grupo y los del clúster más cercano.
 - B. Se basan en medir la similitud de los puntos dentro de un mismo grupo y la separación entre distintos clústeres.
 - C. Se basan en medir la distancia mínima entre distintos grupos y la máxima dentro de un mismo clúster.
 - D. Todas las anteriores son correctas.
17. El aprendizaje automático puede beneficiar en el proceso de desarrollo *software* mediante:
- A. La generación automática de código con chatbots o asistentes (por ejemplo, ChatGPT).
 - B. La generación automática de código para pruebas de *software*.
 - C. El diseño automático de los diagramas UML.
 - D. La redacción automática de los manuales de usuario.

18. Cuando se habla de sesgo y discriminación en el aprendizaje automático se hace referencia a:

- A. Los sesgos que tienen los desarrolladores que escriben código.
- B. Los sesgos que pueden contener los conjuntos de datos empleados en el entrenamiento.
- C. La inyección de código en los modelos que los haga discriminatorios.
- D. Las decisiones discriminatorias dentro del equipo de desarrollo.

19. ¿Cuál es una característica del MPE?

- A. Es especialmente útil cuando se trabaja con variables que tienen valores muy dispersos.
- B. Nos permite ver si nuestro modelo subestima sistemáticamente (más error negativo) o sobreestima.
- C. Penaliza fuertemente los errores grandes debido a su naturaleza cuadrática.
- D. Ninguna de las anteriores es correcta.

20. ¿Cuál es una característica del RMSLE?

- A. Es especialmente útil cuando se trabaja con variables que tienen valores muy dispersos.
- B. Nos permite ver si nuestro modelo subestima sistemáticamente (más error negativo) o sobreestima.
- C. Penaliza fuertemente los errores grandes debido a su naturaleza cuadrática.
- D. Ninguna de las anteriores es correcta.

Fundamentos de Inteligencia Artificial para Ingenieros de
Software

Tema 2. Fundamentos de redes neuronales y arquitecturas

Índice

Esquema

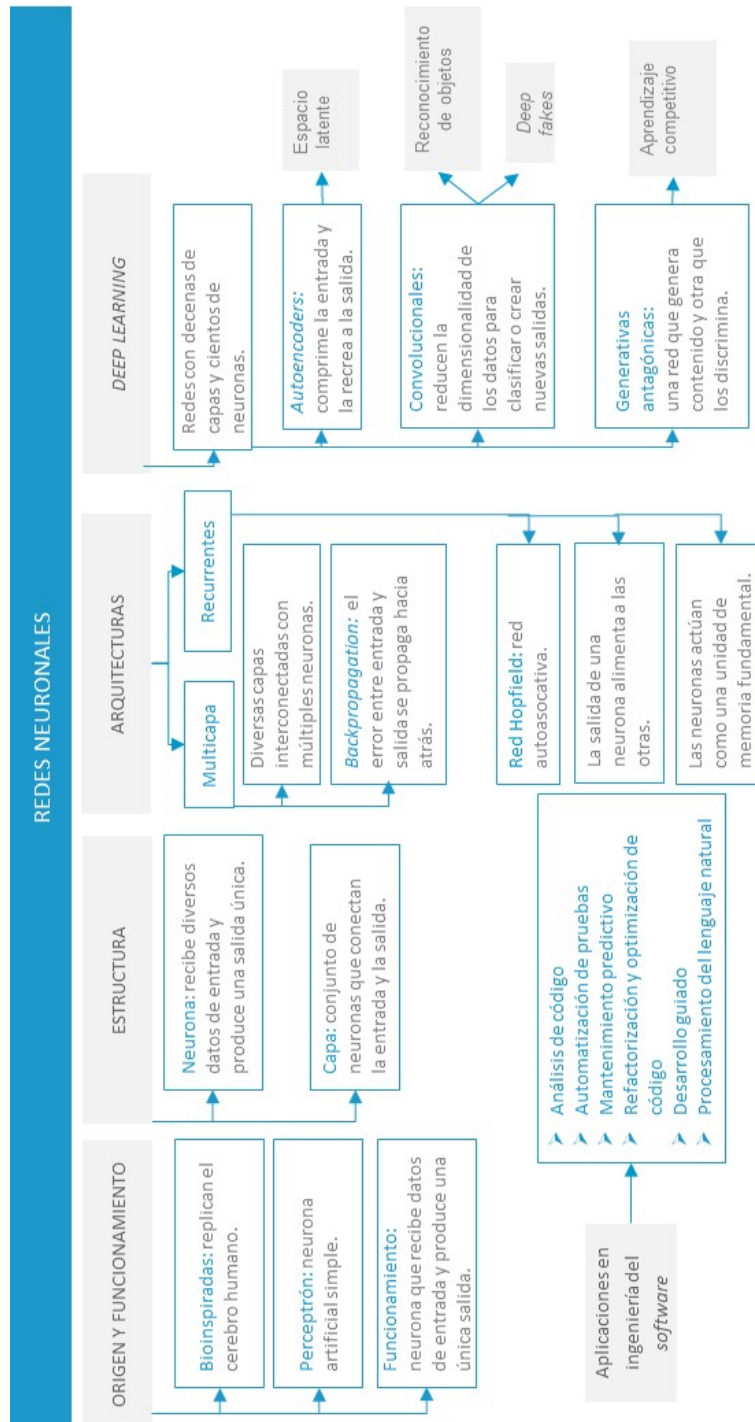
Ideas clave

- 2.1. Introducción y objetivos
- 2.2 Estructura y funcionamiento de las redes neuronales
- 2.3. Tipos de arquitecturas: feedforward, recurrentes
- 2.4. Algoritmo de Backpropagation: principios y aplicación
- 2.5. Deep learning: conceptos y ejemplos
- 2.6. Casos prácticos en ingeniería de software
- 2.7. Referencias bibliográficas

A fondo

- Implementación del Perceptrón
- Redes neuronales convolucionales en Tensorflow y PyTorch
- Introducción a los autoencoders

Test



2.1. Introducción y objetivos

En este tema introduciremos el concepto de la red neuronal y su arquitectura. Las redes neuronales artificiales son una de las técnicas de aprendizaje automático por excelencia (Alanis, Arana-Daniel y Lopez-Franco, 2019; Graupe, 2019; Rogers y Girolami, 2017). De forma muy resumida, el modelo computacional de las redes neuronales se basa en el funcionamiento del cerebro humano.

El cerebro humano se compone de unidades de procesamiento y transmisión de información denominadas neuronas. Las neuronas reciben estímulos y transmiten los impulsos nerviosos conectándose con otras neuronas o con los músculos. A esta conexión se la denomina sinapsis.

El cerebro humano, especialmente cuando se es un niño, es muy maleable. Es en esta fase de la vida cuando, precisamente, es más fácil aprender, procesar y retener información. Ante la información externa que llega el cerebro es capaz de procesar esa información y almacenarla. Para ello, las neuronas crean nuevas conexiones con otras neuronas, las cuales se establecen y fortalecen a largo plazo. De este modo, el cerebro aprende las conexiones que dan lugar a las respuestas correctas, las cuales se fortalecen, mientras que aquellas que dan lugar a respuestas incorrectas se debilitan y olvidan (Negnevitsky, 2011).

El cerebro humano se puede considerar como un sistema de procesamiento de información no lineal y muy complejo. La información se almacena y se procesa en las redes neuronales como un todo, de manera global y simultáneamente, utilizando toda la red y no localizaciones específicas (Negnevitsky, 2011).

Tal y como se ha indicado anteriormente, las redes neuronales basan su funcionamiento en la forma de aprender del cerebro. Son capaces de realizar tareas de aprendizaje de manera eficaz, incluso llevando a cabo tareas de reconocimiento que un humano es incapaz de realizar.

Las redes neuronales son apropiadas en problemas donde las instancias tienen un gran número de atributos y cuando la salida puede tener cualquier valor: real, discreto o un vector con una combinación de valores reales y discretos.

Al igual que las neuronas biológicas, las neuronas artificiales pueden recibir varios estímulos como entradas, pero únicamente cuentan con una salida. Esta única salida podrá estar ramificada para así conectar con las entradas a otras neuronas a través de unos enlaces ponderados. La relevancia de una determinada entrada a una neurona vendrá determinada por una serie de pesos que se establecen para los enlaces que conectan las neuronas. Son precisamente estos pesos los que se han de ajustar en la tarea de aprendizaje mediante las redes neuronales.

En este tema abordaremos en detalle la estructura y funcionamiento de las redes neuronales, los tipos de arquitectura sobre las que se construyen y los algoritmos empleados para el aprendizaje de las neuronas. A continuación, presentaremos el concepto de aprendizaje profundo y los casos prácticos de aplicación de las redes neuronales y las redes neuronales profundas.

Los objetivos del tema son:

- ▶ Entender el concepto de red neuronal, su estructura y funcionamiento básico.
- ▶ Diferenciar entre los tipos de arquitecturas empleadas en las redes neuronales.
- ▶ Conocer el algoritmo de Backpropagation y su aplicación en las redes neuronales.
- ▶ Entender el concepto de aprendizaje profundo y sus aplicaciones.

- ▶ Explorar las distintas aplicaciones de las redes neuronales en la ingeniería del *software*.

2.2 Estructura y funcionamiento de las redes neuronales

El componente básico de una red neuronal es la **neurona artificial** y cuya estructura básica se muestra en la Figura 1. Tal y como se ha indicado, la neurona artificial está compuesta por **varias entradas** y **una salida** que puede ramificarse en varias señales iguales.

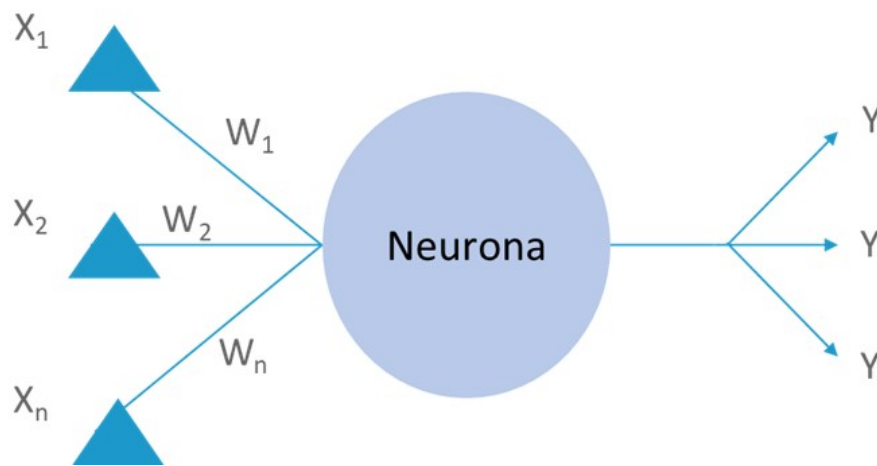


Figura 1. Estructura básica de una neurona artificial. Fuente: elaboración propia.

La salida de una neurona se puede calcular utilizando la función **signo**:

$$Y = \begin{cases} +1 & \text{si } X \geq 0 \\ -1 & \text{si } X < 0 \end{cases} \quad (1)$$

$$X = w_0 + \sum_{i=1}^n x_i w_i \quad (2)$$

La salida de la neurona también se puede calcular con otras funciones comúnmente utilizadas y que se denominan **funciones de activación**. Algunos ejemplos de la función de activación son: la función escalón, la función lineal, la función sigmoide, la tangente hiperbólica, ReLU (*rectified linear unit*), Leaky ReLU o softmax.

El perceptrón

La red neuronal artificial más sencilla es el **perceptrón**, cuya estructura se muestra en la Figura 2 (Mitchell, 1997). El perceptrón está formado por una sola capa con las entradas y salidas que se necesitan para resolver el problema planteado. La máxima simplificación de este perceptrón es lo que se conoce como **perceptrón simple**, el cual está formado por **una neurona**.

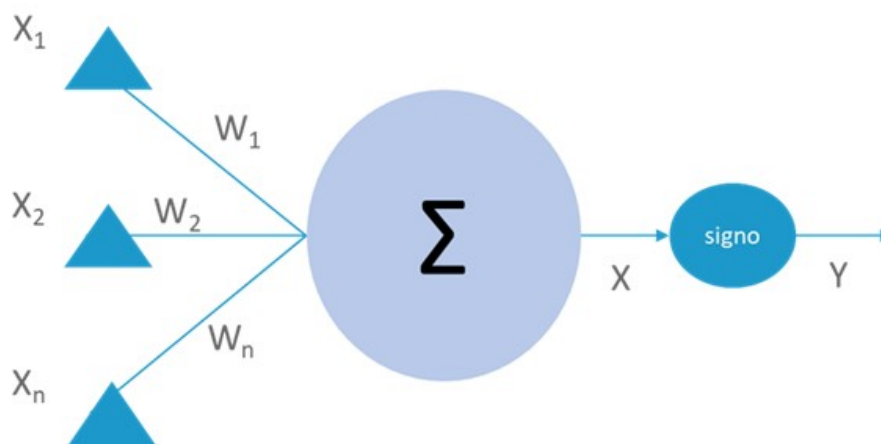


Figura 2. Estructura del perceptrón. Fuente: adaptado de Mitchell, 1997.

En esta red neuronal simple se toman las entradas y se aplican las expresiones (1) y (2) para obtener la salida. Entonces, dada esta red tan simple, ¿en qué se basa el aprendizaje de la red neuronal? A grandes rasgos, el objetivo del aprendizaje se reduce a que, dado un conjunto de datos de entrenamiento que consiste en una serie de entradas a esa red y a las salidas conocidas correspondientes, se escogen los **pesos** que se ajusten mejor a esas entradas y salidas definidas *a priori*. Este objetivo en principio es el mismo para las redes **simples** o más **complejas** y, como podemos comprobar, encaja con un modelo de **aprendizaje supervisado** en el cual los datos de entrenamiento se encuentran **etiquetados**.

Al formar la red neuronal se desconocerá el valor de los pesos, por lo que estos se asignarán inicialmente de forma **aleatoria**. De este modo, la salida que se obtiene al

aplicar los pesos en cada iteración será diferente a la salida conocida o esperada. Esta diferencia entre la **salida obtenida** y la **esperada** es lo que se conoce como el **error** (también llamado pérdida, en inglés *loss*, o costo, en inglés *cost*), que es un valor que se utiliza para **ajustar** los pesos siguiendo la regla de aprendizaje del perceptrón (Rosenblatt, 1960). La formulación matemática de esta idea es la siguiente:

$$w_i(t+1) = w_i + \alpha \times x_i(t) \times e(t) \quad (3)$$

Siendo $e(t)$ la **diferencia** entre la salida esperada y la salida real en la entrada procesada en la iteración t . En cada iteración se procesaría uno de los datos de entrenamiento disponibles. α es la tasa de aprendizaje, un valor entre 0 y 1 que pondera la **relevancia** del error obtenido en última iteración.

El algoritmo de aprendizaje de la red neuronal consta de los siguientes **pasos** (Negnevitsky, 2011):

- ▶ Se asignan **valores aleatorios** en el intervalo $[-0.5, 0.5]$ al umbral w_0 y a los pesos $w_1, w_2, \dots, w_n$.
- ▶ Se **activa** el perceptrón aplicando las entradas $x_1(t), x_2(t), \dots, x_n(t)$ y teniendo en cuenta la **salida deseada** $y_d(t)$. Se calcula la salida real en la iteración utilizando la función de activación signo (Ecuación 1). Se puede utilizar cualquier otra función de activación, tal y como se indicó al inicio de esta sección.
- ▶ A partir de la salida real obtenida en el paso 2 se **actualizan** los **pesos** aplicando la expresión de la Ecuación 3.
- ▶ Se retorna al paso 2 hasta alcanzar la **convergencia** que puede estar determinada por un error máximo admisible, por ejemplo, o por un número máximo de iteraciones.

El perceptrón es una red neuronal muy simple y es un tipo de **clasificador** o **discriminador lineal**. Es decir, durante el entrenamiento trata de elegir la mejor

recta (o hiperplano, en realidad) que separa los **vectores** de las características que forman los datos de entrenamiento a la entrada y devolviendo un valor -1 o +1 a la salida (si utilizamos la función signo como función de activación).

Esto implica que puede no encontrar una recta discriminante perfecta, sino una **aproximada**. Por eso hay que poner **límites** a la convergencia.

Sin embargo, esta simplicidad no obsta para que sobre ella esté basado el funcionamiento de diversos tipos de redes neuronales. El funcionamiento de la red neuronal vendrá determinado por los siguientes **parámetros**:

- ▶ **Arquitectura de la red:** esto es, el número de capas, número de neuronas por capa y las conexiones entre neuronas entre diferentes capas.
- ▶ **Función de activación:** función signo, función escalón, etc.
- ▶ **Algoritmo de aprendizaje** determinado principalmente por la regla de aprendizaje para ajustar pesos (por ejemplo, la expresión de la Ecuación 3).

2.3. Tipos de arquitecturas: feedforward, recurrentes

A partir del diseño del perceptrón simple, surgen las conocidas hoy en día como redes neuronales. Distinguimos dos tipos principales de redes neuronales: **redes prealimentadas** o multicapa y **redes recurrentes**.

Redes neuronales multicapa

La evolución del perceptrón simple en el perceptrón multicapa mediante la incorporación de capas de neuronas intermedias u ocultas da lugar a las redes neuronales multicapa. Las redes neuronales multicapa son redes **unidireccionales** de **alimentación** hacia adelante (*feedforward*). Dado que se trata de redes de alimentación en una **única dirección**, de atrás hacia adelante, la señal de entrada se va propagando a lo largo de las capas hacia la salida.

Estas redes tienen al menos una capa intermedia oculta, como la mostrada en la Figura 3, aunque pueden tener más de una **capa oculta**. En este sentido, las redes neuronales comerciales (en los casos en los que no hablamos de *deep learning*) incorporan 1 o 2 capas ocultas y estas capas pueden tener desde 10 a 1000 neuronas (Negnevitsky, 2011). Aunque, como es esperable, estas cifras aumentan con la mejora de la capacidad de proceso y la optimización de los algoritmos. Dado que incorporar nuevas capas supone ampliar la **carga computacional** de manera exponencial, en la práctica no es recomendable trabajar con un número elevado de capas.

En una red multicapa se puede **ajustar** el número de **capas** y **neuronas** en cada capa. Un mayor número de neuronas o capas no implica un mayor rendimiento de la red, por lo que se debe estudiar en cada caso cual es la **configuración óptima**.

El aprendizaje en las **redes multicapas** se realiza de la misma manera que el aprendizaje en la red simple del perceptrón. A partir de unos datos de entrenamiento disponibles se **alimenta** la red con las entradas, se calcula la salida y, finalmente, la diferencia entre esta salida y la salida esperada (error) se utiliza para ajustar los **pesos** con el objetivo de reducir el error.

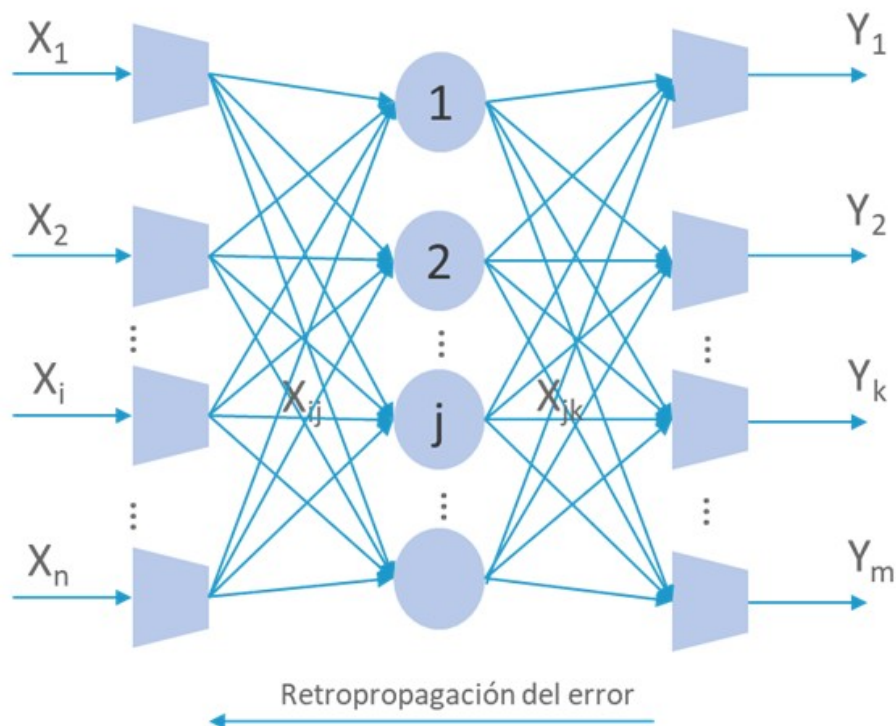


Figura 3. Retropropagación del error en una red neuronal. Fuente: elaboración propia.

A medida que se aumentan las capas y las neuronas por capa, se incrementa la **complejidad y coste computacional** de la red. Es por ello, por lo que se debe tratar de encontrar una red óptima con el menor número de neuronas y capas posibles.

Redes recurrentes

Las redes neuronales recurrentes tratan de emular las **características asociativas** de la memoria humana (Mitchell, 1997). La peculiaridad de estas redes es que sus salidas alimentan las entradas y forman un **bucle**. Se calcula la salida de la red a

partir de una entrada dada y se aplica la salida obtenida en la iteración (t) como entrada en la iteración $(t+1)$, repitiendo el proceso hasta que se alcanza una **estabilidad**, siendo la salida constante en las distintas iteraciones.

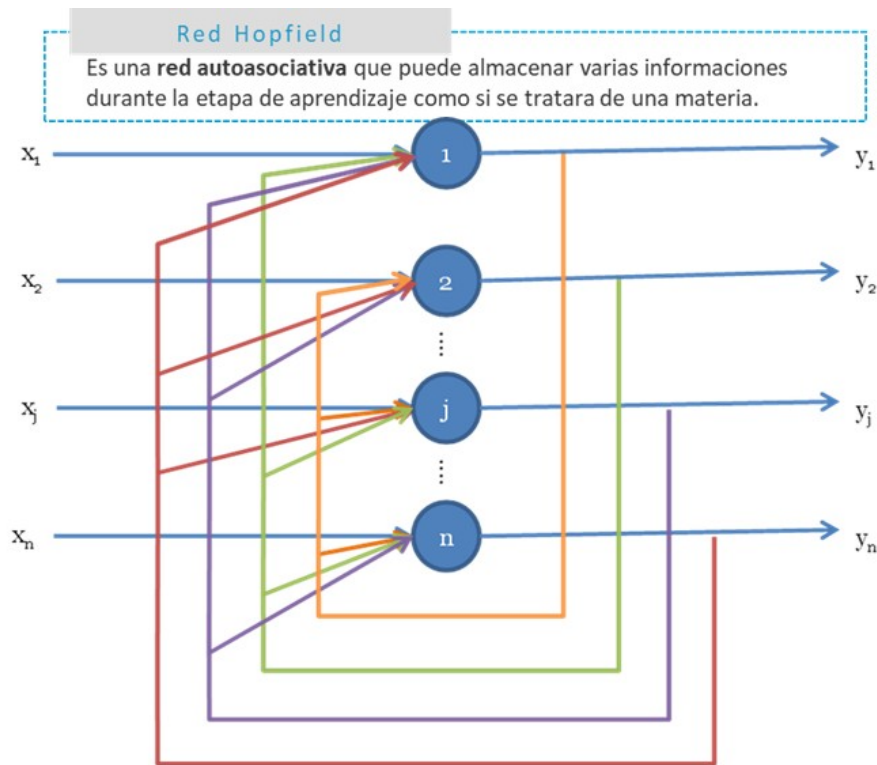


Figura 4. Arquitectura de una red neuronal recurrente monocapa (red Hopfield). Fuente: elaboración propia.

La Figura 4 muestra una red Hopfield monocapa con n neuronas. Las salidas de las n neuronas **retroalimentan** las entradas a las **diferentes neuronas**, pero nunca a ellas mismas (no hay autoretroalimentación). En una red Hopfield las salidas pueden ser **binarias** o **números reales**. Estas redes pueden utilizar diversas funciones, como la función de activación, aunque es habitual utilizar la siguiente función signo (Negnevitsky, 2011).

$$y^{signo} = \begin{cases} +1, si X > 0 \\ -1, si X < 0 \\ Y, si X = 0 \end{cases} \quad (11)$$

El estado de la red viene determinado por el conjunto de salidas $[y_1, y_2, \dots, y_n]$ y el objetivo es que la red sea capaz de **almacenar** unos determinados estados $Y_1, Y_2, \dots, Y_m, \dots, Y_M$, denominados **memorias fundamentales**.

Estos estados los almacena la **red** durante un período de entrenamiento o aprendizaje. Una vez finalizada la etapa de aprendizaje, si se presenta alguna de las informaciones almacenadas en la entrada, la red se **estabiliza** ofreciendo a la salida la misma información que se ha presentado y que coincide con la **información almacenada**. Si la entrada no coincide con una de las memorias fundamentales, la red evoluciona hacia una salida lo más **parecida** posible a una de las informaciones almacenadas.

Con este objetivo, los pasos del algoritmo de entrenamiento de la red Hopfield son los siguientes:

- ▶ Cálculo de la **matriz de pesos**.
- ▶ Comprobación de que la red es capaz de **memorizar** las memorias fundamentales.
- ▶ Comprobación con entradas de prueba de que la red recupera un **estado estable**.

Un problema de la red Hopfield es que si alcanza un estado estable no siempre ese estado es una memoria fundamental. Por otra parte, las redes Hopfield tienen una **limitación de capacidad**, por lo que el número de informaciones que puede ser aprendido o almacenado es limitado y necesita precisar de una gran cantidad de **neuronas** y de **conexiones** para almacenar pocas informaciones.

Las redes de Hopfield son redes **autoasociativas** que pueden recuperar memorias **incompletas** o información completa a partir de información incompleta. Son redes utilizadas en **reconocimiento de imágenes** o de **patrones** y en problemas de optimización, por ejemplo.

Sin embargo, las redes de Hopfield no pueden asociar una información con otra. Si en vez de una red recurrente con una capa de neuronas se tienen dos niveles, se pueden generar **memorias bidireccionales asociativas** que sí son capaces de asociar informaciones diferentes.

2.4. Algoritmo de Backpropagation: principios y aplicación

Hay muchos métodos y algoritmos para el aprendizaje de redes neuronales. En este tema se va a exponer un método comúnmente utilizado para el aprendizaje en redes neuronales denominado **retropropagación del error**, más conocido por su nombre en inglés: *backpropagation*. Las redes de retropropagación son aquellas que utilizan este método y las cuales están formadas por capas en las que **todas las neuronas** de una capa se **conectan** con todas las neuronas de la capa anterior y posterior.

La salida de una neurona X se determina con la expresión de la Ecuación 2 y la función de activación es una **función sigmoide**, es decir, una función $S(x)$ que tiene forma de S. Un ejemplo de función sigmoide es la **función logística**, cuya expresión viene dada por la Ecuación 4.

$$Y = \frac{1}{1 + e^{-x}} \quad (4)$$

Otros posibles ejemplos alternativos de funciones sigmoides son la arcotangente o la tangente hiperbólica, entre otras.

Una vez definida la arquitectura, habitualmente de 3 o 4 capas, con todas las neuronas entre capas adyacentes conectadas entre sí y conocida la función de activación, únicamente queda determinar la **regla de aprendizaje**. En las redes multicapa se utiliza el **gradiente del error** para ajustar los pesos, como se muestra en la Ecuación 5:

$$\delta_k(t) = \frac{\partial y_k(t)}{\partial X_k(t)} \times e_k(t) = \frac{\partial y_k(t)}{\partial X_k(t)} \times (y_{d,k}(t) - y_k(t)) \quad (5)$$

Siendo:

- ▶ $X_k(t)$: **entrada ponderada** a la neurona k (calculada según la Ecuación 2) en la iteración t .
- ▶ $y_k(t)$: **salida real** de la neurona k en la iteración t (calculada según la función de activación de la Ecuación 4).
- ▶ $y_{d,k}(t)$: **salida esperada** en la neurona k .

En la Ecuación 5 si se sustituye $Y_k(t)$ por la expresión dada en la Ecuación 4 se obtiene la fórmula para el **cálculo del gradiente del error**, que se detalla en la Ecuación 6:

$$\delta_k(t) = y_k(t) \times (1 - y_k(t)) \times e_k(t)$$

$$y_k(t) \times (1 - y_k(t)) \times (y_{d,k}(t) - y_k(t)) \quad (6)$$

Para **ajustar** los **pesos** de los enlaces entre la neurona j y la neurona k , es decir, w_{jk} , en la capa de salida se utiliza la expresión indicada en la Ecuación 7:

$$w_{jk}(t+1) = w_{jk}(t) + \alpha \times y_i(t) \times \delta_k(t) \quad (7)$$

Siendo $y_j(t)$ la salida de la neurona j en la **capa oculta**.

Para el caso de la capa oculta los pesos se **reajustan** según la expresión dada por la Ecuación 7 utilizando el gradiente del error. Sin embargo, el cálculo de este es diferente, ya que en la capa oculta no solo hay una salida en la neurona, sino **múltiples salidas** a las que se aplicarán diferentes pesos. Se emplea, por tanto, la expresión dada por la Ecuación 8:

$$\delta_i(t) = y_i(t) \times (1 - y_i(t)) \times \sum_{k=1}^m \delta_k(t) w_{jk}(t) \quad (8)$$

Siendo m el número de neuronas en la capa de salida, $\delta_k(p)$ es el gradiente del error calculado para la neurona de salida k -ésima y w_{jk} es el peso de la conexión entre la neurona j y la neurona k . $y_j(t)$ es la salida obtenida mediante la Ecuación 4 a partir de las entradas a la neurona j en la iteración t , es decir, $x_1(t), x_2(t), \dots, x_n(t)$:

$$y_j(t) = \frac{1}{1 + e^{-X_j(t)}} \quad (9)$$

$$X_j(t) = w_{0j} + \sum_{i=1}^n x_i(t) \times w_{ij}(t) \quad (10)$$

Por tanto, el algoritmo de aprendizaje para una red multicapa de **tres capas** consta de las siguientes **fases**:

- ▶ Establecimiento inicial de los **pesos** con valores aleatorios pequeños.
- ▶ Para cada dato de entrada $x_1(t), x_2(t), \dots, x_n(t)$ **calcular** las **salidas** de la red al activarla con esos datos de entrada.
- ▶ Cálculo del **gradiente del error** para las neuronas de la capa de salida según la Ecuación 6 y **reajuste** de sus pesos según la Ecuación 7.
- ▶ Cálculo del **gradiente del error** para las neuronas en la **capa oculta** según la Ecuación 8 y **reajuste** de sus pesos según la Ecuación 7.
- ▶ Repetir los pasos 2 y 3 hasta que se cumpla la **convergencia**, es decir, que los errores sean lo suficientemente pequeños o utilizando un criterio de parada específico.

De acuerdo con Negnevitsky (2011) los algoritmos puros de retropropagación son raramente utilizados en la práctica por la alta carga computacional, lo que da lugar a un **entrenamiento lento**. Sin embargo, se han desarrollado muchas variaciones (Mitchell, 1997) que permiten mejorar el **rendimiento**, como utilizar una función de activación del tipo tangente hiperbólica o añadiendo un «momento» en el cálculo de

los ajustes de los pesos de tal manera que la actualización de un peso en la iteración t dependa parcialmente de la actualización realizada en la iteración anterior $t-1$.

Realmente, las redes de retropropagación de dos o tres niveles tienen un gran potencial a la hora de representar **diferentes funciones** (Mitchell, 1997), como funciones booleanas, continuas, funciones de aproximación arbitraria, etc., y este tipo de redes son muy utilizadas en, por ejemplo, problemas de **reconocimiento** y **clasificación** de patrones.

2.5. Deep learning: conceptos y ejemplos

A pesar de resolver múltiples problemas eficazmente, las redes neuronales de retropropagación presentan una serie de **limitaciones** que deben ser resueltas. Estas limitaciones reducen principalmente la eficacia de este tipo de redes en **problemas complejos** en los que el número de nodos intermedios es elevado. El elevado número de nodos aumenta el número de conexiones y, por ende, el cálculo de los pesos asociados. De este modo, el entrenamiento de las redes se convierte en un proceso **ineficiente** y **costoso**, por lo que han de buscarse soluciones que nos permitan la creación de redes neuronales con decenas y cientos de nodos y capas.

Es entonces cuando aparece el **aprendizaje profundo**, más conocido por su nombre en inglés *deep learning* (Graupe, 2019; Pouyanfar, Sadiq, Yan, et. al., 2018). El *deep learning* es un área específica del aprendizaje automático que se basa en la utilización de redes neuronales con un **alto número de nodos y capas**. Este nuevo concepto dentro del aprendizaje automático lidia con procesos complejos que trabajan con volúmenes elevados de datos, así como con la **interconexión** necesaria entre los diferentes sistemas que forman parte de la solución completa.

Por otro lado, el *deep learning* se encarga del desarrollo de nuevos algoritmos y la configuración de redes que mejoren la **eficacia** de la red neuronal y, a su vez, permitan mejorar su **eficiencia**. Para ello, entre otras técnicas, esta área incluye la utilización de técnicas de **aprendizaje no supervisado** en las capas intermedias para que estas aprendan automáticamente en base a la experiencia, incluso conceptos que antes no conocían. Una de las aplicaciones más típicas del *deep learning* es en sistemas que incluyen la extracción de características y la **clasificación** en la misma solución.

Existe una amplia y creciente variedad de **redes neuronales artificiales** profundas clasificadas según su arquitectura (Liu, Cheng, Hsueh, 2017). Entre las más

relevantes se encuentran: *autoencoders* (AE), redes neuronales convolucionales (CNN por *Convolutional Neural Networks*) y las redes generativas antagónicas (GAN por *Generative Adversarial Networks*).

Autoencoders

Los **autocodificadores** o *autoencoders* (AE) son **redes neuronales simétricas** en forma de reloj de arena en las que las capas ocultas son más pequeñas que las capas de entrada y de salida (que son células de entrada y de salida que coinciden). Los autocodificadores son simétricos alrededor de las **capas medias** (que pueden ser una o dos dependiendo de si el número de capas es impar o par) y se denominan el código. De la entrada al código el autocodificador actúa como un **codificador** (comprimiendo la información) y del código a la salida el autocodificador actúa como un **decodificador**. Entre las aplicaciones de los autocodificadores se encuentran la compresión de imágenes (Tan y Eswaran, 2011), la reducción de la dimensionalidad, la generación de imágenes o su uso en sistemas de recomendación.

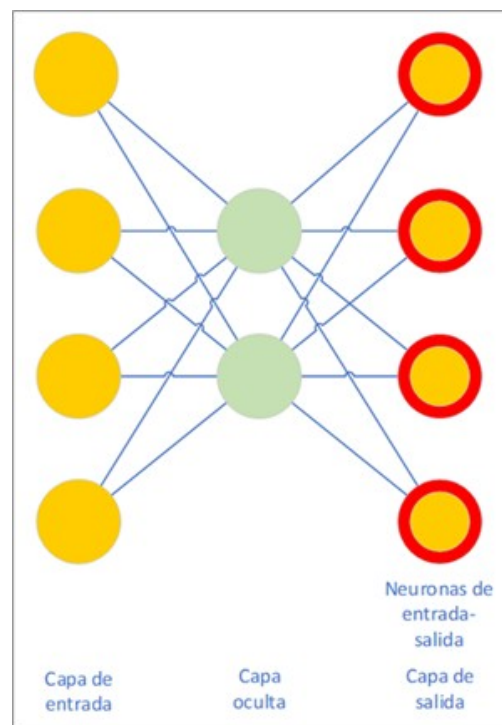


Figura 5. *Autoencoders* (AE). Fuente: elaboración propia.

Las capas medias de los *autoencoders* representan el conocido **espacio latente**. Este espacio latente actúa como una caja negra que contiene la **información** más **relevante** de los datos de entrada.

Los autoencoders pueden actuar como sistemas de **comprensión**, reduciendo los datos de entrada (imagen o sonido, por ejemplo) a una representación de su espacio latente.

Un *autoencoder*, por tanto, se puede separar en dos partes:

- ▶ **El codificador:** dados unos datos de entrada, los reduce a su espacio latente.
- ▶ **El decodificador:** dada una representación de espacio latente, esta se descomprime para devolverla al estado original.

Los autocodificadores de **supresión de ruido** (*DAE* o *Denoising Autoencoders*) se utilizan para eliminar el ruido de la **imagen** (utilizando el ruido como entrada en lugar de los datos) (Alex, Vaidhya, Thirunavukkarasu, et. al., 2017), mientras que para la extracción de **características** se utilizan los **autocodificadores de dispersión** (*SAE* o *Sparse Autoencoders*). Estos últimos se basan en una estructura en la que las capas **medias** son **mayores** que las capas de entrada y salida, a diferencia del resto de los autocodificadores (Zabalza, Ren, Zheng, et al., 2016).

Los **autocodificadores variacionales** (*VAE* o *Variational Autoencoders*) tienen una estructura similar a la de los autocodificadores, pero están relacionados con las **máquinas de Boltzmann** (BM) y las máquinas de Boltzmann restringidas (RBM). Se basan en las matemáticas bayesianas para modelar la **distribución de probabilidad** aproximada de las muestras de entrada. Entre sus aplicaciones tenemos el aprendizaje de representaciones latentes, la generación de imágenes y textos (Semeniuta, Severyn y Barth, 2017), lograr resultados de última generación en el aprendizaje semisupervisado e interpolar textos perdidos entre frases.

Redes neuronales convolucionales

Los modelos lineales no funcionan bien para el **reconocimiento de imágenes**. Imaginemos, por ejemplo, que queremos reconocer animales u objetos en imágenes y tomar una imagen o plantilla promedio de cada clase (por ejemplo, una para perros, otra para gatos, etc.) para usarla en los datos de entrenamiento y luego usar, por ejemplo, un algoritmo clasificador como el k-NN (u otro) en la fase de prueba para medir la distancia a los valores de píxeles de cada imagen no clasificada. La imagen modelo resultante de promediar todos los perros, por ejemplo, sería una imagen borrosa con una cabeza a cada lado. Esto no funcionaría. Lo que necesitamos es aprovechar las características de las redes neuronales profundas para la **clasificación** de las imágenes utilizando **capas de abstracción**. A través de este caos oculto las redes pueden aprender características cada vez más abstractas (Karpathy, 2016).

En este sentido, son especialmente útiles las **redes neuronales convolucionales** (CNN o *Convolutional Neural Networks*) y las redes neuronales convolucionales profundas (DCNN o *Deep Convolutional Neural Networks*) (Shin, Roth, Gao, *et al.*, 2016) que se utilizan para el reconocimiento de imágenes, reconocimiento de patrones o el análisis del sentimiento en los textos, entre otras aplicaciones.

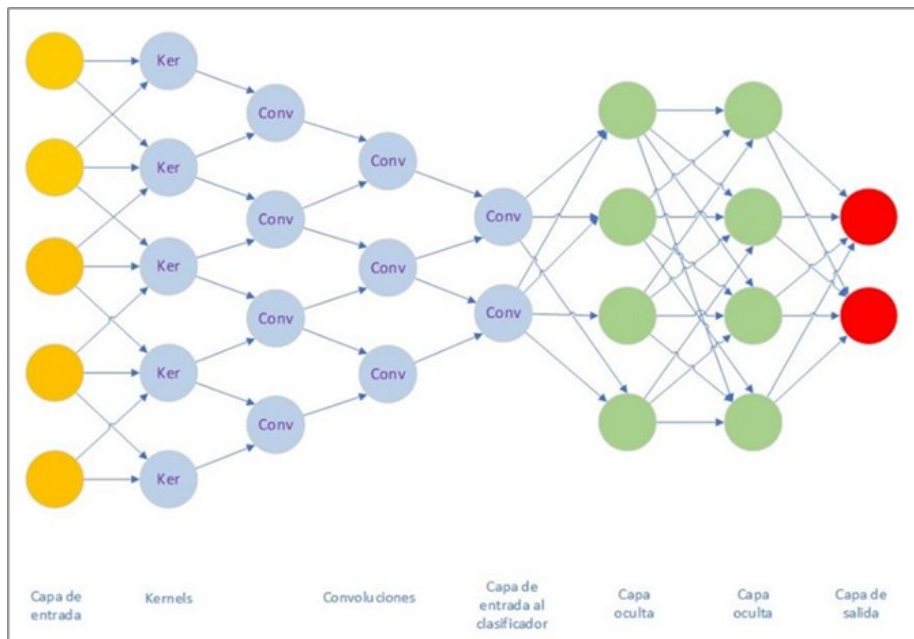


Figura 6. Redes convolucionales profundas (DCNN). Fuente: elaboración propia.

Son las más utilizadas en las aplicaciones de **búsqueda de objetos** en imágenes y vídeos, reconocimiento facial, transferencia de estilos (Gatys, Ecker y Bethge, 2016) o mejora de la **calidad de las imágenes**. Imaginemos una fotografía representada por los píxeles en escala de grises de la imagen. Primero dividimos toda la imagen en bloques de 8×8 píxeles y asignamos a cada uno un tipo de línea dominante (horizontal, vertical, las dos diagonales, un bloque completamente opaco o vacío, etc.). El resultado es una **matriz de líneas** que representa los bordes de la imagen. En la siguiente capa tomamos de nuevo un bloque de 8×8 (bloques obtenidos en la etapa anterior) y extraemos una **nueva salida** con nuevas características cada vez más abstractas y repetimos la operación una y otra vez. Esta operación se llama **convolución** y puede ser representada por una capa de la red neuronal, ya que cada neurona puede actuar como cualquier función.

A este conjunto de capas que reducen la dimensionalidad de los datos se les denomina **capas de pooling**.

En las primeras etapas las neuronas se activan representando la **línea dominante** en cada célula de 8×8 píxeles. En las etapas intermedias las neuronas representan **rasgos**, como la pata o la cabeza. En las etapas posteriores las neuronas se activan representando **conceptos**, como el gato o el perro. La salida de la última etapa de convolución se conecta a un MLP que actúa como **clasificador** basado en las características más abstractas y determina la probabilidad de pertenecer a una **clase final** (perro o gato, por ejemplo).

Las **redes deconvolucionales** (DN o DNN), también conocidas como redes gráficas inversas, son CNN invertidas en las que se pueden alimentar valores, como perro o gato, y obtener una **imagen** de uno de los animales como resultado, así como detectar **cambios** en las imágenes, por ejemplo (Alcantarilla, Stent, Ros, *et al.*, 2018).

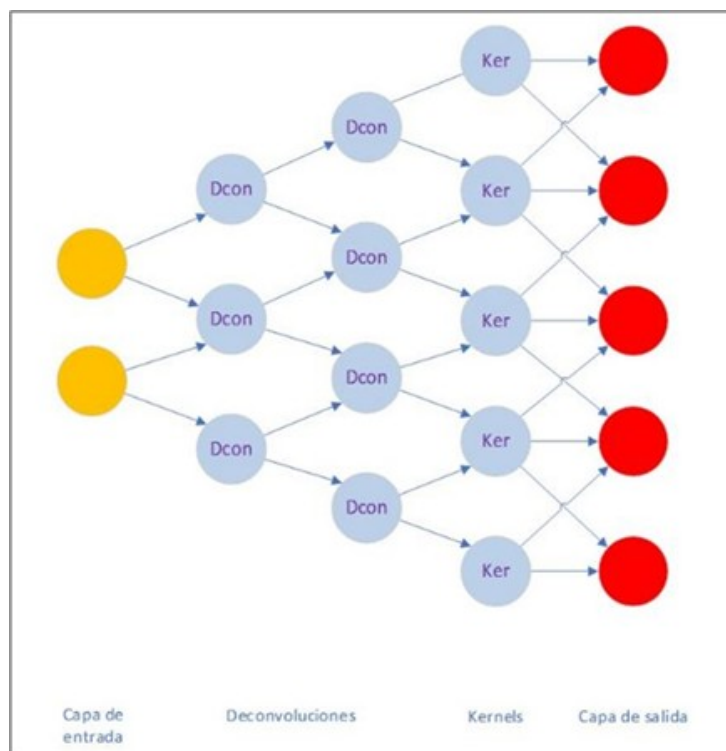


Figura 6. Redes deconvolucionales (DN o DNN). Fuente: elaboración propia.

Las redes gráficas inversas convolucionales profundas (DCIGN) son en realidad VAE en las que el codificador es una CNN y el decodificador es una DNN. Estas redes tratan de **modelar características** como probabilidades y pueden ser utilizadas para **unir** dos objetos en una sola imagen, eliminar un objeto de una imagen, rotar un objeto en una imagen 3D, modificar la luz, etc. (Kulkarni, Whitney, Kohli, et al., 2015).

Redes generativas antagónicas

Por último, las redes generativas antagónicas (GAN o *Generative Adversarial Networks*) están formadas por **dos redes neuronales** que trabajan juntas, normalmente una combinación de una FFNN y una CNN. Una de ellas se encarga de **generar contenido** (es decir, red generativa), mientras que la otra se encarga de juzgar o **discriminar el contenido** (es decir, red discriminativa) generado por la primera. También es habitual combinar una red deconvolucional (usada como generativa) con una red neuronal convolucional (usada como discriminativa) para que generen y juzguen imágenes con el fin de **sintetizar imágenes artificiales**. Por ejemplo, para convertir vídeos de caballos en cebras o viceversa o incorporar caras de personas conocidas, como políticos a vídeos de series famosas (lo que se conoce como técnica *deepfake*).

La red discriminativa recibe como entrada los datos de formación o el contenido generado por la red generadora. Esto forma un sistema de **competición** en el que la red discriminativa es cada vez mejor para distinguir los **datos reales** de los datos generados por la red generadora y, al mismo tiempo, la red generadora es cada vez mejor para **generar datos** que la red discriminativa es incapaz de distinguir (Goodfellow, Pouget-Abadie, Mirza, et al., 2014).

Entre las **aplicaciones** de las **redes generativas antagónicas** se incluye la generación de ejemplos de conjuntos de datos de imágenes, la generación de fotografías de rostros humanos y las poses (Ma, Jia, Sun, et al., 2017), la generación de fotografías realistas o de dibujos animados, traducción de imagen a imagen,

traducción de texto a imagen, envejecimiento facial, edición de fotografías, transformación de la ropa en una imagen o predicción de vídeo, entre muchos otros.

2.6. Casos prácticos en ingeniería de software

Las redes neuronales han encontrado diversas aplicaciones en la ingeniería del *software* al mejorar y automatizar varios aspectos del **desarrollo, mantenimiento y gestión**. A continuación, se describen algunos casos prácticos de implementación de redes neuronales en este campo:

Detección de defectos y análisis de código

Las redes neuronales se utilizan para analizar el código fuente y detectar posibles defectos o **vulnerabilidades de seguridad**. Las empresas como Microsoft han integrado redes neuronales en sus herramientas de análisis estático, como Visual Studio Code Analysis, para detectar errores y **defectos de código** antes de la compilación.

Automatización de pruebas de software

Las redes neuronales pueden automatizar la generación y ejecución de casos de prueba para mejorar la **eficiencia** y **cobertura** del *testing*. Las herramientas como AppliTools utilizan redes neuronales para la **comparación visual** de interfaces de usuario y detectar automáticamente discrepancias en las pruebas de regresión visual.

Mantenimiento predictivo y gestión de incidentes

Los modelos de redes neuronales pueden predecir posibles fallos en el *software* y ayudan en la **priorización** y **resolución** de incidentes. Las empresas como IBM han implementado soluciones de mantenimiento predictivo utilizando redes neuronales para **anticipar fallos** en sistemas críticos y programar el mantenimiento preventivo.

Refactorización y optimización de código

Las redes neuronales asisten en la refactorización automática de código para mejorar su **calidad, rendimiento y mantenibilidad**. Las herramientas como Codota

y Tabnine utilizan aprendizaje profundo para sugerir refactorizaciones y optimizaciones de código basándose en buenas prácticas y patrones de código.

Desarrollo guiado por inteligencia artificial

La utilización de redes neuronales permite **asistir** a los desarrolladores durante el proceso de codificación mediante **sugerencias** y **correcciones**. GitHub Copilot basado en OpenAI Codex utiliza modelos de redes neuronales para proporcionar sugerencias de código en tiempo real que ayude a los desarrolladores a escribir código más rápido y con menos errores.

Reconocimiento y procesamiento de lenguaje natural (NLP)

La aplicación de redes neuronales permite comprender y procesar **documentos técnicos** y de requisitos en **lenguaje natural**. Las herramientas como GPT-3 se utilizan para generar automáticamente documentación técnica a partir del código fuente y comentarios, lo que facilita la mantenibilidad y comprensión del *software*.

Ejemplo guiado

Podemos entrenar un modelo de inteligencia artificial apoyándonos en redes neuronales recurrentes, como las que hemos visto anteriormente para **códigos defectuosos**.

Se considera que un código es **defectuoso** cuando no presenta errores de ningún tipo y aparentemente es correcto, pero no realiza la tarea esperada o la realiza de manera **ineficiente**.

Para ello, primero seleccionamos el **origen** y **características** de los datos. La mayoría de los analizadores de código se basan en fuentes de acceso abierto y extensas, como GitHub o StackOverflow, que, además, presentan código en múltiples lenguajes de programación (C++, Python, Java, etc.). En este ejemplo por simplicidad utilizaremos fragmentos de código sencillos en lenguaje Python creados por nosotros mismos.

Dispondremos de **4 funciones** que realizan las operaciones aritméticas básicas: suma, resta, multiplicación y división. El defecto que introduciremos en el código se presenta en el método `div`, que realizará una multiplicación. Este es un tipo de error muy común en desarrollo de *software* debido al abuso del *copy and paste* que, en ocasiones, deriva en que olvidemos **modificar** los **fragmentos** de código copiados.

```
def suma(a, b):
```

```
    return a+b
```

```
def resta(a, b):
```

```
    return a-b
```

```
def mult(a, b):
```

```
    return a*b
```

```
def div(a, b):
```

```
    return a*b
```

Una vez tenemos seleccionado (o en este caso creado) el conjunto de datos, procedemos al **procesamiento** de estos antes del entrenamiento. Puesto que el modelo que deseamos construir se engloba dentro del área del procesamiento del **lenguaje natural** (NLP), en este paso realizaremos las siguientes **tareas**:

- ▶ **Tokenización del código:** el código fuente se tokeniza para convertirlo en una representación adecuada para el análisis por redes neuronales. Se extraen tokens numéricos que representan instrucciones, funciones, variables, etc.
- ▶ **Etiquetado:** los fragmentos de código se etiquetan como defectuosos o no defectuosos basándose en el historial de versiones del proyecto.
- ▶ **División de datos:** aplicaremos una división de datos en conjuntos de entrenamiento y test. Seguiremos una distribución 80/20 para estas categorías.

Estas son solo algunas de las tareas básicas que se pueden realizar para el preprocesamiento de datos cuando se trabaja con NLP.

El siguiente fragmento de código Python muestra cómo realizar las tareas mencionadas anteriormente. Primero, agruparemos los fragmentos de código en un *array*.

```
import re

import numpy as np

from sklearn.model_selection import train_test_split

from keras.preprocessing.sequence import pad_sequences

from keras.preprocessing.text import Tokenizer

# Fragmentos de código
```

```
raw_codes = [

    "def suma(a, b):\n    return a+b",

    "def resta(a, b):\n    return a-b"

    "def mult(a, b):\n    return a*b",

    "def div(a, b):\n    return a*b",

]


# Tokenización simple basada en espacios y operadores

def tokenize_code(code):

    code = re.sub(r'[\n\t\s]+', ' ', code)

    tokens = re.findall(r'\w+|[\^\s\w] ', code)

    return tokens


# Tokenización de los fragmentos de código

code_tokens = [tokenize_code(code) for code in raw_codes]


# Crear un tokenizer para convertir los tokens en secuencias numéricas

tokenizer = Tokenizer()

tokenizer.fit_on_texts(code_tokens)

sequences = tokenizer.texts_to_sequences(code_tokens)
```

```
# Padding para que todas las secuencias tengan la misma longitud

max_length = max(len(seq) for seq in sequences)

X = pad_sequences(sequences, maxlen=max_length, padding='post')


# Etiquetas: 0 = Correcto, 1 = Defectuoso

y = np.array([0, 0, 0, 1])


# Dividimos los datos en conjuntos de entrenamiento y test

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=0)


print("Entrada al Modelo (X):\n", X)

print("Etiquetas (y):\n", y)
```

La librería `Re` permite trabajar con **cadenas de texto** y **expresiones regulares**. Mediante esta podemos separar los fragmentos de código en palabras y operadores independientes que luego se pueden tokenizar en valores numéricos.

Como se puede observar, nos hemos apoyado en las distintas librerías que nos proporciona Keras, como Tokenizer o Pad_Sequences. También empleamos la librería de Scikit-Learn para realizar de forma aleatoria la **división** de los datos en los conjuntos de entrenamiento y test. La salida de ejecución del fragmento de código anterior mostraría lo siguiente:

Entrada al Modelo (X):

```
[[ 3 15 4 1 5 2 6 7 8 1 9 2]
```

```
[ 3 10 4 1 5 2 6 7 8 1 11 2]
```

```
[ 3 12 4 1 5 2 6 7 8 1 13 2]]
```

Etiquetas (y):

```
[1 0 0]
```

Con los datos de entrada ya procesados y etiquetados debidamente, procedemos al **entrenamiento** del modelo. Como hemos comentado, utilizaremos una red neuronal recurrente basada en la **arquitectura LSTM** (*long short-term memory*).

La capa de entrada será la secuencia de tokens del código fuente. A continuación, emplearemos una capa de *embedding* que convierta los tokens en vectores de baja dimensionalidad para así capturar **relaciones semánticas**. Seguiremos con la capa LSTM que capture las **dependencias** de código a largo plazo y como capa de salida tendremos una capa densa con activación softmax para clasificar el código como **defectuoso** o **no defectuoso**. En la Figura 7 podemos ver un diagrama del modelo descrito.

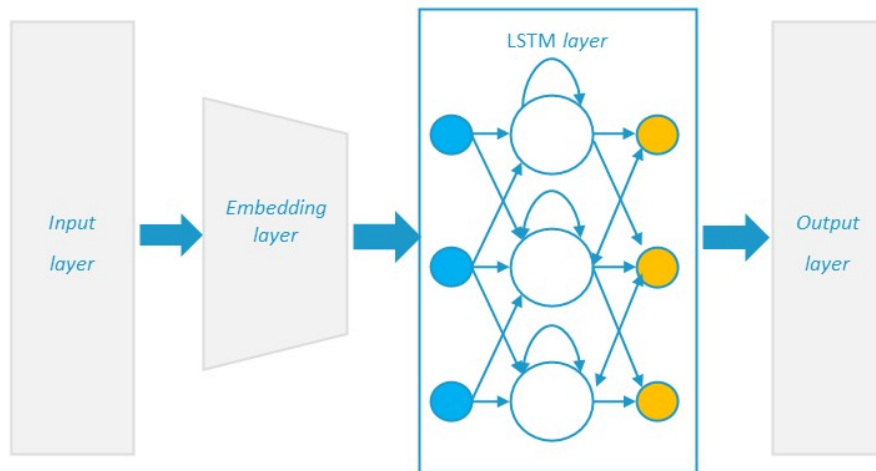


Figura 7. Modelo de red neuronal para clasificación de código. Fuente: elaboración propia.

El siguiente código Python muestra cómo se construiría dicho modelo de red neuronal mediante la librería Keras:

```
from keras.models import Sequential

from keras.layers import LSTM, Dense, Embedding

# Definición del modelo LSTM

model = Sequential()

model.add(Embedding(input_dim=16, output_dim=4))           #(1)

model.add(LSTM(units=8))                                    #(2)

model.add(Dense(1, activation='sigmoid'))                   #(3)

# Compilación del modelo

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=
```



```
['accuracy'])
```

```
# Entrenamiento del modelo
```

```
model.fit(X_train, y_train, validation_split=0.1, epochs=10)    #(4)
```

En este código se genera una capa de *embedding* (1) que reduce la **dimensionalidad** de los datos de entrada de 16 a 4. La capa LSTM (2) tendrá 8 unidades de memoria y la capa de activación (3) utilizará la función sigmoide. Se ha escogido estos valores dada la **simplicidad** de los códigos empleados. Con el uso fragmentos más extensos se deberían ajustar dichos parámetros e ir probando distintas combinaciones para mejorar el **rendimiento** del modelo.

A la tarea de ajustar parámetros en un modelo de inteligencia artificial para optimizar los resultados de entrenamiento se la conoce como **hiperparametrización**.

Durante el entrenamiento (4) se realizará una **división** entre conjuntos de entrenamiento y validación con una proporción 90/10 y solo se entrenará durante 10 iteraciones.

Con el modelo entrenado, procederemos a la evaluación de este sobre el conjunto de test. El siguiente fragmento de código muestra cómo se realizaría esta tarea en Python y Keras:

```
# Predicción en el conjunto de test
```

```
y_pred= model.predict(X_test)
```

```
print("Predicción sobre el conjunto de test:", y_pred)
```

```
print("Valor real del conjunto de test:", y_test)
```

```
# Evaluación del modelo en el conjunto de test
```

```
loss, accuracy = model.evaluate(X_test, y_test)
```

```
print(f'Precisión del modelo: {accuracy:.2f}')
```

Lo que nos arrojaría estos resultados:

Predicción sobre el conjunto de test: [[0.4489627]]

Valor real del conjunto de test: [0]

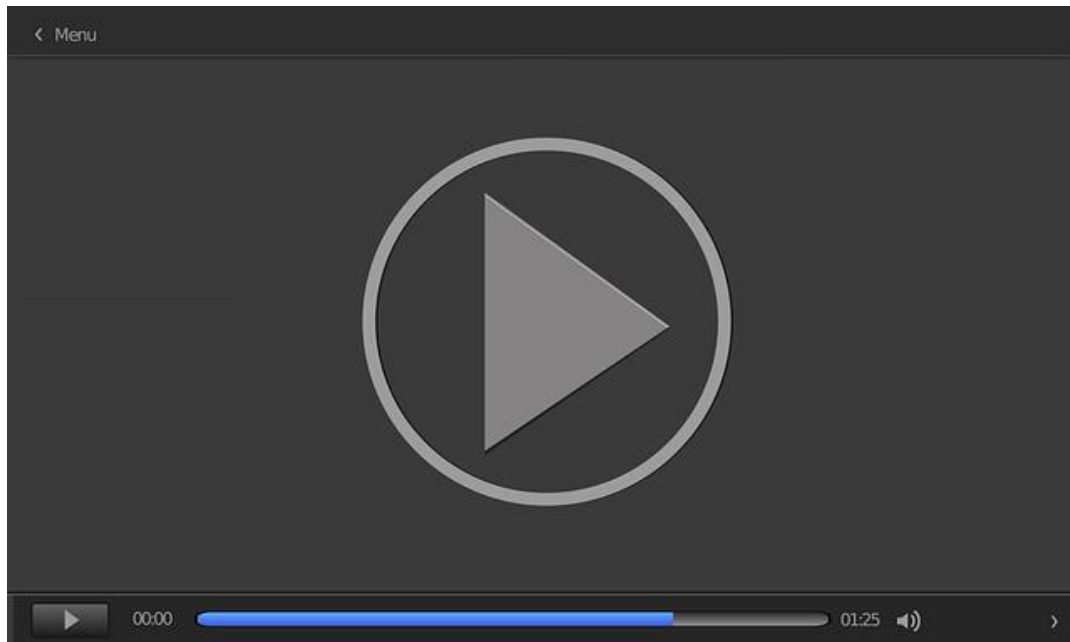
Precisión del modelo: 1.00

Una vez tenemos entrenado un modelo que nos de buenos resultados, lo podemos integrar en el *pipeline* de *CI/CD* del proyecto de modo que se **analice** el código en cada *commit* realizado. Podemos configurar también **notificaciones automáticas** para alertar a los desarrolladores cuando se detecten **defectos**.

Por último, es clave que en todo el proceso de integración de modelos de IA en un proyecto *software* se busque la **mejora continua**, por lo que el modelo debe ser **reentrenado** continuamente con datos nuevos para mejorar su precisión. Este ejemplo utiliza solo cuatro fragmentos de código sencillos, por lo que tiene un gran margen de mejora con una mayor cantidad de códigos debidamente etiquetados.

Los modelos actuales de análisis de código han sido entrenados con miles de fragmentos de código y millones de líneas de código. Solo mediante un entrenamiento intenso y exhaustivo se han logrado resultados lo suficientemente buenos que permitan ayudar a los desarrolladores *software*.

En el vídeo Herramientas para el desarrollo de redes neuronales se muestran las principales herramientas y librerías empleadas en el desarrollo de redes neuronales.



Herramientas para el desarrollo de redes neuronales

Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=3d3a8254-b835-4314-a05a-b1e300eef7cc>

2.7. Referencias bibliográficas

Alanis, A. Y., Arana-Daniel, N. y Lopez-Franco, C. (2019). *Artificial neural networks for engineering applications*. Academic Press. <https://www.sciencedirect.com/science/book/9780128182475>

Alcantarilla, P. F., Stent, S., Ros, G., Arroyo, R. y Gherardi, R. (2018). Street-view change detection with deconvolutional networks. *Autonomous Robots*, 42, 1301-1322.

Alex, V., Vaidhya, K., Thirunavukkarasu, S., Kesavadas, C. y Krishnamurthi, G. (2017). Semisupervised learning using denoising autoencoders for brain lesion detection and segmentation. *Journal of Medical Imaging*, 4(4).

Gatys, L. A., Ecker, A. S. y Bethge, M. (2016). *Image style transfer using convolutional neural networks* (pp. 2414-2423). IEEE.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. y Bengio, Y. (2014). Generative adversarial nets. En Z. Ghahramani (ed.), *Advances in neural information processing systems* (pp. 2672–2680). NIPS.

Graupe, D. (2019). *Principles of artificial neural networks: basic designs to deep learning* (4ª ed.). World Scientific Publishing Company. <https://books.google.co.uk/books?id=77uSDwAAQBAJ>

Karpathy, A. (2016). Cs231n convolutional neural networks for visual recognition. *Neural networks*, 1, 1.

Kulkarni, T. D., Whitney, W. F., Kohli, P. y Tenenbaum, J. (2015). Deep convolutional inverse graphics network. En *Advances in neural information processing systems* (pp. 2539–2547). Cornell University.

- Liu, Y. J., Cheng, S. M. y Hsueh, Y. L. (2017). eNB selection for machine type communications using reinforcement learning based markov decision process. *IEEE Transactions on Vehicular Technology*, 66(12), 11330-11338.
- Ma, L., Jia, X., Sun, Q., Schiele, B., Tuytelaars, T. y Van Gool, L. (2017). *Pose guided person image generation* (pp. 406-416) [Conferencia]. Conference on Neural Information Processing Systems, Long Beach, Estados Unidos.
- Mitchell, T. M. (1997). *Machine learning*. McGraw-Hill.
- Negnevitsky, M. (2011). *Artificial intelligence: a guide to intelligent systems* (3ª ed). Addison Wesley/Pearson.
- Pouyanfar, S., Sadiq, S., Yan, Y., Tian, H., Tao, Y., Reyes, M. P., Shyu, M.-L., Chen, S.-C. y Lyengar, S. S. (2018). A survey on deep learning: algorithms, techniques, and applications. *ACM Computing Surveys*, 51(92), 1- 36. <https://doi.org/10.1145/3234150>
- Rogers, S. y Girolami, M. (2017). *A first course in machine learning* (2ª ed.). CRC Press.
- Rosenblatt, F. (1960). Perceptron simulation experiments. *Proceedings of the IRE*, 48(3), 301-309. <https://doi.org/10.1109/JRPROC.1960.287598>
- Semeniuta, S., Severyn, A. y Barth, E. (2017). A hybrid convolutional variational autoencoder for text generation. *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 627-637.
- Shin, H. C., Roth, H. R., Gao, M., Lu, L., Xu, Z., Nogues, I. y Summers, R. M. (2016). Deep convolutional neural networks for computer-aided detection: CNN architectures, dataset characteristics and transfer learning. *IEEE Transactions on Medical Imaging*, 35(5), 1285-1298.

Tan, C. C. y Eswaran, C. (2011). Using autoencoders for mammogram compression. *Journal of Medical Systems*, 35(1), 49-58.

Zabalza, J., Ren, J., Zheng, J., Zhao, H., Qing, C., Yang, Z. y Marshall, S. (2016). Novel segmented stacked autoencoder for effective dimensionality reduction and feature extraction in hyperspectral imaging. *Neurocomputing*, 185, 1-10.

Implementación del Perceptrón

Garau, G. (2021). El Perceptrón Simple: Implementación en Python. *Blog Data Machine Learning Visualization*. <https://blog.damavis.com/el-perceptron-simple-implementacion-en-python/>

Hemos visto la teoría de qué es y cómo funciona el perceptrón. En este tutorial veremos también cómo se realiza su implementación en el lenguaje Python con ayuda de la librería de Scikit-Learn.

Redes neuronales convolucionales en Tensorflow y PyTorch

Novac, O. C., Chirodea, M. C., Novac, C. M., Bizon, N., Oproescu, M., Stan, O. P. y Gordan, C. E. (2022). Analysis of the application efficiency of TensorFlow and PyTorch in convolutional neural network. *Sensors*, 22(22), 8872. <https://www.mdpi.com/1424-8220/22/22/8872>

PyTorch y Tensorflow son dos de las principales librerías empleadas en el mundo de la inteligencia artificial para el desarrollo de redes neuronales. En este artículo de 2022 se analiza la eficiencia de ambos *frameworks* para el desarrollo de redes neuronales convolucionales

Introducción a los autoencoders

Introducción a los codificadores automáticos. (s. f.). https://www.tensorflow.org/tutorials/generative/autoencoder#import_tensorflow_and_other_libraries

Tensorflow es una de las principales librerías de código abierto para el desarrollo de redes neuronales. En este recurso web se presenta un tutorial para aprender a crear tu propio *autoencoder* con 3 ejemplos: *autoencoder* básico para comprimir/descomprimir imágenes, *autoencoder* para la eliminación de ruido en imágenes y *autoencoder* para la detección de anomalías en conjuntos de datos.

1. ¿Cuál de las siguientes funciones se utiliza para calcular la salida de una neurona artificial?
 - A. Función coseno.
 - B. Función signo.
 - C. Función logarítmica.
 - D. Función tangente.

2. ¿Qué tipo de red neuronal es el perceptrón?
 - A. Una red recurrente.
 - B. Una red convolucional.
 - C. Una red de una sola capa.
 - D. Una red de *autoencoders*.

3. ¿Cuál es el objetivo del aprendizaje en una red neuronal simple?
 - A. Maximizar el error entre la salida obtenida y la esperada.
 - B. Seleccionar los pesos que mejor se ajusten a las entradas y salidas definidas *a priori*.
 - C. Utilizar funciones no supervisadas para ajustar los pesos.
 - D. Minimizar la cantidad de capas en la red.

4. ¿Qué nombre recibe la diferencia entre la salida obtenida y la salida esperada en una red neuronal?
 - A. Gradiente.
 - B. Error.
 - C. Peso.
 - D. Umbral.

5. ¿Qué valor toma la tasa de aprendizaje α en el ajuste de pesos de una red neuronal?
 - A. Entre 0 y 1.
 - B. Entre -1 y 1.
 - C. Mayor a 1.
 - D. Menor a 0.

6. ¿Cuál es la principal característica de las redes neuronales recurrentes?
 - A. Sus salidas alimentan las entradas formando un bucle.
 - B. Tienen múltiples capas ocultas.
 - C. Utilizan la retropropagación del error.
 - D. Se basan en arquitecturas convolucionales.

7. ¿Qué problema presentan las redes de Hopfield?
 - A. No pueden almacenar memorias fundamentales.
 - B. Alcanzar un estado estable no siempre corresponde a una memoria fundamental.
 - C. Necesitan pocas neuronas para almacenar mucha información.
 - D. Son adecuadas para la asociación de informaciones diferentes.

8. ¿Qué tipo de redes se utilizan para el reconocimiento de imágenes y patrones?
 - A. Redes *autoencoders*.
 - B. Redes neuronales convolucionales.
 - C. Redes recurrentes.
 - D. Redes Hopfield.

9. ¿Qué caracteriza a las redes *autoencoders*?
- A. Su capacidad para clasificar datos.
 - B. Su estructura de capas completamente conectadas.
 - C. Su uso en reducción de dimensionalidad y eliminación de ruido.
 - D. Su aplicación en redes convolucionales.
10. ¿Qué es la función de activación en una red neuronal?
- A. Un método para entrenar la red.
 - B. Un algoritmo para ajustar los pesos.
 - C. Una función que introduce la no linealidad en el modelo.
 - D. Una técnica de optimización.
11. ¿Qué tipo de función de activación es la función sigmoide?
- A. Lineal.
 - B. No lineal.
 - C. Exponencial.
 - D. Logarítmica.
12. ¿En qué consisten las redes prealimentadas (*feedforward*)?
- A. En tener conexiones que forman bucles.
 - B. En procesar secuencias temporales de datos.
 - C. En tener conexiones unidireccionales de las entradas a las salidas.
 - D. En utilizar *autoencoders*.

- 13.** ¿Cuál es la ventaja principal de las redes convolucionales sobre las redes tradicionales?
- A. Tienen menos capas.
 - B. Requieren menos datos para entrenar.
 - C. Pueden captar características espaciales y jerárquicas.
 - D. No necesitan función de activación.
- 14.** ¿Cuál es una de las características principales de las redes de Hopfield?
- A. Son redes recurrentes con salidas que alimentan las entradas.
 - B. Utilizan retropropagación para ajustar los pesos.
 - C. Se utilizan principalmente para tareas de clasificación.
 - D. Tienen una estructura jerárquica.
- 15.** ¿Qué define la capacidad de una red de Hopfield para almacenar memorias?
- A. El número de capas ocultas.
 - B. La cantidad de patrones de entrada.
 - C. El tamaño de la red (número de neuronas).
 - D. La tasa de aprendizaje.
- 16.** ¿Cuál de las siguientes afirmaciones sobre las redes autoasociativas es incorrecta?
- A. Pueden recordar patrones completos a partir de fragmentos de estos.
 - B. Se basan en el principio de autoaprendizaje.
 - C. Necesitan grandes conjuntos de datos para entrenarse.
 - D. Son utilizadas para tareas de memoria asociativa.

17. ¿Qué técnica se utiliza frecuentemente para ajustar los pesos en las redes neuronales?
- A. Algoritmo genético.
 - B. Algoritmo de optimización de enjambre de partículas.
 - C. Retropropagación del error.
 - D. Análisis de componentes principales.
18. ¿Qué característica de las redes convolucionales les permite procesar imágenes de manera eficiente?
- A. La función sigmoide.
 - B. El uso de capas de *pooling*.
 - C. La estructura completamente conectada.
 - D. La tasa de aprendizaje variable.
19. ¿Cuál es la función principal de las redes neuronales recurrentes (RNN)?
- A. Procesar datos de entrada en lotes.
 - B. Captar dependencias temporales en secuencias de datos.
 - C. Realizar clasificación de imágenes.
 - D. Reducir la dimensionalidad de datos.
20. ¿Qué es una red generativa adversarial (GAN)?
- A. Un tipo de red que clasifica los datos en múltiples categorías.
 - B. Un tipo de red que genera datos nuevos a partir de datos existentes mediante dos redes que compiten.
 - C. Un tipo de red que realiza tareas de segmentación de imágenes.
 - D. Una red que se especializa en la compresión de datos.

Fundamentos de Inteligencia Artificial para Ingenieros de
Software

Tema 3. Procesamiento del lenguaje natural (NLP) y su aplicación en software

Índice

Esquema

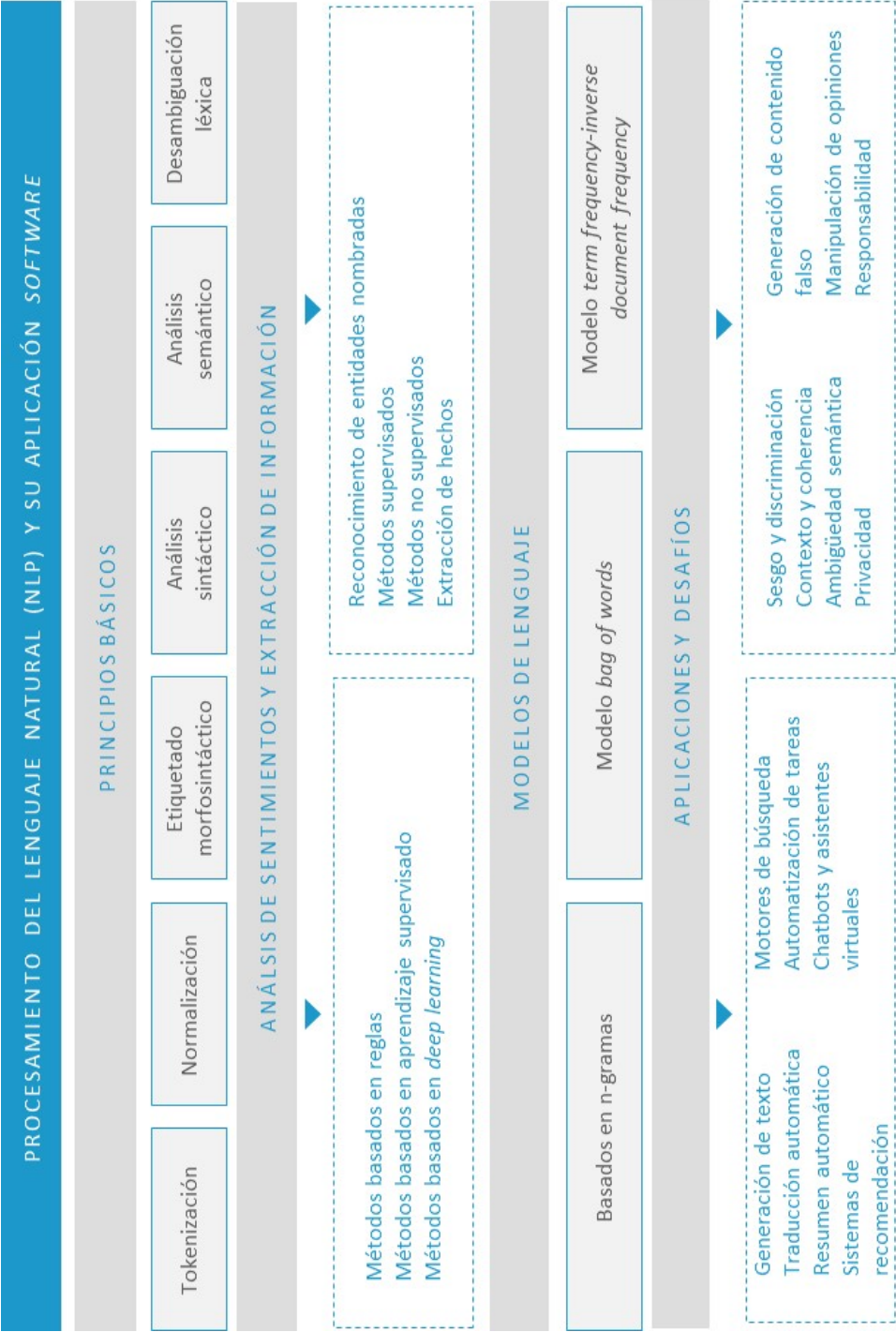
Ideas clave

- 3.1. Introducción y objetivos
- 3.2 Principios básicos de NLP
- 3.3. Análisis de sentimientos y extracción de información
- 3.4. Modelos de lenguaje: aplicaciones y desafíos
- 3.5. Implementación práctica en el desarrollo de software
- 3.6. Consideraciones éticas en el procesamiento del lenguaje natural
- 3.7. Referencias bibliográficas

A fondo

- Modelos de lenguaje pre-entrenados: revisión y avances
- Incrustación y representación vectorial de palabras
- Mejores prácticas éticas en el procesamiento del lenguaje natural

Test



3.1. Introducción y objetivos

El procesamiento del lenguaje y del habla ha sido tratado históricamente de forma muy diferente en la informática, la ingeniería, la lingüística, la psicología o la ciencia cognitiva. Hoy en día se concibe el procesamiento del lenguaje natural como un área que abarca varios campos diferentes y diversos, pero superpuestos.

Por ello, el procesamiento del lenguaje natural es un campo interdisciplinario que une a informáticos, ingenieros electrónicos y de telecomunicaciones con lingüistas, sociólogos y psicólogos.

El reconocimiento de la voz, que incluye tareas del procesamiento de la señal, se ha tratado tradicionalmente en la ingeniería electrónica y de telecomunicaciones. El análisis sintáctico y la interpretación semántica de las palabras y frases son áreas tradicionales del procesamiento del lenguaje natural que se estudia en el campo de la informática. La morfología, la fonología y la pragmática son tareas de investigación en la lingüística computacional. Psicolingüistas y sociolingüistas estudian respectivamente los mecanismos cognitivos para la adquisición del lenguaje y cómo la sociedad influye en el uso de la lengua.

El ancho espectro que abarca el campo del procesamiento del lenguaje natural hace que se conozca con diferentes nombres debido a las diferentes vertientes involucradas. Algunos de estos nombres, que provienen de estas diferentes facetas, serían procesamiento del lenguaje y del habla, tecnología del lenguaje, procesamiento del lenguaje natural, lingüística computacional o reconocimiento y síntesis del habla.

En la inteligencia artificial el procesamiento del lenguaje natural es un campo que tiene como objetivo que las máquinas sean capaces de realizar tareas que involucren el lenguaje humano. Algunas de las tareas que debe realizar una máquina para ser

capaz de procesar el lenguaje natural incluyen funcionalidades tales como la de habilitar a la máquina de habilidades para comunicarse con personas, la de mejorar la comunicación entre humanos o, simplemente, la de procesar un texto o el habla.

Los objetivos de este tema son:

- ▶ Entender qué es y cómo funciona el procesamiento del lenguaje natural.
- ▶ Ver las principales técnicas para el análisis de sentimientos en textos.
- ▶ Explorar los distintos métodos empleados para la extracción de información relevante en textos.
- ▶ Analizar los diferentes modelos del lenguaje, sus fortalezas y debilidades.
- ▶ Aprender los pasos por seguir para incorporar el procesamiento del lenguaje natural en el desarrollo *software*.
- ▶ Reflexionar sobre las implicaciones del desarrollo y aplicación de los modelos de procesamiento del lenguaje natural.

3.2 Principios básicos de NLP

El procesamiento del **lenguaje natural** se basa en la idea de que el **lenguaje humano** puede ser procesado por máquinas de manera similar a como los humanos lo hacen, aunque a través de diferentes mecanismos. El NLP se apoya en diversos principios y técnicas que permiten a las máquinas **procesar** y **entender** el lenguaje humano.

Tokenización

La tokenización es la principal técnica aplicada en el procesamiento del lenguaje natural. Esta es el proceso de **dividir** un **texto** en palabras, frases, oraciones o símbolos denominados *tokens*. Estos *tokens* son las **unidades básicas** que se procesan en los algoritmos de NLP (Manning, Surdeanu, Bauer, et al., 2014).

La tokenización puede ser aplicada de distintas formas según el tipo de *token* que se desee procesar. Las **principales técnicas** de tokenización son:

- ▶ **Tokenización por palabras y espacios:** esta técnica divide un texto en palabras individuales. Es la forma más común de tokenización y se utiliza cuando se quiere analizar el texto a nivel de palabra. Por ejemplo: «El perro ladra» se tokenizaría como ["El", "perro", "ladra"].
- ▶ **Tokenización por subpalabras:** se emplea cuando se necesita descomponer las palabras en unidades más pequeñas que pueden capturar estructuras morfológicas, como prefijos y sufijos. Por ejemplo: la palabra 'puntualmente' podría tokenizarse como ["puntual", "mente"] .
- ▶ **Tokenización por caracteres:** el texto se descompone a nivel de caracteres. Esta técnica es útil cuando se trabaja con idiomas con morfología compleja o cuando se necesita capturar errores ortográficos. Por ejemplo: 'Hola' se tokenizaría como ["H", "o", "l", "a"] .

- ▶ **Tokenización por n-gramas:** los n-gramas son secuencias de n palabras o caracteres que aparecen consecutivamente en el texto. Se utilizan comúnmente para análisis de texto en tareas, como la detección de *spam*. Por ejemplo: para la frase «¡Felicidades! Has ganado la lotería nacional», la tokenización por trigramas sería ["¡Felicidades! Has ganado", "Has ganado la", "ganado la lotería", "la lotería nacional"] .
- ▶ **Tokenización por frases:** es el proceso de división del texto en frases individuales. Para ello, se emplean habitualmente los signos de puntuación, como puntos (.).

El siguiente fragmento de código nos muestra cómo aplicar tokenización por palabras y frases en Python mediante la librería nltk.

```
import nltk

from nltk.tokenize import word_tokenize, sent_tokenize

# Descargar recursos necesarios de NLTK

nltk.download('punkt')

# Fragmento de texto

texto = """

En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivía un
hidalgo de los de lanza en astillero, adarga antigua, rocín flaco y galgo corredor. Una olla de algo mas
vaca que carnero, salpicón las más noches, duelos y quebrantos los sábados, lantejas los viernes,
algún palomino de añadidura los domingos, consumían las tres partes de su hacienda.

"""
```

```
# Tokenización por palabras
```

```
tokens_palabras = word_tokenize(texto)
```

```
# Tokenización por oraciones
```

```
tokens_oraciones = sent_tokenize(texto)
```

```
# Mostrar los resultados
```

```
print("Tokenización por palabras:", tokens_palabras)
```

```
print("Tokenización por oraciones:", tokens_oraciones)
```

Cuya salida mostraría en **tokenización por palabras**:

```
['En', 'un', 'lugar', 'de', 'la', 'Mancha', ',', 'de', 'cuyo', 'nombre', 'no', 'quiero', 'acordarme', ',', 'no', 'ha',  
'mucho', 'tiempo', 'que', 'vivía', 'un', 'hidalgo', 'de', 'los', 'de', 'lanza', 'en', 'astillero', ',', 'adarga', 'antigua',  
,', 'rocín', 'flaco', 'y', 'galgo', 'corredor', '.', 'Una', 'olla', 'de', 'algo', 'mas', 'vaca', 'que', 'carnero', ',',  
'salpicón', 'las', 'más', 'noches', ',', 'duelos', 'y', 'quebrantos', 'los', 'sábados', ',', 'lantejas', 'los', 'viernes',  
,', 'algún', 'palomino', 'de', 'añadidura', 'los', 'domingos', ',', 'consumían', 'las', 'tres', 'partes', 'de', 'su',  
'hacienda', '.']
```

Y en la **tokenización por frases**:

```
['En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivía un  
hidalgo de los de lanza en astillero, adarga antigua, rocín flaco y galgo corredor.', 'Una olla de algo  
mas vaca que carnero, salpicón las más noches, duelos y quebrantos los sábados, lantejas los  
viernes, algún palomino de añadidura los domingos, consumían las tres partes de su hacienda. ']
```

Normalización

Es la técnica mediante la cual se **estandarizan** los **datos de entrada** para los modelos: conversión de texto a minúsculas, la eliminación de puntuaciones y la corrección ortográfica (Jurafsky y Martin, 2000). Dependiendo del idioma a emplear se deben valorar diferentes aspectos: la relevancia o no de palabras en **letra capital**, en las **mayúsculas** o la presencia de **acentos**.

La mayoría de las aplicaciones y modelos de NLP utilizan el **inglés** como **idioma base** con el que han sido entrenados, esto hace que se presenten ciertos problemas cuando se trata de emplear dichos modelos o procesos de entrenamiento en otras lenguas debido a que el inglés carece de acentos u otros **caracteres** que si están presentes en **otros idiomas**.

En la lengua castellana tenemos un fenómeno conocido como **parónimo tónico** que indica palabras que tienen una **semejanza** en su **sonido**, pero no en su sílaba tónica.

Por ejemplo, depósito/depositó, cobre/cobré, notaria/notaría, género/genero/generó.

En el proceso de normalización de las palabras deben tenerse en cuenta las **particularidades** de cada idioma para evitar que en este paso se pierda el **significado** de las palabras y la **esencia** de los textos. Por ejemplo, en la frase «Él vino de Francia para cenar» el acento es crítico, pues no solo afecta a la **interpretación** del pronombre 'él', sino que la palabra 'vino' puede ser vista como un verbo o un sustantivo («Él vino de Francia para cenar» → «El vino de Francia para cenar»).

Entre las principales **técnicas de normalización** encontramos:

- **Eliminación de stopwords:** las *stopwords* son palabras comunes en un idioma que generalmente no aportan un valor significativo en el análisis de texto (Manning, Surdeanu, Bauer, et al., 2014). Estas palabras incluyen artículos, preposiciones,

conjunciones y pronombres, como 'el', 'de', 'y', 'pero', etc. En muchas tareas de procesamiento del lenguaje natural estas palabras se eliminan para reducir el ruido y centrarse en las palabras más relevantes que realmente aportan al significado del texto.

- ▶ **Conversión a minúsculas (*lowercasing*):** convertir todo el texto a minúsculas es una técnica simple, pero efectiva, para reducir la variabilidad en los datos. Esto es especialmente útil en idiomas donde las mayúsculas y minúsculas no cambian el significado de las palabras. Por ejemplo: «Hola Mundo» → «hola mundo».
- ▶ **Eliminación de caracteres especiales y puntuación:** la eliminación de caracteres no alfabéticos, como signos de puntuación y números, ayuda a reducir el ruido en el texto. Estos caracteres a menudo no son útiles para el análisis semántico. Ejemplo: «¡Hola, mundo!» → «Hola mundo».
- ▶ **Expansión de contracciones:** en muchos textos informales, como redes sociales o chats, las contracciones son comunes. La expansión de contracciones es la técnica de convertirlas a su forma completa. Por ejemplo: 'tmb' → 'también'.
- ▶ **Corrección ortográfica:** consiste en corregir errores ortográficos antes de continuar con el análisis del texto. Esto es importante para evitar que los errores de escritura afecten la calidad del procesamiento. Por ejemplo: 'kasa' → 'casa'.
- ▶ **Lematización:** la lematización es el proceso de reducir las palabras a su forma base o lema. Por ejemplo, las palabras 'corriendo' y 'corrió' se reducen a su lema 'correr'. La lematización considera el contexto gramatical y utiliza diccionarios para encontrar la forma base correcta. Ejemplo: 'corriendo' → 'correr'.
- ▶ **Derivación (*stemming*):** la derivación es una técnica más agresiva que la lematización, ya que corta los sufijos de las palabras para reducirlas a una raíz común. Sin embargo, este proceso puede resultar en formas que no son palabras reales. Ejemplo: 'jugar', 'jugando', 'jugador' → 'jug'.

Tanto la lematización como la derivación son técnicas muy útiles cuando se quiere **agrupar palabras** con el mismo significado, por ejemplo, conjugaciones verbales, o para **traducciones** de texto entre idiomas con una misma base, como el castellano/catalán/gallego/italiano/etc.

Para realizar la **normalización** del texto del ejemplo anterior podemos emplear el siguiente **código**:

```
from nltk.corpus import stopwords

...

...

# Lista de stopwords en español

stop_words = set(stopwords.words('spanish'))

...

# Eliminación de stopwords

tokens_filtrados = [word for word in tokens_palabras if word.lower() not in stop_words]

...

# Mostrar los resultados

print("Texto sin stopwords:", tokens_filtrados)

...

# Conversión a minúsculas

texto = texto.lower()
```

```
# Eliminación de puntuación
```

```
texto = re.sub(r'^\w\s', "", texto)
```

```
...
```

```
...
```

```
# Lemmatización
```

```
lemmatizer = WordNetLemmatizer()
```

```
tokens_lemmatized = [lemmatizer.lemmatize(token) for token in tokens_filtrados]
```

```
# Mostrar los resultados
```

```
print("Texto lematizado:", tokens_lemmatized)
```

Etiquetado morfosintáctico

El etiquetado morfosintáctico, llamado *POS tagging* (*part-of-speech tagging* en inglés), es el proceso para **identificar** las diferentes **partes** de la **oración** que consiste en asignar una etiqueta (*tag*) sobre la **categoría gramatical** a cada una de las palabras de un texto de entrada.

La **entrada** del algoritmo de etiquetado morfosintáctico es una **secuencia de palabras**, y la **salida** del algoritmo es una **secuencia de pares** formada por la palabra y la correspondiente etiqueta que indica la categoría gramatical a la que pertenece dicha palabra.

Hoy en día, la mayoría de los algoritmos de procesamiento del lenguaje natural que procesan palabras en inglés utilizan el Penn Treebank (Marcus, Santorini y Marcinkiewicz, 1993), como muestra la Figura 1.

Tag	Description	Example	Tag	Description	Example
CC	coordin. conjunction	<i>and, but, or</i>	SYM	symbol	<i>+, %, &</i>
CD	cardinal number	<i>one, two</i>	TO	"to"	<i>to</i>
DT	determiner	<i>a, the</i>	UH	interjection	<i>ah, oops</i>
EX	existential 'there'	<i>there</i>	VB	verb base form	<i>eat</i>
FW	foreign word	<i>mea culpa</i>	VBD	verb past tense	<i>ate</i>
IN	preposition/sub-conj	<i>of, in, by</i>	VBG	verb gerund	<i>eating</i>
JJ	adjective	<i>yellow</i>	VBN	verb past participle	<i>eaten</i>
JJR	adj., comparative	<i>bigger</i>	VBP	verb non-3sg pres	<i>eat</i>
JJS	adj., superlative	<i>wildest</i>	VBZ	verb 3sg pres	<i>eats</i>
LS	list item marker	<i>1, 2, One</i>	WDT	wh-determiner	<i>which, that</i>
MD	modal	<i>can, should</i>	WP	wh-pronoun	<i>what, who</i>
NN	noun, sing. or mass	<i>llama</i>	WPS	possessive wh-	<i>whose</i>
NNS	noun, plural	<i>llamas</i>	WRB	wh-adverb	<i>how, where</i>
NNP	proper noun, sing.	<i>IBM</i>	\$	dollar sign	<i>\$</i>
NNPS	proper noun, plural	<i>Carolinas</i>	#	pound sign	<i>#</i>
PDT	predeterminer	<i>all, both</i>	"	left quote	<i>' or "</i>
POS	possessive ending	<i>'s</i>	"	right quote	<i>' or "</i>
PRP	personal pronoun	<i>I, you, he</i>	(	left parenthesis	<i>[, (, {, <</i>
PRPS	possessive pronoun	<i>your, one's</i>	)	right parenthesis	<i>],), }, ></i>
RB	adverb	<i>quickly, never</i>	,	comma	<i>,</i>
RBR	adverb, comparative	<i>faster</i>	.	sentence-final punc	<i>. ! ?</i>
RBS	adverb, superlative	<i>fastest</i>	:	mid-sentence punc	<i>: ; ... --</i>
RP	particle	<i>up, off</i>			

Figura 1. Etiquetas para las categorías gramaticales en Penn Treebank. Fuente: Jurafsky y Martin, 2000.

Por ejemplo, si el **etiquetador morfosintáctico** analiza las frases:

"Bebo un vaso del vino tinto."

"Vino de un lugar lejano."

Una posible salida siguiendo las categorías gramaticales definidas en el Penn Treebank sería:

bebo/VVP un/DT vaso/NN de/IN el/DT vino/NN tinto/JJ

vino/VVD de/IN un/DT lugar/NN lejano/JJ

En el primer ejemplo, la palabra «vino» pertenece a la categoría gramatical de los sustantivos o nombres (NN). Mientras que, en el segundo ejemplo, pertenece a la categoría gramatical de los verbos (VBZ). Para realizar el POS *tagging* se emplean diversas técnicas. Las más habituales son:

- ▶ **Modelos ocultos de Markov (HMM):** esta técnica consiste en construir un modelo de lenguaje estadístico que se utiliza para obtener, a partir de una frase de entrada, la secuencia de etiquetas gramaticales que tiene mayor probabilidad.
- ▶ **Aprendizaje automático:** los modelos más vanguardistas de etiquetado morfosintáctico utilizan diversas técnicas de aprendizaje automático tanto supervisado como no supervisado.

El etiquetado morfosintáctico nos puede ayudar a mejorar nuestros modelos de PNL en tareas como la **traducción**. Por ejemplo, *water* en inglés puede traducirse como ‘agua’ (sustantivo) o ‘regar’(verbo) según el contexto en que aparezcan.

‘agua’ (sustantivo) o ‘regar’(verbo) según el contexto en que aparezca.

Además de poder obtener el POS *tag* de una palabra usando algoritmos estadísticos o técnicas de aprendizaje automático, también se puede identificar a qué **entidad** nombrada hacen referencia los sustantivos (a una persona, a una organización o a una ubicación, por ejemplo). Esta tarea se denomina **NER** (*named-entity recognition*), y con ella se suelen identificar dentro de un texto **categorías**, como las siguientes (a modo de ejemplo, ya que se pueden usar más categorías):

- ▶ PER: categoría de **personas**, como, por ejemplo, el nombre de alguien (ej.: Frodo Baggins).
- ▶ GPE: categoría para identificar **países**, ciudades o estados (ej.: Madrid).
- ▶ LOC: categoría para identificar **ubicaciones concretas** (ej.: Vesubio).
- ▶ ORG: categoría para identificar **organizaciones** o empresas (ej.: Microsoft).

- ▶ **MONEY**: categoría para identificar referencias a **dinero** (ej.: \$5).
- ▶ **DATE**: categoría para identificar **fechas** (ej.: 05/02/2021 o jueves).

De igual manera que ocurre con los algoritmos de POS *tagging*, la identificación de las NER se puede realizar mediante:

- ▶ **Diccionarios** donde se tengan ya identificadas las palabras (o combinaciones de palabras) junto con su NE.
- ▶ **Sistemas basados en reglas** que identifiquen las NE con base en ciertos patrones.
- ▶ **Modelos estadísticos** o de **aprendizaje automático** que con base en un entrenamiento previo y en una serie de variables que modelen el contexto de las palabras, puedan predecir qué *token* o *tokens* hacen referencia a una NER.

En la práctica estas relaciones se **tokenizan** para realizar el **entrenamiento** del modelo. Por ejemplo, la frase «Domingo vino un domingo de Francia donde bebió un buen vino» tendría un etiquetado y tokenización como el que muestra la Tabla 1.

	POS tag	NER	Token
Domingo	NNP	PER	1
vino	VVD		2
un	DT		3
domingo	NN	DATE	4
de	IN		5
Francia	NNP	GPE	6
donde	RB		7
bebió	VVD		8
un	DT		3
buen	JJ		9
vino	NN		10

Tabla 1. Ejemplo de etiquetado morfosintáctico y tokenización. Fuente: elaboración propia.

Obsérvese que las palabras como ‘domingo’ o ‘vino’ tienen **tokens distintos** gracias al etiquetado morfosintáctico. Sin este paso se les habría asignado el mismo *token*, lo que daría lugar a **imprecisiones** en el modelo generado.

Análisis sintáctico

En el análisis sintáctico se determinan las **relaciones estructurales** entre palabras y es muy relevante en el procesamiento del lenguaje natural no solo por la **información sintáctica** que aporta, sino también porque es un paso esencial para la posterior identificación de las **relaciones semánticas** de las oraciones.

Por lo general, el resultado del análisis sintáctico es construir el **árbol sintáctico**. Sus nodos son los constituyentes sintácticos y las hojas las palabras que componen

la oración analizada. La **estructura jerárquica** del árbol permite observar que un constituyente está formado por una o varias palabras y por otros constituyentes.

- ▶ **Chunking**: es una forma más sencilla de análisis sintáctico que agrupa palabras en frases o sintagmas sin construir un árbol completo. Es útil para las tareas donde la comprensión completa de la estructura de la oración no es necesaria. Por ejemplo: «El perro grande come» se puede dividir en [SN: "El perro grande"], [SV: "come"] .
- ▶ **Árboles de dependencias (*dependency parsing*)**: esta técnica analiza la oración y establece relaciones entre las palabras en forma de dependencias. Cada palabra tiene una cabeza a la que está subordinada y crea una estructura en forma de árbol. Por ejemplo, en la frase «El perro come huesos», 'perro' es el sujeto que depende del verbo 'come' y 'huesos' es el objeto directo.
- ▶ **Árboles de constituyentes (*constituency parsing*)**: aquí, la oración se descompone en unidades más pequeñas llamadas constituyentes que pueden ser sintagmas nominales, verbales, etc. El resultado es un árbol de constituyentes que muestra cómo las palabras se agrupan en frases. Por ejemplo, la frase «El perro grande» se descompone en un sintagma nominal (SN) formado por el determinante 'El' y el sintagma adjetival 'perro grande'.

El texto de los ejemplos anteriores se podría analizar **sintácticamente** con este código:

```
from nltk import pos_tag

from nltk.chunk import RegexpParser

...

...

#Etiquetado de partes del discurso

tags = pos_tag(tokens_lemmatized)
```

```
# Definición de gramática para chunking

gramatica = "SN: {<DT>?<JJ><NN>}"

chunker = RegexpParser(gramatica)


# Aplicación de chunking

resultado = chunker.parse(tags)

print("Resultado del chunking:", resultado)
```

Análisis semántico

El objetivo del análisis semántico es representar el **significado** de la oración y requiere de múltiples fuentes de conocimiento, como el conocimiento sobre los significados de las palabras. Entonces, el análisis semántico dirigido por la sintaxis se basa en el **principio de composición**.

La idea fundamental del principio de composición es que el significado de una oración se construye a partir del **significado** de sus **partes**.

El significado de una oración no se basa solamente en el significado de las palabras que la componen, sino que también en el **orden**, la **agrupación** de palabras y en las **relaciones** entre las palabras en la frase. Es decir, que el significado de una frase está también basado en su estructura sintáctica.

Por lo tanto, en el análisis semántico dirigido por la sintaxis, la composición del significado de la oración se hace a partir de la estructura sintáctica de la frase.

La Figura 2 muestra una posible estructura del proceso de análisis semántico dirigido por la sintaxis. Se observa que el resultado del análisis sintáctico de una oración sirve como entrada para el **análisis semántico**. Así, un analizador sintáctico pasa primero por una frase que extraiga la información sintáctica de las relaciones estructurales entre palabras. La estructura sintáctica extraída, que se suele representar como un árbol sintáctico, sirve para alimentar el **analizador semántico**, que representará el significado de la oración.

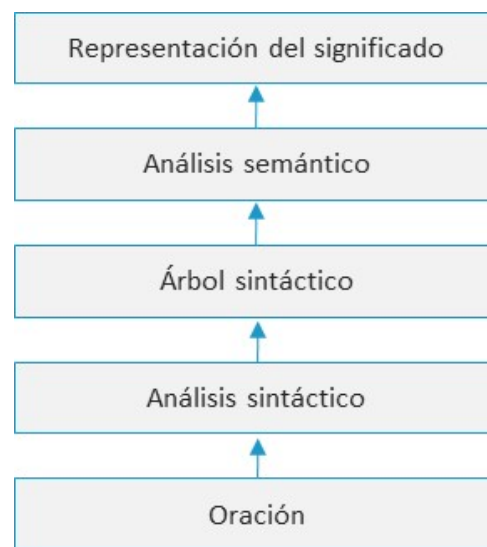


Figura 2. Estructura del proceso de análisis semántico dirigido por la sintaxis. Fuente: elaboración propia.

Como resultado del análisis semántico se obtiene la **representación del significado** de la oración, y esta representación se modela utilizando un lenguaje formal, como puede ser la lógica de primer orden o la lógica descriptiva. El analizador semántico necesitará acceder a una **descripción formal** del conocimiento sobre el significado de las palabras con las que trabaja. Entre las principales **técnicas** de análisis semántico encontramos:

- ▶ **Redes semánticas:** representan el significado de las palabras y las relaciones entre ellas en una red. WordNet es un ejemplo común donde las palabras están conectadas por relaciones semánticas, como sinonimia, antonimia, etc. Por ejemplo, 'Mujer' y 'Fémina' están conectadas como sinónimos en WordNet.
- ▶ **Análisis de roles semánticos (*semantic role labeling* o SRL):** asigna roles semánticos a las palabras en una oración e identifica qué palabras cumplen funciones, como sujeto, objeto, acción, etc. Por ejemplo, en la frase «María le dio el libro a Juan» SRL identifica a 'María' como el agente, 'libro' como el tema y 'Juan' como el destinatario.
- ▶ **Representaciones semánticas distribuidas:** modelos como Word2Vec, GloVe y BERT capturan significados semánticos de palabras en vectores de alta dimensionalidad. Estos modelos se entrenan sobre grandes corpus de texto y aprenden las relaciones semánticas a partir de coocurrencias. Por ejemplo: Word2Vec puede representar rey y reina de manera similar al capturar la relación de género entre ellos.

En el caso del análisis semántico podemos emplear el algoritmo Lesk para la **desambiguación** de palabras sobre el texto del ejemplo anterior, como muestra el siguiente código:

```
from nltk.wsd import lesk

...

...

# Aplicar Lesk para desambiguar cada palabra

for token in tokens_lemmatized:

    synset = lesk(tokens_lemmatized, token, lang='spa') # Lesk para
    desambiguación en español

    if synset:

        print(f"Palabra: {token}")

        print(f"Significado: {synset.definition()}\n")
```

Desambiguación léxica

La desambiguación del sentido de las palabras es la tarea de seleccionar el **sentido correcto** para una palabra. Los algoritmos de desambiguación del sentido toman como entrada una palabra en su **contexto** y una lista de posibles significados de esa palabra y devuelven como salida el sentido correcto para ese **uso concreto** de la palabra. Existen diferentes opciones para implementar un algoritmo de desambiguación del sentido de las palabras:

- ▶ **Aprendizaje supervisado:** estos algoritmos de desambiguación requieren tener un corpus de palabras etiquetadas con sus sentidos correctos para poder entrenar el clasificador. Por eso, tener acceso a estos datos etiquetados puede ser complejo y, aunque los algoritmos de desambiguación basados en aprendizaje supervisado sean los que proporcionan mejores resultados, a veces no se usan por el elevado coste asociado a la obtención de los datos etiquetados.
- ▶ **Basados en conocimiento:** si no se dispone de un corpus etiquetado, una alternativa es utilizar diccionarios, tesauros u otras bases de conocimiento para realizar un entrenamiento indirecto al aplicar algoritmos de aprendizaje supervisado débil.
- ▶ **Aprendizaje semisupervisado o *bootstrapping*.** estos algoritmos de desambiguación no requieren de grandes recursos lingüísticos generados a mano ni de un gran conjunto de datos de entrenamiento ni de un gran diccionario, sino que para funcionar es suficiente con tener un pequeño conjunto de datos de entrenamiento etiquetados a mano.
- ▶ **Aprendizaje no supervisado:** los algoritmos basados en este tipo de aprendizaje realizan la desambiguación sin utilizar las definiciones de los sentidos de las palabras por humanos. El conjunto de sentidos de cada palabra se crea automáticamente a partir de las instancias de la palabra disponibles en los datos de entrenamiento. Esta tarea se llama también inducción del sentido de las palabras.

Una de las relaciones entre palabras que más se usa en el procesamiento del lenguaje natural es la **sinonimia**. La sinonimia es una **relación binaria** entre dos palabras, es decir, las palabras son sinónimas o no lo son. Sin embargo, se puede utilizar una métrica más relajada que permita calcular la similitud entre palabras llamada **distancia semántica**.

Dos palabras son más similares si comparten más **características** de su **significado**, es decir, si son casi sinónimos. Por el contrario, dos palabras son menos similares o tienen mayor distancia semántica si comparten menos elementos de su significado.

Por ejemplo, las palabras 'coche' y 'gasolina' están estrechamente relacionadas, pero no son similares; mientras que las palabras 'coche' y 'bicicleta', además de estar relacionadas, son más similares. De hecho, los **antónimos** son palabras que están **relacionadas**, pero que no son similares en absoluto. Para medir la similitud entre palabras o, mejor dicho, la relación entre sentidos de las palabras, existen dos tipos de **algoritmos**:

- ▶ Los primeros calculan la similitud entre palabras utilizando la **estructura** de un **tesauro**.
- ▶ Los segundos calculan la similitud entre palabras aplicando métodos de **distribución** y encontrando directamente palabras que tienen **distribuciones similares** en un corpus.

3.3. Análisis de sentimientos y extracción de información

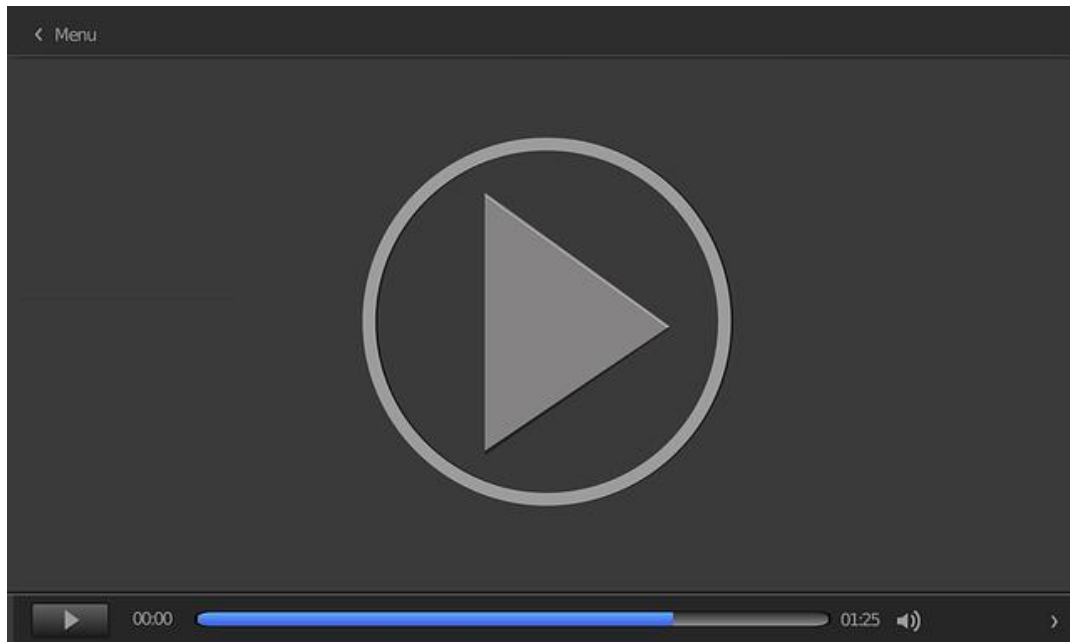
El análisis de sentimientos y la extracción de información son dos aplicaciones clave del NLP que han ganado popularidad en diversas industrias.

Análisis de sentimientos

El análisis de sentimientos es una técnica en el procesamiento del lenguaje natural (NLP) que busca identificar y extraer la **subjetividad** en un texto, es decir, determinar si una pieza de texto tiene un **tono** positivo, negativo o neutral. Este análisis se utiliza ampliamente en **aplicaciones**, como la monitorización de redes sociales, encuestas de satisfacción del cliente y análisis de opiniones. Entre los métodos de análisis de sentimientos encontramos:

- ▶ **Métodos basados en reglas:** utilizan diccionarios predefinidos de palabras con sentimientos asignados. Por ejemplo: el diccionario SentiWordNet contiene palabras asociadas a puntajes positivos y negativos.
- ▶ **Métodos basados en aprendizaje supervisado:** se entrenan modelos de clasificación utilizando *datasets* etiquetados con sentimientos. Algunos algoritmos comunes son: Naive Bayes, máquinas de soporte vectorial (SVM), redes neuronales (RNN-LSTM)
- ▶ **Métodos basados en *deep learning*:** las redes neuronales profundas, como las redes neuronales recurrentes profundas (RDNN), permiten analizar contextos más complejos y capturar la semántica del texto. El modelo BERT (*Bidirectional Encoder Representations from Transformers*), desarrollado por Google, es un modelo avanzado que ha mejorado significativamente el rendimiento en análisis de sentimientos (Devlin, Chang, Lee, et al., 2019).

El vídeo *Transformers* nos muestra cómo funcionan y trabajan los *transformers* para el procesamiento y comprensión del contexto en los textos.



Transformers

Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=dc931c-b7e1-4dff-a79e-b1de008ba713>

En Python podemos emplear la librería TextBlob para realizar esta tarea, como muestra el siguiente código:

```
from textblob import TextBlob
```

```
# Fragmento de texto
```

```
texto = "Estoy muy feliz con el servicio recibido, pero podría ser más rápido."
```

```
# Análisis de sentimientos
```

```
blob = TextBlob(texto)

sentimiento = blob.sentiment

print(f"Polaridad: {sentimiento.polarity}, Subjetividad:
{sentimiento.subjectivity}")
```

Extracción de información

La extracción de información (*information extraction* o IE) es el proceso de **identificar y extraer información estructurada** a partir de texto no estructurado. Esta información puede incluir **entidades** nombradas (como personas, lugares, organizaciones), **relaciones** entre entidades y **hechos** concretos. Entre las principales técnicas de extracción de información encontramos:

- ▶ **Reconocimiento de entidades nombradas** (*Named Entity Recognition* o NER): identifica entidades, como personas, lugares, organizaciones, fechas, etc., dentro de un texto. Algunos algoritmos comunes son: CRF (*Conditional Random Fields*), HMM (*Hidden Markov Models*) y BERT para NER (relación entre entidades).
- ▶ **Métodos supervisados**: se entrenan clasificadores sobre ejemplos etiquetados de relaciones.
- ▶ **Métodos no supervisados**: se descubren relaciones basadas en coocurrencias de entidades.
- ▶ **Extracción de hechos**: identificación de triples (sujeto, predicado, objeto) que representan hechos extraídos del texto. Se puede utilizar técnicas como el análisis de dependencias o gramáticas semánticas.

El siguiente código Python emplea la librería SpaCy para realizar la tarea de **extracción de información** mediante NER:

```
import spacy

# Cargar modelo en español

nlp = spacy.load('es_core_news_sm')

# Texto de ejemplo

texto = "Pedro trabaja en Google desde 2020."

# Procesar el texto

doc = nlp(texto)

# Extracción de Entidades Nombradas

for ent in doc.ents:

    print(ent.text, ent.label_)
```

3.4. Modelos de lenguaje: aplicaciones y desafíos

Un modelo de lenguaje (LM) es una distribución de **probabilidad** sobre las palabras de una secuencia. Estas probabilidades se obtienen de entrenar el modelo en uno o varios corpus de la lengua. Los LM son útiles para distintas tareas de PLN, como el **reconocimiento de voz** o la **traducción automática**.

Entre las distintas **técnicas** aplicadas para el modelado estadístico del lenguaje encontramos:

- ▶ **Basados en N-gramas:** un modelo de lenguaje basado en n-gramas predice la probabilidad de que tenga lugar un determinado n-grama dentro de una secuencia de palabras de la lengua.
- ▶ **Modelo *bag of words*:** el modelo *bag of words* representa un texto como un conjunto de palabras y sus frecuencias. Ignora el orden de las palabras y la estructura gramatical. Cada texto se convierte en un vector donde cada dimensión representa la frecuencia de una palabra específica en el texto.
- ▶ **Modelo *term frequency-inverse document frequency* (TF-IDF):** el modelo TF-IDF no solo cuenta la frecuencia de las palabras, sino que también ajusta estos valores según la importancia de las palabras en el corpus. Esto ayuda a reducir el peso de palabras comunes y a resaltar las más relevantes.

Modelado de temas

El modelado de temas es una técnica avanzada en el procesamiento del lenguaje natural (NLP) que se utiliza para descubrir **temas subyacentes** en un conjunto de documentos. Un tema es un **grupo de palabras** que suelen aparecer juntas en los textos y pueden dar una idea de los tópicos discutidos en los documentos. Dos de los métodos más comunes para el modelado de temas son *Latent Dirichlet Allocation* (LDA) y *Non-Negative Matrix Factorization* (NMF).

- ▶ *Latent Dirichlet Allocation* (LDA) es un método probabilístico de modelado de temas que se utiliza para descubrir los **temas ocultos** en un conjunto de documentos. La idea básica detrás de LDA es que los documentos están compuestos de **varios temas** y cada uno está representado por un conjunto de palabras con diferentes probabilidades.
- ▶ *Non-Negative Matrix Factorization* (NMF) es un método **algebraico lineal** que también se utiliza para modelar temas. A diferencia de LDA, NMF no es un modelo probabilístico, sino una técnica de **factorización** de matrices que busca descomponer una matriz en dos **matrices** más pequeñas con valores no negativos. Los conjuntos de documentos se representan como una matriz de **alta dimensión** donde cada vector es un documento y cada dimensión la **frecuencia** con la que aparece una palabra.

Ambas técnicas se pueden aplicar fácilmente en Python gracias a la librería Scikit-Learn, como muestra este código:

```
from sklearn.feature_extraction.text import CountVectorizer

from sklearn.decomposition import LatentDirichletAllocation

from sklearn.decomposition import NMF

# Fragmento de texto

texto = ["En un lugar de la Mancha, de cuyo nombre no quiero acordarme..."]

# Eliminar stopwords

stop_words = stopwords.words('spanish')

vectorizer = CountVectorizer(stop_words=stop_words)
```

```
# Crear la matriz de palabras

X = vectorizer.fit_transform(texto)

# Aplicar LDA

lda = LatentDirichletAllocation(n_components=1, random_state=42) lda.fit(X)

# Mostrar las palabras más representativas del tema

n_words = 10

words = vectorizer.get_feature_names_out()

for topic_idx, topic in enumerate(lda.components_):

    print(f"Tema {topic_idx}:")

    print(" ".join([words[i] for i in topic.argsort()[:-n_words - 1:-1]]))

# Aplicar NMF

nmf = NMF(n_components=1, random_state=42)

nmf.fit(X)

# Mostrar las palabras más representativas del tema

for topic_idx, topic in enumerate(nmf.components_):
```

```
print(f"Tema {topic_idx}:")

print(" ".join([words[i] for i in topic.argsort()[::-n_words - 1:-1]]))
```

Aplicaciones y desafíos de los modelos de lenguaje

Los modelos de lenguaje tienen multitud de aplicaciones dentro del NLP. Entre las principales encontramos:

- ▶ **Generación de texto:** modelos como GPT pueden generar texto coherente en varios estilos y temas.
- ▶ **Traducción automática:** modelos como BERT son utilizados para mejorar la precisión en la traducción de texto entre diferentes idiomas.
- ▶ **Chatbots y asistentes virtuales:** los LM son clave para el funcionamiento de asistentes virtuales, como Siri, Alexa y Google Assistant.
- ▶ **Resumen automático:** es la capacidad de generar resúmenes concisos de documentos largos.

Sin embargo, emplear estos modelos afronta diversos **desafíos** entre los que destacan:

- ▶ **Bias y fairness:** los modelos de lenguaje entrenados en grandes volúmenes de datos pueden heredar y perpetuar sesgos presentes en los datos, lo que plantea preocupaciones éticas.
- ▶ **Contexto y coherencia:** los modelos deben entender y mantener el contexto de una conversación o documento para generar respuestas coherentes.
- ▶ **Ambigüedad semántica:** las palabras y frases en lenguaje natural a menudo tienen múltiples significados, lo que complica la tarea de interpretación para las máquinas.
- ▶ **Escalabilidad:** el entrenamiento de modelos de lenguaje grandes requiere recursos computacionales significativos.

- ▶ **Interpretabilidad:** entender y explicar las decisiones tomadas por modelos complejos sigue siendo un desafío importante (Bender, Gebru, McMillan-Major, et al., 2021).

3.5. Implementación práctica en el desarrollo de software

El NLP se implementa en diversas áreas del desarrollo de *software*, desde aplicaciones móviles hasta sistemas empresariales:

- ▶ **Chatbots y asistentes virtuales:** integran NLP para comprender y responder consultas de los usuarios y mejorar la interacción humano-computadora.
- ▶ **Motores de búsqueda:** utilizan NLP para interpretar y procesar las consultas de los usuarios y ofrecer resultados más relevantes.
- ▶ **Sistemas de recomendación:** análisis de opiniones y reseñas de productos para ofrecer recomendaciones personalizadas.
- ▶ **Automatización de tareas de soporte:** NLP permite automatizar tareas, como la clasificación de correos electrónicos y la respuesta a preguntas frecuentes.

La implementación en *software* generalmente implica el uso de *frameworks* y bibliotecas, como NLTK, SpaCy, TensorFlow y PyTorch, que facilitan la **creación** y **despliegue** de modelos de NLP.

Ejemplo guiado

El siguiente código en Python muestra un ejemplo de cómo desarrollar un **modelo de NLP** mediante **redes neuronales**:

Preparación del *dataset*

Creemos un **conjunto de datos sencillo** con operaciones aritméticas simples y código correcto, pero que puede o no ser óptimo:

```
import pandas as pd
```

```
# Crear un dataset de ejemplo

data = {

    'code': [

        'for i in range(len(arr)): print(arr[i])', # No optimizado

        'print(arr)', # Optimizado

        'while i < len(arr): print(arr[i]); i += 1', # No optimizado

        'for item in arr: print(item)' # Optimizado

    ],

    'label': [0, 1, 0, 1]

}

df = pd.DataFrame(data)
```

Preprocesamiento de datos

El siguiente paso es aplicar las **técnicas de NLP** vistas para convertir el texto en una **representación numérica**. Utilizaremos la tokenización para preparar los datos del modelo:

```
from tensorflow.keras.preprocessing.text import Tokenizer

from tensorflow.keras.preprocessing.sequence import pad_sequences

# Configurar el tokenizador
```



```
tokenizer = Tokenizer(char_level=True) # Tokenización a nivel de caracteres

tokenizer.fit_on_texts(df['code'])

sequences = tokenizer.texts_to_sequences(df['code'])

X = pad_sequences(sequences, padding='post')


# Preparar las etiquetas

y = np.array(df['label'])
```

Construcción del modelo

Utilizaremos un modelo simple de RNN. Una capa de *embedding* para reducir la **dimensionalidad** de los datos de entrada y captar características principales, un modelo LSTM para capturar **dependencias** en las secuencias y la capa de salida con una activación sigmoide para clasificar el código como **óptimo** o **no óptimo**.

```
from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Embedding, LSTM, Dense


# Parámetros

vocab_size = len(tokenizer.word_index) + 1

max_length = X.shape[1]


# Construir el modelo

model = Sequential()
```

```
model.add(Embedding(vocab_size, 10, input_length=max_length))

model.add(LSTM(50, return_sequences=False))

model.add(Dense(1, activation='sigmoid'))

# Compilar el modelo

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=
['accuracy'])

# Mostrar resumen del modelo

model.summary()
```

Entrenamiento y evaluación

Con el modelo ya compilado, procedemos a su entrenamiento. Aplicamos una división 90/10 para conjunto de **entrenamiento** y **validación**:

```
# Entrenar el modelo

model.fit(X, y, epochs=10, verbose=1, validation_split=0.1)
```

Ahora ya podemos utilizar nuevos fragmentos de código para evaluar y clasificar código óptimo y no óptimo

```
def predict_code_optimization(code_fragment):

    # Preprocesar el código

    seq = tokenizer.texts_to_sequences([code_fragment])
```

```
padded_seq = pad_sequences(seq, maxlen=max_length, padding='post')
```

```
# Predicción
```

```
prediction = model.predict(padded_seq)
```

```
# Interpretar la predicción
```

```
return 'Optimizado' if prediction[0][0] > 0.5 else 'No optimizado'
```

```
# Probar con un nuevo fragmento de código
```

```
nuevo_codigo = 'for i in range(len(arr)): print(arr[i])'
```

```
print(f'El código es: {predict_code_optimization(nuevo_codigo)}')
```

Este es un modelo sencillo que todavía tiene mucho margen de mejora, pues solo se han empleado pequeños fragmentos de código para su entrenamiento. Los modelos actuales más avanzados han sido entrenados con **miles de archivos y millones de líneas de código**.

3.6. Consideraciones éticas en el procesamiento del lenguaje natural

El uso de NLP en el *software* plantea varios desafíos éticos, entre ellos la **privacidad** de los datos, la **equidad** y el sesgo y la posibilidad de generar **contenido dañino** o desinformación.

- ▶ **Privacidad:** la manipulación de grandes volúmenes de datos textuales puede aplicar la exposición involuntaria de información personal sensible. Es crucial implementar técnicas de anonimización y respetar las regulaciones de protección de datos (Floridi, 2019).
- ▶ **Equidad y sesgo:** los modelos de NLP pueden perpetuar sesgos existentes en los datos de entrenamiento, lo que puede llevar a resultados injustos o discriminatorios. Es necesario desarrollar técnicas para detectar y mitigar estos sesgos en los modelos (Blodgett, Barocas, Daumé III, et. al., 2020).
- ▶ **Generación de contenido:** los modelos de lenguaje potentes, como GPT, pueden generar contenido realista que podría usarse para difundir desinformación o contenido malicioso. Es importante desarrollar mecanismos para supervisar y controlar el uso de estos modelos en aplicaciones sensibles (Zellers, Holtzman, Bisk, et. al., 2019).
- ▶ **Manipulación de opiniones:** el análisis de sentimientos puede ser utilizado para manipular la percepción pública en medios sociales y otras plataformas.
- ▶ **Bias y discriminación:** los modelos de NLP pueden perpetuar y amplificar sesgos de género, raza y otros, lo que puede llevar a resultados injustos o discriminatorios (Barocas Hardt, y Narayanan, 2019).

- ▶ **Responsabilidad y rendición de cuentas:** los desarrolladores y las organizaciones que implementan NLP deben ser responsables de los resultados de sus sistemas y deben estar preparados para rendir cuentas en casos de mal uso o resultados perjudiciales.

Además, el uso de NLP en la **automatización de decisiones** en áreas como la contratación, la justicia y la atención médica puede tener implicaciones profundas, ya que las decisiones tomadas por algoritmos pueden afectar significativamente la **vida** de las personas. Por lo tanto, es crucial que los desarrolladores e investigadores consideren estos **aspectos éticos** en todas las etapas del desarrollo de soluciones basadas en NLP.

3.7. Referencias bibliográficas

Barocas, S., Hardt, M. y Narayanan, A. (2019). *Fairness and machine learning: limitations and opportunities*. MIT Press.

Bender, E. M., Gebru, T., McMillan-Major, A. y Shmitchell, S. (2021). *On the dangers of stochastic parrots: can language models be too big?* (pp. 610-623) [Conferencia]. Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency, Nueva York, Estados Unidos.

Blodgett, S. L., Barocas, S., Daumé III, H. y Wallach, H. (2020). Language (technology) is power: a critical survey of “bias” in NLP. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 5454-5476.

Devlin, J., Chang, M. W., Lee, K. y Toutanova, K. (2019). BERT: pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, 4171–4186.

Floridi, L. (2019). Translating principles into practices of digital ethics: five risks of being unethical. *Philosophy & Technology*, 32, 185–193.

Jurafsky, D. y Martin, J. H. (2000). *Speech and language processing: an introduction to natural language processing, computational linguistics, and speech recognition with language models*. Pearson.

Manning, C. D., Surdeanu, M., Bauer, J., Finkel, J., Bethard, S. J., y McClosky, D. (2014). The Stanford CoreNLP natural language processing toolkit. *Proceedings of 52nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 55-60.

Marcus, M., Santorini, B. y Marcinkiewicz, M. A. (1993). Building a large annotated corpus of english: the penn treebank. *Computational Linguistics*, 19(2), 313-330.

Zellers, R., Holtzman, A., Rashkin, H., Bisk, Y., Farhadi, A., Roesner, F., y Choi, Y. (2019). Defending against neural fake news. *Advances in neural information processing systems*, 32.

Modelos de lenguaje pre-entrenados: revisión y avances

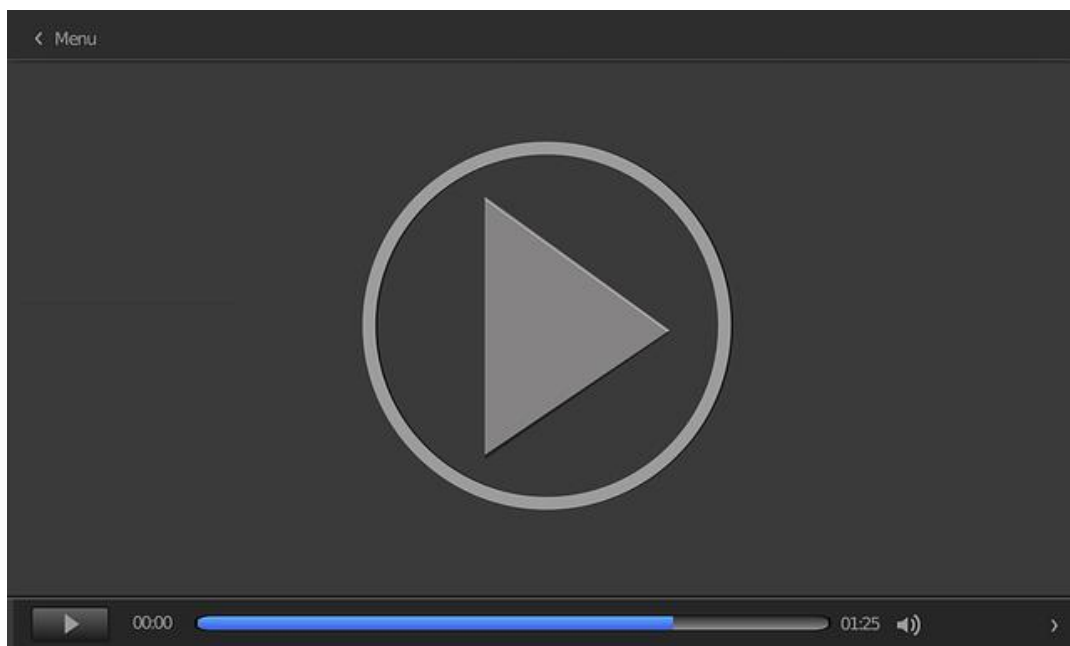
Qiu, X., Sun, T., Xu, Y., Shao, Y., Dai, N. y Huang, X. (2020). Pre-trained models for natural language processing: a survey. *Science China Technological Sciences*, 63(10), 1872-1897. <https://arxiv.org/abs/2003.08271>

Este artículo revisa los avances recientes en el procesamiento del lenguaje natural, especialmente en lo que respecta a los modelos de lenguaje preentrenados, como BERT, GPT y sus variantes. Se exploran las técnicas de ajuste fino, las transferencias de aprendizaje y los desafíos éticos asociados con el uso de modelos grandes.

Incrustación y representación vectorial de palabras

Starmer, J. (2023, julio 24). *Redes neuronales de transformadores, la base de ChatGPT, ¡claramente explicado!* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=zxQyTK8quyY>

Este vídeo ofrece una explicación detallada de las técnicas de *word embedding* y Word3Vec que no se han abordado en el tema. Estas técnicas son la base del desarrollo de *transformers* que están revolucionando el mundo actual del procesamiento del lenguaje natural.



Accede al vídeo:

<https://www.youtube.com/embed/zxQyTK8quyY>

Mejores prácticas éticas en el procesamiento del lenguaje natural

Leidner, J. L. y Plachouras, V. (2017). Ethical by design: ethics best practices for natural language processing. *Proceedings of the First ACL Workshop on Ethics in Natural Language Processing*, 30-40. <https://aclanthology.org/W17-1604/>

Este artículo analiza los desafíos éticos en el desarrollo y la implementación de tecnologías de procesamiento del lenguaje natural. Se abordan temas como el sesgo en los modelos de lenguaje, la privacidad de los datos y el impacto social de la automatización del lenguaje.

1. ¿Qué es el procesamiento del lenguaje natural (NLP)?
 - A. Un campo que estudia la programación de sistemas.
 - B. Un área interdisciplinaria que abarca varios campos relacionados con el lenguaje humano.
 - C. Una rama de la lingüística que solo estudia la semántica.
 - D. Un proceso automático para la traducción de textos.

2. ¿Cuál de las siguientes técnicas no es un tipo de tokenización?
 - A. Tokenización por palabras.
 - B. Tokenización por frases.
 - C. Tokenización por n-gramas.
 - D. Tokenización por números.

3. ¿Cuál es la técnica de normalización que elimina palabras comunes, como 'el', 'de', 'y'?
 - A. Lematización.
 - B. Expansión de contracciones.
 - C. Eliminación de *stopwords*.
 - D. Corrección ortográfica.

4. ¿Cuál es el propósito de la lematización?
 - A. Convertir todo el texto a minúsculas.
 - B. Reducir las palabras a su forma base o lema.
 - C. Eliminar caracteres especiales y puntuación.
 - D. Corregir errores ortográficos.

5. ¿Cuál de las siguientes opciones describe mejor el etiquetado morfosintáctico (POS *tagging*)?
- A. Un proceso para traducir textos.
 - B. Un proceso para asignar categorías gramaticales a las palabras.
 - C. Un proceso para corregir errores de ortografía.
 - D. Un método para eliminar *stopwords*.
6. ¿Qué es el Penn Treebank?
- A. Un conjunto de reglas ortográficas.
 - B. Un corpus anotado utilizado para el etiquetado morfosintáctico en inglés.
 - C. Un algoritmo de traducción automática.
 - D. Un diccionario de sinónimos.
7. ¿Qué técnica de normalización se aplica para convertir un texto a minúsculas?
- A. Expansión de contracciones.
 - B. Conversión a minúsculas.
 - C. Lematización.
 - D. Derivación.
8. ¿Qué técnica se utiliza para manejar palabras como 'tmb' en textos informales?
- A. Derivación.
 - B. Eliminación de *stopwords*.
 - C. Expansión de contracciones.
 - D. Corrección ortográfica.

9. ¿Cuál de los siguientes campos no está relacionado directamente con el NLP?
- A. Informática.
 - B. Ingeniería de telecomunicaciones.
 - C. Biología molecular.
 - D. Lingüística.
10. ¿Cuál es una de las aplicaciones del análisis de sentimientos en NLP?
- A. Predecir el clima.
 - B. Analizar las emociones expresadas en textos.
 - C. Calcular estadísticas matemáticas.
 - D. Mejorar la calidad del audio.
11. ¿Qué problema resuelve la normalización en NLP?
- A. Elimina caracteres innecesarios y estandariza el texto.
 - B. Aumenta el tamaño del texto.
 - C. Traduce textos automáticamente.
 - D. Analiza la estructura gramatical del texto.
12. ¿Qué técnica se utiliza para agrupar palabras con significados similares?
- A. Tokenización.
 - B. Lematización.
 - C. Conversión a minúsculas.
 - D. Expansión de contracciones.

13. ¿Qué método se emplea para dividir un texto en oraciones?
 - A. Tokenización por palabras.
 - B. Tokenización por frases.
 - C. Tokenización por caracteres.
 - D. Tokenización por n-gramas.

14. ¿Qué técnica se aplica para reducir las palabras a su raíz común?
 - A. Derivación.
 - B. Lematización.
 - C. Normalización.
 - D. Tokenización.

15. ¿Cuál es el principal desafío de aplicar modelos de NLP en idiomas diferentes al inglés?
 - A. La falta de tokenización.
 - B. La falta de caracteres especiales.
 - C. La falta de acentos y otros caracteres en el inglés.
 - D. La falta de palabras comunes.

16. ¿Qué técnica de tokenización es más adecuada para el análisis de *spam*?
 - A. Tokenización por caracteres.
 - B. Tokenización por n-gramas.
 - C. Tokenización por palabras.
 - D. Tokenización por frases.

17. ¿Cuál es el principal objetivo del etiquetado morfosintáctico en NLP?
- A. Traducir textos automáticamente.
 - B. Eliminar *stopwords*.
 - C. Reducir el tamaño de los datos.
 - D. Asignar etiquetas gramaticales a cada palabra.
18. ¿Qué factor es crítico al aplicar técnicas NLP a múltiples idiomas?
- A. Las diferencias gramaticales y semánticas entre idiomas.
 - B. El tamaño del vocabulario.
 - C. El número de palabras.
 - D. La longitud de las oraciones.
19. ¿Cuál es el propósito principal de eliminar *stopwords* en el NLP?
- A. Reducir el ruido en el análisis de texto.
 - B. Aumentar la precisión del etiquetado POS.
 - C. Incrementar la longitud del texto.
 - D. Mejorar la corrección ortográfica.
20. ¿Qué técnica se usa para identificar el contexto semántico en un *corpus*?
- A. Tokenización.
 - B. Análisis sintáctico.
 - C. Modelos de lenguaje.
 - D. Eliminación de *stopwords*.

Fundamentos de Inteligencia Artificial para Ingenieros de
Software

Tema 4. Ética en la inteligencia artificial

Índice

Esquema

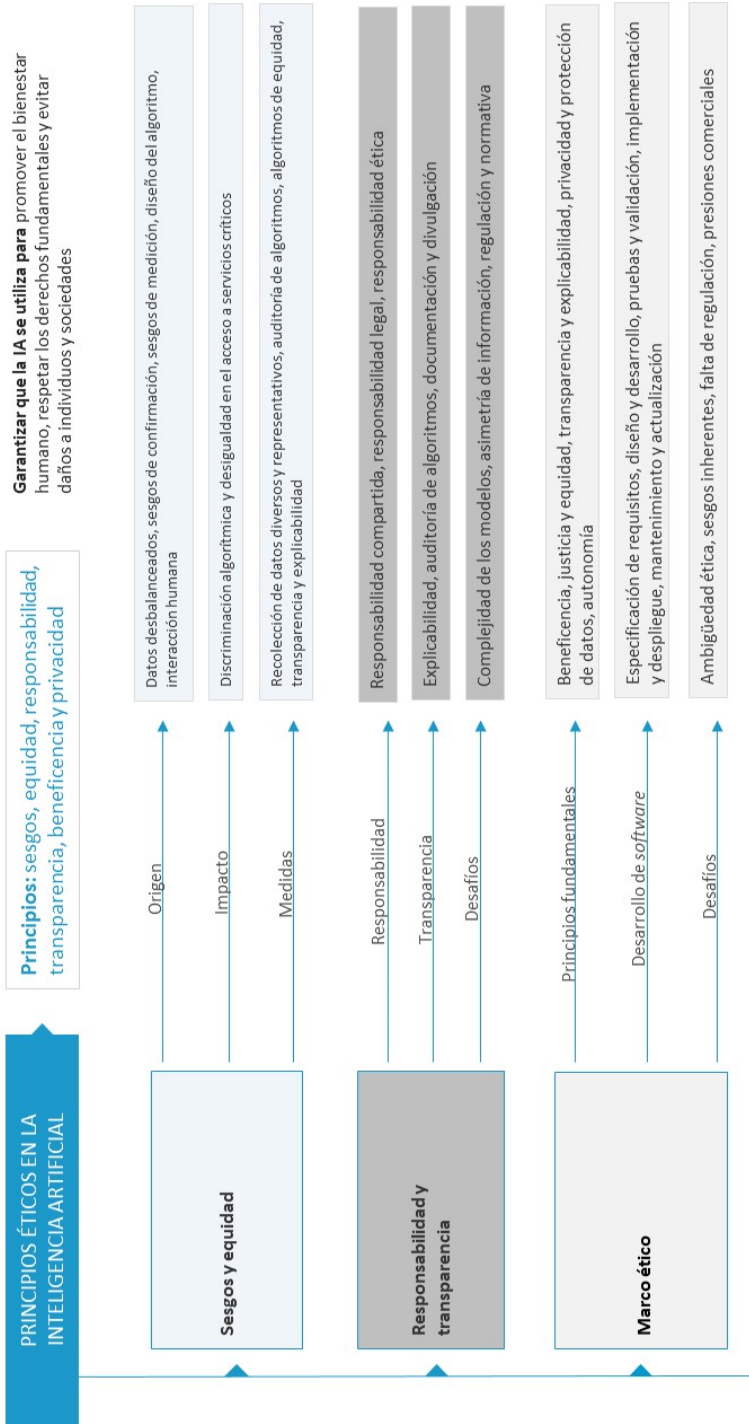
Ideas clave

- 4.1. Introducción y objetivos
- 4.2 Principios éticos en la inteligencia artificial
- 4.3. Sesgo y equidad en modelos de IA
- 4.4. Responsabilidad y transparencia
- 4.5. Casos de estudio: ética en proyectos de IA
- 4.6. Marco ético en el desarrollo de software
- 4.7. Referencias bibliográficas

A fondo

- Xplainable Artificial Intelligence
- IBM AI Fairness
- El fin de la realidad
- Principios de IA de Google

Test



4.1. Introducción y objetivos

El desarrollo y la implementación de la inteligencia artificial (IA) ha transformado significativamente diversos sectores, desde la tecnología hasta la salud, y continúan evolucionando a un ritmo acelerado. Sin embargo, este avance tecnológico trae consigo desafíos éticos complejos que requieren una consideración cuidadosa para garantizar que la IA se utilice de manera responsable y equitativa. Los principios éticos son fundamentales en el desarrollo de la IA, ya que guían la creación de tecnologías que respeten los derechos humanos, promuevan la equidad y minimicen los riesgos asociados.

En este contexto, este tema presenta una serie de contenidos que exploran los aspectos éticos más relevantes en el desarrollo de *software* y proyectos de IA que proporcionan un marco comprensivo para abordar estas cuestiones. A través de estudios de casos reales y principios fundamentales se busca proporcionar una comprensión detallada de las implicaciones éticas en el desarrollo de modelos de IA y cómo estas pueden ser gestionadas eficazmente.

Los objetivos de este tema son:

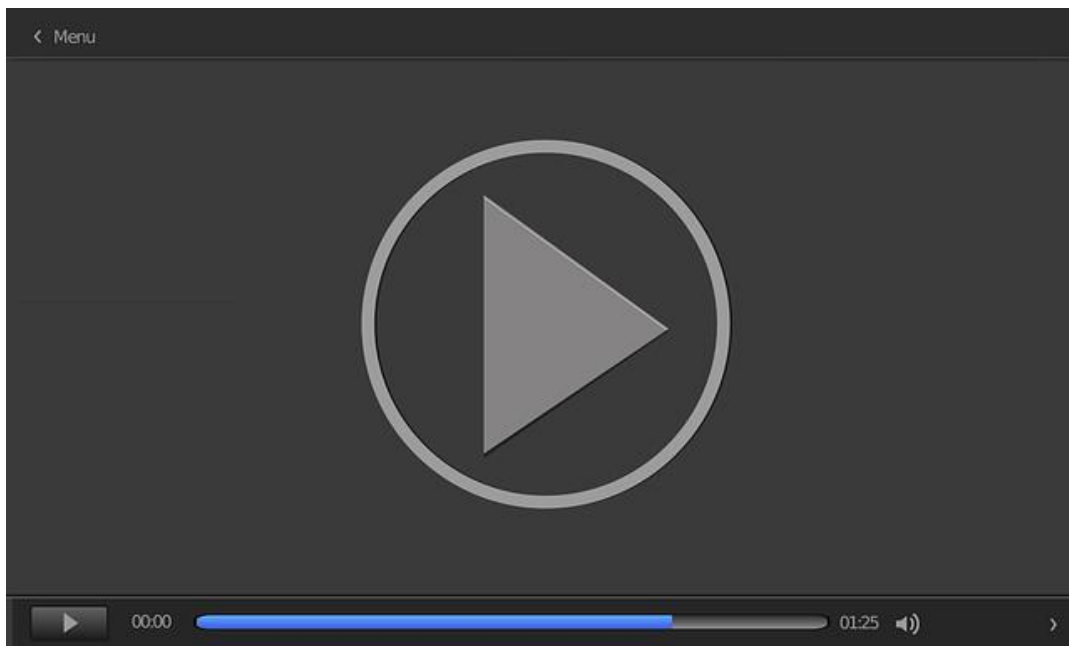
- ▶ Desarrollar una comprensión integral de los principios éticos fundamentales que deben guiar el desarrollo de *software* y tecnologías de IA.
- ▶ Analizar la importancia de un marco ético en el desarrollo de *software* destacando la necesidad de integrar la ética en todas sus etapas del ciclo de vida.
- ▶ Examinar casos de estudio reales en los que los desafíos éticos en proyectos de IA han sido evidentes.
- ▶ Explorar los conceptos de sesgo y equidad en los modelos de IA identificando sesgos inherentes y proponiendo estrategias para mitigar estos riesgos.

- ▶ Fomentar la capacidad crítica en la evaluación de proyectos de IA promoviendo un enfoque ético que priorice la transparencia, la equidad y la protección de los derechos fundamentales.

4.2 Principios éticos en la inteligencia artificial

El avance de la inteligencia artificial (IA) ha generado un debate global sobre los principios éticos que deben guiar su desarrollo y aplicación. Estos principios buscan garantizar que las tecnologías de IA se utilicen de manera que promuevan el **bienestar humano**, respeten los **derechos fundamentales** y eviten **daños** a individuos y sociedades.

El vídeo *Ley Europea de IA* muestra los distintos reglamentos y normativas elaborados dentro de la Unión Europea (EU) con el fin de regular y clasificar el *software* desarrollado mediante IA, así como promover un uso ético y responsable.



Ley Europea de IA

Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=2aa31d5f-d6aa-45c3-b536-b1de00922136>

A continuación, se presentan los principales principios éticos que se han propuesto en el ámbito de la IA ilustrados con ejemplos reales y sustentados en investigaciones académicas.

- ▶ **Sesgos y equidad:** los sistemas de IA deben ser diseñados y operados de manera que no perpetúen ni exacerbén las desigualdades sociales existentes. Esto incluye evitar sesgos en los datos de entrenamiento y en los algoritmos que podrían llevar a resultados discriminatorios.
- Los algoritmos de IA si no se diseñan con cuidado pueden reforzar **sesgos históricos**, como lo demuestran casos en los que los sistemas de reconocimiento facial han mostrado tasas de error más altas para personas de raza negra en comparación con personas de raza blanca (Buolamwini y Gebru, 2018).
- ▶ **Responsabilidad:** este principio establece que los desarrolladores y operadores de sistemas de IA deben rendir cuentas por sus decisiones y acciones. La IA no debe ser utilizada para eludir responsabilidades humanas, especialmente en decisiones críticas, como las relacionadas con la vida, la libertad y los derechos fundamentales.
- Un incidente que subraya la necesidad de responsabilidad ocurrió en 2018 cuando un vehículo autónomo de Uber atropelló y mató a una peatona en Arizona. Este caso planteó cuestiones sobre quién es responsable en tales situaciones: los desarrolladores del *software*, los fabricantes del *hardware* o los operadores humanos que supervisan el sistema. Cath (2018) explora cómo la responsabilidad en sistemas de IA debe ser claramente definida y compartida entre los desarrolladores, operadores y reguladores para prevenir la evasión de responsabilidades.

- ▶ **Transparencia:** la transparencia implica que los procesos, decisiones y algoritmos utilizados por los sistemas de IA deben ser comprensibles y accesibles para los usuarios y reguladores. Esto es crucial para generar confianza en la tecnología y para permitir la auditoría de los sistemas de IA. La caja negra de los modelos de IA, especialmente aquellos basados en aprendizaje profundo, puede llevar a situaciones en las que los usuarios no comprendan cómo o por qué se ha llegado a una determinada decisión.
 - En el artículo de Burrell (2016) se discute cómo la opacidad en los algoritmos puede derivar en falta de **confianza** y cómo la transparencia es esencial para la **legitimidad** de las decisiones algorítmicas.
- ▶ **Beneficencia:** el principio de beneficencia establece que la IA debe ser utilizada para promover el bienestar y el beneficio de las personas y la sociedad en general, minimizando los riesgos y daños potenciales.
 - Por ejemplo, el uso de IA para predecir la sepsis en **pacientes hospitalizados** ha demostrado ser prometedor, pero también plantea riesgos si las predicciones son incorrectas o malinterpretadas (Komorowski, Celi, Badawi, et. al., 2018).
- ▶ **Privacidad:** la privacidad es un principio fundamental en la ética de la IA, ya que exige la protección de la información personal y el respeto por la intimidad de los individuos. Esto es particularmente importante en sistemas que recopilan y analizan grandes cantidades de datos personales. Las regulaciones, como el Reglamento General de Protección de Datos (GDPR) de la Unión Europea, buscan proteger la privacidad de los usuarios imponiendo estrictos requisitos sobre cómo se deben manejar los datos personales.
 - Un ejemplo de la importancia de este principio es el escándalo de Cambridge Analytica, donde se utilizó IA para influir en **decisiones políticas** a través de la **explotación** indebida de **datos personales** (Zuboff, 2019).

4.3. Sesgo y equidad en modelos de IA

El uso de modelos de inteligencia artificial (IA) en diversas aplicaciones ha revelado uno de los desafíos más críticos: la presencia de **sesgos** que pueden comprometer la **equidad**:

- ▶ El sesgo en modelos de IA se refiere a las **desviaciones sistemáticas** que conducen a resultados **injustos** o inexactos para ciertos grupos de personas. Este sesgo puede surgir en diferentes etapas del ciclo de vida de un modelo de IA, desde la recolección de datos hasta la fase de entrenamiento y la interpretación de los resultados (Angwin, Larson, Mattu, et al., 2016).
- ▶ La equidad en modelos de IA se refiere a la necesidad de que estos sistemas traten de manera **justa** a todos los **grupos y personas**, independientemente de características, como raza, género, religión o estatus socioeconómico (Obermeyer, Powers, Vogeli, et al., 2019).

Lograr la equidad requiere implementar **estrategias específicas** para identificar y corregir los sesgos en los datos y en los algoritmos

A continuación, se analizan los conceptos de sesgo y equidad en el contexto de los modelos de IA con ejemplos reales que ilustran la importancia de abordar estos problemas de manera efectiva.

Origen de los sesgos en los modelos de IA

Los sesgos en los modelos de IA pueden surgir de varias **fuentes**, siendo las más comunes:

- ▶ **Datos de entrenamiento desbalanceados:** los modelos de IA se entrenan utilizando grandes conjuntos de datos. Si estos datos reflejan desequilibrios o prejuicios existentes en la sociedad, entonces el modelo puede aprender y replicar esos sesgos.

- Por ejemplo, en un estudio realizado por Angwin, Larson, Mattu, et. al. (2016), el algoritmo COMPAS, utilizado en Estados Unidos para predecir la reincidencia criminal, mostró una tendencia a sobreestimar el riesgo de reincidencia para personas afroamericanas y a subestimarlo para personas blancas.
- ▶ **Sesgo de confirmación:** se da cuando los diseñadores de modelos, conscientes o inconscientemente, seleccionan características o datos que confirman sus expectativas o suposiciones previas.
- Por ejemplo, si se está creando un sistema de IA para evaluar el riesgo financiero y el equipo cree que ciertos datos demográficos, como la edad o la zona donde viven, son indicativos de mayor riesgo, entonces pueden ajustar el modelo para que enfatice esos datos, ignorando otras variables relevantes.
- ▶ **Sesgo de medición:** sucede cuando las variables utilizadas en un modelo son *proxies* imperfectos de los fenómenos que se pretende medir, lo cual puede distorsionar los resultados.
- Un *proxy* imperfecto se da cuando se intenta medir una característica con una variable o indicador que no puede representarla correctamente. Por ejemplo, si tratásemos de medir la probabilidad de cometer delitos con el nivel de pobreza
- ▶ **Diseño del algoritmo:** incluso con datos balanceados los algoritmos pueden introducir sesgos si no se diseñan cuidadosamente. Ciertas técnicas de aprendizaje automático, como la regresión logística o las redes neuronales, pueden sesgarse hacia clases mayoritarias si no se implementan mecanismos correctivos, como la ponderación de clases o la inclusión de regularizaciones.
- Por ejemplo, un modelo de recomendación musical mediante regresión lineal puede favorecer productos que ya tienen una alta popularidad, ignorando así productos nuevos o menos conocidos. Esto es así porque esta técnica está diseñada para minimizar el error cuadrático sin considerar la equidad o diversidad en las

recomendaciones

- ▶ **Interacción humana:** los sesgos también pueden introducirse en la fase de desarrollo y ajuste de los modelos debido a las decisiones de los desarrolladores, que pueden tener prejuicios conscientes o inconscientes.
- Por ejemplo, algunos estudios han demostrado que la falta de diversidad en los equipos de desarrollo puede conducir a la creación de sistemas que no contemplen las necesidades de grupos minoritarios (West, Whittaker y Crawford, 2019).

Impacto de los sesgos en la equidad

La equidad en modelos de IA se refiere a la necesidad de que estos sistemas traten de manera **justa** a todos los grupos y personas, independientemente de características, como raza, género, religión o estatus socioeconómico. Lograr la equidad requiere implementar estrategias específicas para **identificar** y **corregir** los sesgos en los datos y en los algoritmos. El **impacto** de los sesgos en los modelos de IA puede ser profundo y amplio:

- ▶ **Discriminación algorítmica:** la discriminación algorítmica ocurre cuando un modelo de IA trata de manera diferente a grupos de personas en función de características protegidas, como raza o género.
- Un caso notable es el de Amazon, que en 2018 tuvo que desechar un sistema de contratación basado en IA porque mostraba un sesgo en contra de candidatas mujeres debido a que el modelo fue entrenado en datos históricos que reflejaban la predominancia masculina en el sector tecnológico (Dastin, 2018).
- ▶ **Desigualdad en el acceso a servicios:** los sesgos pueden llevar a desigualdades en el acceso a servicios críticos, como atención médica o crédito financiero.
- Un estudio encontró que los sistemas de IA utilizados para la clasificación de riesgo en servicios de salud asignaban menos recursos a pacientes afroamericanos, lo que perpetuaba desigualdades en la calidad de la atención recibida (Obermeyer, Powers,

Vogeli, et. al., 2019).

Estrategias para mitigar el sesgo y mejorar la equidad

Para abordar el sesgo y mejorar la equidad en los modelos de IA se han propuesto diversas **estrategias**:

- ▶ **Recolección de datos diversos y representativos:** para evitar sesgos es crucial utilizar conjuntos de datos que sean representativos de la población a la que se aplicará el modelo. Esto implica la recolección de datos que incluyan adecuadamente a grupos minoritarios y la corrección de desequilibrios en las muestras. Los conjuntos de datos desbalanceados son el principal problema para corregir en el entrenamiento. El *overasampling* (sobremuestreo) y *undersampling* (submuestreo) se dan cuando se presentan conjuntos de datos en que una clase esta sobrerrepresentada o infrarrepresentada.
- Existen diversas técnicas para balancear los datos, como SMOTE y ADASYN para el sobremuestreo y RUS, ENN y TL para el submuestreo. En ambos casos, utilizar métricas, como AUC, ayuda a mitigar los sesgos presentes en los conjuntos de datos desbalanceados
- ▶ **Auditoría de algoritmos:** la auditoría de algoritmos implica la revisión sistemática de los modelos de IA para identificar y corregir sesgos. Esta práctica es esencial para garantizar que los modelos sean equitativos.
- Un ejemplo es el trabajo realizado por Buolamwini y Gebru (2018), quienes auditaron sistemas de reconocimiento facial y demostraron la existencia de sesgos raciales, lo que llevó a mejoras en las tecnologías auditadas.
- ▶ **Algoritmos de equidad:** se han desarrollado algoritmos específicamente diseñados para reducir los sesgos, como los algoritmos de *fairness-aware* (conciencia de equidad). Estos métodos incluyen ajustes durante el entrenamiento del modelo para garantizar que las decisiones sean equitativas entre diferentes grupos demográficos (Hardt, Price y Srebro, 2016).

- ▶ **Transparencia y explicabilidad:** promover la transparencia y la explicabilidad en los modelos de IA ayuda a identificar posibles sesgos y a tomar decisiones informadas para su corrección. Esto se alinea con la creciente demanda de IA explicable (XAI, por sus siglas en inglés), que busca que los modelos no solo sean precisos, sino también comprensibles para los usuarios y auditores.

Uno de los avances más recientes en este campo es el desarrollo de **marcos de trabajo** y herramientas para **evaluar** la **equidad** en los modelos de IA. Por ejemplo, IBM ha lanzado el AI Fairness 360 Toolkit, un conjunto de herramientas de código abierto que permite a los desarrolladores evaluar y **mitigar** los sesgos en sus modelos (Bellamy, Dey, Hind, et. al., 2019). Este tipo de herramientas son esenciales para impulsar la equidad en la **práctica diaria** del desarrollo de IA.

4.4. Responsabilidad y transparencia

La inteligencia artificial (IA) ha permitido avances significativos en diversos campos, desde la medicina hasta la seguridad pública. Sin embargo, estos avances también han generado preocupaciones sobre la **responsabilidad** y la **transparencia** en el **desarrollo** y la **implementación** de estas tecnologías. A medida que los sistemas de IA asumen un papel más destacado en la toma de decisiones, es crucial garantizar que estas tecnologías se utilicen de manera **ética y responsable**.

Responsabilidad en la inteligencia artificial

La responsabilidad en la IA se refiere a la necesidad de identificar claramente quién es **responsable** de las **decisiones** y **acciones** tomadas por los sistemas de IA. Este es un tema complejo debido a la naturaleza autónoma de muchos sistemas de IA y a la intervención de **múltiples actores** en su desarrollo y despliegue.

- ▶ **Responsabilidad compartida:** en el ecosistema de la IA la responsabilidad suele estar distribuida entre varios actores, incluyendo desarrolladores, proveedores de datos, usuarios finales e, incluso, la propia IA. En casos donde ocurren fallos o decisiones cuestionables, puede ser difícil determinar quién es responsable.
 - Por ejemplo, en el caso del accidente mortal del vehículo autónomo de Uber en 2018 surgieron preguntas sobre si la responsabilidad recaía en los desarrolladores del *software*, la empresa que implementó la tecnología o los reguladores que autorizaron las pruebas en carreteras públicas (Goodall, 2019).
- ▶ **Responsabilidad legal:** la asignación de responsabilidad legal en casos que involucran la IA es un área en evolución. Actualmente, en muchas jurisdicciones las leyes no están completamente adaptadas para abordar los desafíos específicos planteados por la IA.
 - Por ejemplo, los marcos legales tradicionales pueden no contemplar adecuadamente

situaciones en las que las decisiones automatizadas de la IA resultan en daños o discriminación (Wagner, 2018).

- ▶ **Responsabilidad ética:** más allá de las implicaciones legales, existe una dimensión ética de la responsabilidad en la IA. Los desarrolladores y organizaciones que crean y despliegan sistemas de IA tienen la responsabilidad de garantizar que sus sistemas no causen daño. Esto incluye tomar medidas proactivas para identificar y mitigar posibles sesgos y fallos antes de que los sistemas se implementen en situaciones del mundo real (Floridi, Cowls, Beltrametti, et. al., 2018).

Transparencia en la inteligencia artificial

La transparencia en la IA es crucial para fomentar la **confianza** y la **comprensión** de cómo funcionan estos sistemas. La falta de transparencia, especialmente en modelos complejos, como las redes neuronales profundas, puede llevar a situaciones donde los usuarios y las personas afectadas no entienden cómo se toman las decisiones, lo que a su vez puede generar desconfianza y **resistencias**.

- ▶ **Explicabilidad:** se refiere a la capacidad de un sistema de IA para proporcionar explicaciones claras y comprensibles de sus decisiones. Este es un aspecto fundamental de la transparencia, ya que permite a los usuarios comprender cómo y por qué un sistema llegó a una decisión particular.
- En la medicina, por ejemplo, un sistema de IA que recomienda un tratamiento debe ser capaz de explicar los factores que influyeron en esa recomendación para que los médicos y pacientes puedan tomar decisiones informadas (Doshi-Velez y Kim, 2017).
- ▶ **Auditoría de algoritmos:** la transparencia también implica la capacidad de auditar los sistemas de IA para evaluar su desempeño, identificar sesgos y garantizar que cumplen con los estándares éticos y legales.
- Algunas herramientas, como el "AI Fairness 360 Toolkit de IBM, permiten a los desarrolladores y auditores evaluar la equidad y la transparencia de los modelos de IA antes de su despliegue (Bellamy, Dey, Hind, et. al., 2019).

- ▶ **Documentación y divulgación:** las prácticas de documentación clara y completa de los sistemas de IA, incluyendo los datos utilizados, los algoritmos empleados y las decisiones de diseño, son esenciales para la transparencia. Esta documentación permite a los reguladores, auditores y usuarios finales evaluar la idoneidad y la confiabilidad de los sistemas de IA (Mitchell, Wu, Zaldivar, et. al., 2019).

Desafíos y consideraciones futuras

A pesar de los avances en responsabilidad y transparencia, aún persisten **desafíos significativos:**

- ▶ **Complejidad de los modelos:** los modelos de IA, especialmente los basados en aprendizaje profundo, son inherentemente complejos y difíciles de interpretar. Esto puede limitar la transparencia, ya que incluso los desarrolladores pueden no comprender completamente cómo un modelo toma decisiones.
 - La investigación en IA explicable (XAI) busca abordar este desafío, pero aún se encuentra en etapas tempranas.
- ▶ **Asimetría de información:** los usuarios de sistemas de IA a menudo tienen menos información y comprensión de cómo funcionan estos sistemas en comparación con los desarrolladores. Esta asimetría puede llevar a un uso indebido o a una confianza excesiva en la tecnología con posibles consecuencias negativas (Ananny y Crawford, 2018).
- ▶ **Regulación y normativa:** la creación de marcos regulatorios que promuevan la transparencia y la responsabilidad en la IA es una tarea compleja que requiere equilibrar la innovación con la protección de los derechos de los ciudadanos.
 - En la Unión Europea el Reglamento General de Protección de Datos (GDPR) incluye disposiciones para el derecho a explicación. Exige la transparencia en las decisiones automatizadas, pero su implementación en la práctica aún plantea desafíos (Wachter, Mittelstadt, y Floridi, 2017).

La responsabilidad y la transparencia son pilares fundamentales para el desarrollo y la implementación ética de la inteligencia artificial. A medida que estos sistemas se integran más profundamente en la sociedad, es crucial que se establezcan y se cumplan **normas claras** que aseguren que las **decisiones automatizadas** sean justas, **comprensibles** y **atribuibles** a los actores correctos. La combinación de medidas legales, éticas y técnicas es esencial para enfrentar los desafíos asociados con la responsabilidad y la transparencia en la IA.

4.5. Casos de estudio: ética en proyectos de IA

El **impacto** de la inteligencia artificial (IA) en la sociedad es profundo y generalizado, lo que plantea importantes cuestiones éticas que deben ser abordadas en cada etapa de desarrollo e implementación. A continuación, se presentan varios casos de estudio que ilustran cómo los **dilemas éticos** han surgido en proyectos de IA y cómo se han **abordado** (o fallado en abordar) estos desafíos.

Proyecto Maven de Google

El Proyecto Maven, iniciado en 2017, fue una colaboración entre Google y el Departamento de Defensa de los Estados Unidos para utilizar la IA en el **análisis** de **imágenes** capturadas por drones militares. El proyecto estaba destinado a mejorar la precisión de los **ataques dirigidos** y, de esta forma, reducir daños colaterales.

El uso de IA en aplicaciones militares plantea serias cuestiones éticas, especialmente en relación con la **autonomía** de los sistemas de armas y la implicación de las empresas tecnológicas en **conflictos armados**. Los empleados de Google expresaron su preocupación por el uso de la tecnología en acciones militares que podrían resultar en la **pérdida de vidas humanas**. Más de 3000 empleados firmaron una carta de protesta exigiendo que la empresa se retirara del proyecto (Shane y Wakabayashi, 2018).

En respuesta a la **protesta interna** y la **cobertura mediática**, Google decidió no renovar su contrato con el Departamento de Defensa en 2018. Además, la empresa publicó sus principios de IA que prohíben explícitamente el desarrollo de tecnologías de IA para armas y otros usos que puedan causar daño (Pichai, 2018).

Este caso subraya la importancia de alinear los proyectos de IA con **principios éticos claros** y demuestra cómo la **presión interna** y **externa** puede influir en las decisiones empresariales en torno al uso de IA.

Reconocimiento facial de Amazon (Rekognition)

Amazon desarrolló una tecnología de reconocimiento facial llamada Rekognition que se ofreció a agencias gubernamentales, incluidas las fuerzas del orden. Esta tecnología fue utilizada para **identificar** y **rastrear** individuos en **tiempo real**, lo que planteó preocupaciones sobre la privacidad y la vigilancia masiva.

El reconocimiento facial ha sido objeto de controversia debido a su potencial para infringir la **privacidad** de los individuos y su tendencia a **presentar errores**, especialmente en la identificación de personas de color y mujeres. Un estudio realizado por Buolamwini y Gebru (2018) demostró que las tasas de error en el reconocimiento facial eran significativamente más altas para personas de raza negra en comparación con personas de raza blanca, lo que planteó riesgos de **discriminación y errores judiciales**.

Frente a la creciente presión de defensores de los derechos civiles y el escrutinio público, Amazon impuso en 2020 una moratoria de un año en el uso de Rekognition por parte de la policía en espera de **regulaciones gubernamentales** más **estrictas** (Godfrey, 2020).

Este caso destaca la importancia de evaluar el **impacto social** y **ético** de las tecnologías de IA, especialmente cuando se implementan en contextos sensibles, como la seguridad pública.

Sistema de predicción de reincidencia COMPAS

COMPAS (*Correctional Offender Management Profiling for Alternative Sanctions*) es un algoritmo utilizado en Estados Unidos para evaluar el riesgo de reincidencia de los acusados y para ayudar a los jueces a tomar decisiones sobre la **libertad bajo fianza** y la **sentencia**.

En 2016 una investigación de ProPublica reveló que COMPAS presentaba un **sesgo racial** significativo: los acusados afroamericanos eran etiquetados erróneamente

como de alto riesgo de reincidencia casi el doble de veces que los acusados blancos, mientras que estos últimos eran etiquetados incorrectamente como de bajo riesgo más frecuentemente (Angwin, Larson, Mattu, et. al., 2016). Este sesgo plantea graves preocupaciones sobre la **equidad** y la **justicia** en el sistema judicial.

A pesar de las críticas, el uso de COMPAS y sistemas similares sigue siendo común en varios estados de EE. UU. Sin embargo, este caso ha impulsado un debate más amplio sobre la ética en el uso de IA en la justicia penal y la necesidad de **transparencia** y **rendición de cuentas** en los algoritmos que influyen en decisiones críticas.

Este caso subraya los peligros de confiar en algoritmos opacos y sesgados en contextos de alto impacto y resalta la necesidad de desarrollar sistemas de IA que sean justos y transparentes.

Microsoft Tay: Chatbot con aprendizaje de comentarios

En 2016 Microsoft lanzó un chatbot llamado Tay diseñado para interactuar con usuarios de Twitter y aprender de esas interacciones para mejorar sus respuestas. Tay estaba diseñado para imitar el lenguaje y el comportamiento de una joven de 19 años y se esperaba que ofreciera una experiencia amigable e interactiva.

En pocas horas, usuarios malintencionados comenzaron a enseñar a Tay comentarios racistas, sexistas y ofensivos, lo que llevó al chatbot a emitir respuestas inapropiadas y dañinas en la plataforma. Esto planteó serias preguntas sobre la **seguridad** y la **supervisión** de los sistemas de IA que interactúan públicamente y aprenden en tiempo real.

Microsoft retiró a Tay del servicio en menos de 24 horas y emitió una disculpa pública. La empresa reconoció que no había previsto adecuadamente los riesgos y anunció que trabajaría en mejorar los mecanismos de seguridad para evitar que futuros chatbots fueran **manipulados** de manera similar (Neff y Nagy, 2016).

Este caso ilustra los riesgos asociados con los sistemas de IA que aprenden en tiempo real de interacciones humanas sin un **control adecuado**. También resalta la necesidad de implementar **salvaguardas** robustas para prevenir el uso malintencionado de tecnologías de IA.

Proyecto de reconocimiento facial de Clearview AI

Clearview AI es una empresa que desarrolló una aplicación de **reconocimiento facial** basada en una base de datos de más de tres mil millones de imágenes recopiladas de redes sociales y sitios web. Este proyecto generó una controversia significativa debido a preocupaciones sobre la **privacidad**, el **consentimiento** y el potencial de **abuso** por parte de los gobiernos y las fuerzas del orden.

Clearview AI recopiló imágenes sin el consentimiento de los individuos, lo que plantea serias preocupaciones sobre la **invasión de la privacidad**. La empresa fue criticada por la falta de transparencia en la forma en que recopilaba y utilizaba los datos. El uso de esta tecnología por parte de agencias gubernamentales podría llevar a la **vigilancia masiva** y a la **represión** de derechos civiles.

Este caso subraya la necesidad de establecer **regulaciones** más estrictas sobre el uso de tecnologías de reconocimiento facial y resalta la importancia de la transparencia y el **consentimiento** en la recopilación de datos. Singer (2020) analiza las implicaciones éticas y legales del uso del reconocimiento facial por parte de Clearview AI y otros actores y destaca la necesidad de regulaciones robustas para proteger la **privacidad individual**.

Proyecto Google Duplex

Google Duplex es un sistema de inteligencia artificial diseñado para realizar **llamadas telefónicas** en nombre de los usuarios simulando una **conversación natural**. La tecnología es capaz de realizar tareas como reservar mesas en restaurantes o programar citas y lo hace de manera tan realista que los interlocutores a menudo no se dan cuenta de que están hablando con una máquina.

El hecho de que los interlocutores no sepan que están hablando con una IA plantea cuestiones sobre el **consentimiento informado**. Además, la automatización de tareas simples mediante IA puede llevar a la reducción de empleos en sectores, como el servicio al cliente.

Este caso resalta la importancia de que las tecnologías de IA sean transparentes y de que las personas sean **informadas** cuando interactúan con una máquina en lugar de un humano. También pone de manifiesto los posibles **impactos económicos** y **sociales** de la automatización impulsada por la IA. Liao y Sundar (2020) investigan la percepción pública de Google Duplex y otros sistemas similares, sugiriendo que la transparencia es crucial para mantener la confianza en estas tecnologías.

Desarrollo de IA para el diagnóstico médico: Watson de IBM

IBM Watson fue desarrollado como una herramienta de IA para ayudar en el diagnóstico médico, especialmente en el tratamiento del cáncer. Aunque inicialmente se promocionó como un **avance revolucionario**, Watson ha enfrentado críticas por proporcionar recomendaciones de **tratamientos inexactos** y basadas en **datos limitados**.

La precisión en el diagnóstico y el tratamiento es crucial, y la IA debe ser rigurosamente evaluada antes de ser utilizada en entornos médicos. Ante esto, surge la cuestión de quién es **responsable** cuando la IA comete un error en un entorno crítico, como el de la salud.

Este caso resalta la importancia de la **validación exhaustiva** y **continua** de las tecnologías de IA en el campo de la salud, así como la necesidad de establecer marcos de **responsabilidad** claros. Cabitza, Rasoini y Gensini (2017) exploran los desafíos éticos y prácticos del uso de IA en el diagnóstico médico y subrayan la necesidad de enfoques más prudentes y responsables

4.6. Marco ético en el desarrollo de software

El desarrollo de *software* no es solo una **disciplina técnica**, sino también una actividad profundamente **ética**. Los sistemas y aplicaciones que se crean pueden tener un impacto significativo en la sociedad, la economía y las vidas individuales. Por lo tanto, es crucial que los desarrolladores de *software* adopten un **marco ético** que guíe sus decisiones a lo largo de todo el ciclo de vida del desarrollo. Este marco debe abordar consideraciones éticas clave que aseguren que el *software* sea desarrollado y utilizado de manera responsable.

Principios fundamentales del marco ético

Los principios éticos fundamentales en el desarrollo de *software* incluyen:

- ▶ **Beneficencia:** los desarrolladores deben procurar que el *software* contribuya al bienestar de los usuarios y la sociedad en general para evitar daños. Esto implica diseñar un *software* que sea seguro, fiable y que respete los derechos y la dignidad de todas las personas.
- Por ejemplo, en el desarrollo de *software* médico es esencial garantizar que los sistemas no introduzcan riesgos para los pacientes, como errores en los diagnósticos o tratamientos.
- ▶ **Justicia y equidad:** el *software* debe ser justo y equitativo para que garantice que todos los usuarios, independientemente de su raza, género, religión, nivel socioeconómico u otras características, tengan un acceso equitativo a sus beneficios. Esto incluye evitar el sesgo en los algoritmos y proporcionar igualdad de oportunidades para todas las personas afectadas por el *software*.

- ▶ **Transparencia y explicabilidad:** es fundamental que los usuarios y las partes interesadas comprendan cómo funciona el *software*, especialmente en sistemas que toman decisiones automatizadas. Esto no solo ayuda a construir la confianza, sino que también permite que los usuarios detecten y corrijan posibles errores o sesgos en el sistema.
 - Un ejemplo claro es la implementación de algoritmos en finanzas donde la transparencia en la toma de decisiones es crucial para evitar discriminaciones o errores que puedan afectar a la economía de los usuarios.
- ▶ **Privacidad y protección de datos:** los desarrolladores deben garantizar que el *software* respete la privacidad de los usuarios y proteja sus datos personales. Esto implica implementar prácticas de seguridad robustas y cumplir con regulaciones, como el Reglamento General de Protección de Datos (GDPR) en Europa.
 - Un caso relevante es la controversia en torno a aplicaciones de rastreo de contactos durante la pandemia de COVID-19 donde se debatió el equilibrio entre la salud pública y la privacidad individual (Morley, Cowls, Taddeo, et al., 2020).
- ▶ **Autonomía:** se debe garantizar y respetar el derecho de los usuarios en tomar decisiones informadas sobre su interacción con el *software*. Esto implica garantizar la transparencia en su funcionamiento y en cómo se recopilan, almacenan y utilizan los datos.

Aplicación del marco ético en el ciclo de vida del software

El ciclo de vida del *software* abarca varias fases, desde la concepción hasta el mantenimiento, y en cada una de estas fases se deben considerar los **principios éticos**.

- ▶ **Especificación de requisitos:** en esta fase es esencial identificar y documentar los posibles impactos éticos del *software*. Esto incluye considerar los derechos de los usuarios, las implicaciones sociales y los riesgos potenciales.

- Por ejemplo, en el desarrollo de *software* para vigilancia es necesario evaluar cómo podría afectar la privacidad y las libertades civiles.
- ▶ **Diseño y desarrollo:** durante el diseño los desarrolladores deben tomar decisiones que reflejen los principios éticos establecidos. Esto puede incluir la elección de algoritmos que minimicen el sesgo, la implementación de interfaces accesibles para todos los usuarios y la incorporación de mecanismos para asegurar la protección de datos. La ética del diseño implica anticipar cómo el *software* será utilizado (y posiblemente mal utilizado) y mitigar estos riesgos.
- ▶ **Pruebas y validación:** las pruebas de *software* deben incluir la evaluación de riesgos éticos. Esto podría involucrar pruebas específicas para detectar sesgos en los algoritmos, así como asegurar que funcione correctamente en diferentes contextos de uso.
- Un ejemplo notable es la validación de sistemas de IA utilizados en la justicia penal donde los errores pueden tener consecuencias graves en la vida de las personas.
- ▶ **Implementación y despliegue:** durante la implementación es crucial que se realicen revisiones éticas para asegurar que el *software* cumple con todas las normativas y directrices éticas. Además, se debe proporcionar documentación clara y completa para los usuarios finales para explicar cómo se recopilarán y utilizarán sus datos, así como sus derechos.
- ▶ **Mantenimiento y actualización:** el compromiso ético no termina con el despliegue del *software*. A medida que se descubren nuevos riesgos o se introducen nuevas tecnologías, es esencial actualizar el *software* y las políticas asociadas para mantener la conformidad ética. Esto incluye abordar vulnerabilidades de seguridad que puedan surgir y garantizar que el *software* siga siendo justo y equitativo en su uso continuo.

Desafíos en la aplicación del marco ético

A pesar de la importancia de un marco ético en el desarrollo de *software*, su implementación enfrenta varios **desafíos** (Zarsky, 2016):

- ▶ **Ambigüedad ética:** no siempre es claro cómo aplicar los principios éticos en situaciones complejas. Los desarrolladores pueden enfrentarse a dilemas donde los principios de beneficencia y no maleficencia, por ejemplo, entran en conflicto.
- ▶ **Sesgos inherentes:** el sesgo en los datos de entrenamiento y en los propios desarrolladores puede resultar en un *software* que perpetúa injusticias sociales. Esto es especialmente preocupante en el caso de la IA donde los modelos pueden amplificar sesgos preexistentes.
- ▶ **Falta de regulación:** la rápida evolución de la tecnología a menudo supera la capacidad de los marcos legales y regulatorios para mantener el ritmo, lo que deja un vacío en la gobernanza ética del *software*.
- ▶ **Presiones comerciales:** los plazos de entrega, la competitividad del mercado y otras presiones comerciales pueden llevar a compromisos éticos donde la rapidez y la rentabilidad se priorizan sobre la consideración ética.

Casos prácticos de implementación ética

- ▶ **Mozilla y el *software* de código abierto:** Mozilla, conocida por su navegador Firefox, se ha comprometido con la ética en el desarrollo de *software* a través de su enfoque en la privacidad, la transparencia y la equidad. Mozilla promueve un desarrollo centrado en el usuario y defiende un internet abierto y accesible para todos, lo que refleja sus principios éticos en la toma de decisiones de desarrollo.

- ▶ **Apple y la privacidad:** Apple ha destacado en la industria por su enfoque en la privacidad al diseñar productos y *software* que minimizan la recolección de datos y maximizan la seguridad del usuario. Esto se refleja en características, como el cifrado de extremo a extremo en iMessage y FaceTime y el compromiso de no monetizar datos personales a través de publicidad dirigida.

4.7. Referencias bibliográficas

Ananny, M. y Crawford, K. (2018). *Seeing without knowing: limitations of the transparency ideal and its application to algorithmic accountability*. New Media & Society.

Angwin, J., Larson, J., Mattu, S. y Kirchner, L. (2016). *Machine bias*. ProPublica.

Barocas, S., Hardt, M. y Narayanan, A. (2019). *Fairness and machine learning*. MIT Press.

Bellamy, R. K., Dey, K., Hind, M., Hoffman, S. C., Houde, S., Kannan, K., Pranay, L., Martino, J., Mehta, S., Mojsilovic, A., Nagar, S., Natesan Ramamurthy, K., Richards, J., Saha, D., Sattigeri, P., Singh, M., Varshney, K. R. y Zhang, Y. (2019). AI Fairness 360: An extensible toolkit for detecting and mitigating algorithmic bias. *IBM Journal of Research and Development*, 63(4/5), 4-1.

Buolamwini, J. y Gebru, T. (2018). Gender shades: intersectional accuracy disparities in commercial gender classification. *Proceedings of Machine Learning Research*, 81, 1-15.

Cabitza, F., Rapisarda, R. y Gensini, G. F. (2017). Unintended consequences of machine learning in medicine. *Journal of the American Medical Association*, 318(6), 517-518.

Dastin, J. (2018). *Amazon scraps secret AI recruiting tool that showed bias against women*. Reuters.

Doshi-Velez, F. y Kim, B. (2017). Towards a rigorous science of interpretable machine learning. *Cornell University*.

Godfrey, C. (2020). Legislating big tech: the effects amazon rekognition technology has on privacy rights. *Intellectual Property and Technology Law Journal*, 25, 163.

Goodall, N. J. (2019). Can you program ethics into a self-driving car? *IEEE Spectrum*, 53(6), 28-58.

Hardt, M., Price, E. y Srebro, N. (2016). Equality of opportunity in supervised learning. *Advances in Neural Information Processing Systems*.

Komorowski, M., Celi, L. A., Badawi, O., Gordon, A. C. y Faisal, A. A. (2018). The artificial intelligence clinician learns optimal treatment strategies for sepsis in intensive care. *Nature Medicine*.

Floridi, L., Cows, J., Beltrametti, M., Chatila, R., Chazerand, P., Dignum, V., Luetge, C., Madelin, R., Pagallo, U., Rossi, F., Schafer, B., Valcke, P. y Vayena, E. (2018). AI4People—An ethical framework for a good AI society: Opportunities, risks, principles, and recommendations. *Minds and Machines*, 28(4), 689-707.

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D. y Gebru, T. (2019). Model cards for model reporting. *Proceedings of the conference on fairness, accountability, and transparency*, 220-229.

Morley, J., Cows, J., Taddeo, M. y Floridi, L. (2020). Ethical guidelines for COVID-19 tracing apps. *Nature*, 582(7810), 29-31.

Neff, G. (2016). Talking to bots: symbiotic agency and the case of Tay. *International Journal of Communication*, 10(2016), 4915-4931.

Obermeyer, Z., Powers, B., Vogeli, C. y Mullainathan, S. (2019). Dissecting racial bias in an algorithm used to manage the health of populations. *Science*, 366(6464), 447-453.

Pichai, S. (2018). AI at Google: our principles. *The Keyword*, 7(2018), 1-3.

Shane, S. y Wakabayashi, D. (2018). 'The business of war': Google employees protest work for the Pentagon. *The New York Times*, 4(04).

Wachter, S., Mittelstadt, B. y Floridi, L. (2017). Why a right to explanation of automated decision-making does not exist in the general data protection regulation. *International Data Privacy Law*, 7(2), 76-99.

Wagner, B. (2018). Ethics as an escape from regulation: from ethics-washing to ethics-shopping? (pp. 84-89). En E. Bayamlioglu, I. Baraliuc, L. Janssens & M. Hildebrandt (Ed.), *Being Profiled: cogitas ergo sum: 10 years of profiling the european citizen*. University Press.

West, S. M., Whittaker, M. y Crawford, K. (2019). Discriminating systems: Gender, race, and power in AI. *AI Now Institute*.

Zuboff, S. (2019). *The age of surveillance capitalism: the fight for a human future at the new frontier of power*. PublicAffairs.

Explainable Artificial Intelligence

Gunning, D., Stefik, M., Choi, J., Miller, T., Stumpf, S. y Yang, G. (2019). Explainable artificial intelligence (XAI). DARPA. <https://www.darpa.mil/program/explainable-artificial-intelligence>

Este recurso proviene del programa de Explainable AI (XAI) de DARPA explora técnicas para desarrollar sistemas de IA cuyos resultados sean interpretables y comprensibles para los humanos. El documento analiza cómo la explicabilidad es crucial para generar confianza en los sistemas de IA, lo que permite a los usuarios comprender y auditar decisiones algorítmicas.

IBM AI Fairness

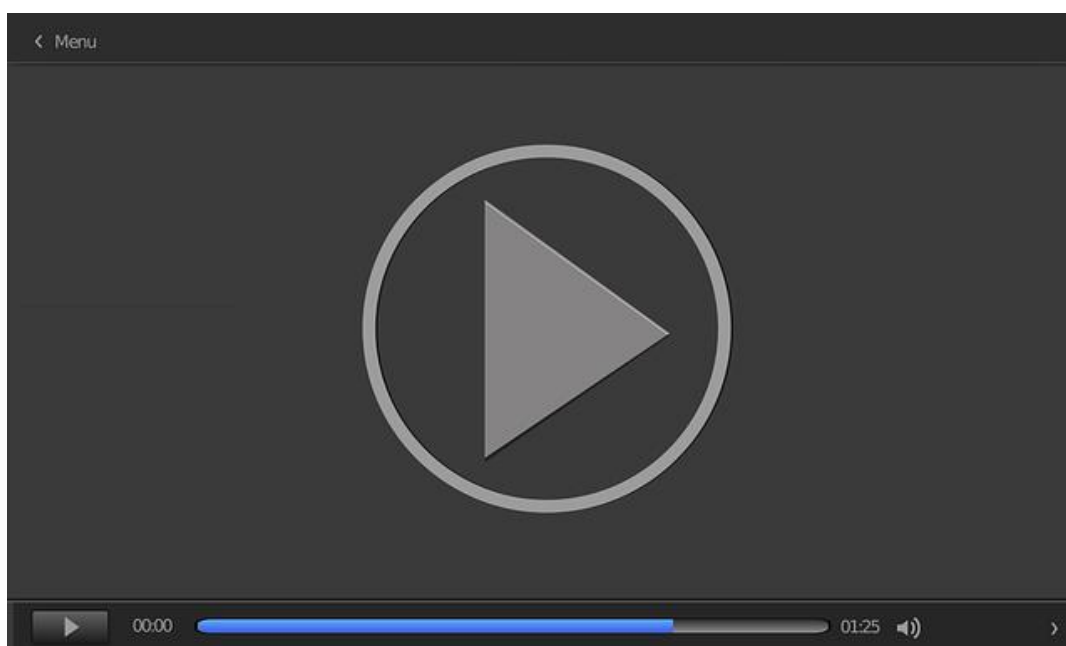
Raji, I. D. y Buolamwini, J. (2019). Actionable auditing: Investigating the impact of publicly naming biased performance results of commercial ai products. *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society*, 7, 429- 435. <https://dl.acm.org/doi/pdf/10.1145/3306618.3314244>

En este artículo de IBM Research se aborda la importancia de la equidad en la IA y presenta un marco de auditoría para detectar y mitigar sesgos en productos comerciales. El documento enfatiza cómo IBM implementa principios de equidad en sus sistemas de IA y se asegura que sus algoritmos operen de manera justa y sin discriminación.

El fin de la realidad

El Confidencial. (2023, marzo 2). *El fin de la realidad: así serán los próximos 10 años de la inteligencia artificial | Control Z Ep 5* [vídeo]. YouTube. https://youtu.be/cXghhnwSW6U?si=QBs7JJ5cF3_4sj04

En este vídeo documental ficticio se analiza una posible evolución de la IA en los próximos 10 años y como su mal uso influye en la sociedad. Fue elaborado en colaboración con diversos expertos en tecnología y nos aporta una visión de las consecuencias de un uso malintencionado o poco ético de la IA y los peligros de la tecnología.



Accede al vídeo:

<https://www.youtube.com/embed/cXghhnwSW6U>

Principios de IA de Google

Pichai, S. (2018). AI at Google: Our Principles. *Google the Keyboard*. <https://www.blog.google/technology/ai/ai-principles/>

Esta publicación describe los principios que guían el desarrollo de la inteligencia artificial en Google. Publicado por Sundar Pichai, CEO de Google, el documento detalla compromisos, como el uso responsable de la IA, la implementación de normas para evitar sesgos y la promoción de la seguridad y la privacidad. Los principios buscan asegurar que la tecnología de IA beneficie a la sociedad y respete los valores éticos fundamentales.

1. ¿Cuál es uno de los principios éticos fundamentales en la inteligencia artificial que busca evitar resultados discriminatorios?
 - A. Beneficencia.
 - B. Responsabilidad.
 - C. Sesgos y equidad.
 - D. Privacidad.

2. ¿Qué principio ético en la IA establece que los desarrolladores deben rendir cuentas por sus decisiones y acciones?
 - A. Transparencia.
 - B. Beneficencia.
 - C. Responsabilidad.
 - D. Privacidad.

3. ¿Por qué es importante la transparencia en los sistemas de IA?
 - A. Para reducir el costo de desarrollo.
 - B. Para generar confianza en la tecnología y permitir la auditoría de los sistemas.
 - C. Para hacer más eficiente el proceso de toma de decisiones.
 - D. Para evitar el uso de datos personales.

4. ¿Cuál de las siguientes es una fuente común de sesgo en los modelos de IA?
 - A. Código de programación abierto.
 - B. Datos de entrenamiento desbalanceados.
 - C. Actualización constante del algoritmo.
 - D. Alta complejidad en el modelo.

5. ¿Cuál es el principio ético que exige la protección de la información personal en sistemas de IA?
- A. Privacidad.
 - B. Beneficencia.
 - C. Responsabilidad.
 - D. Equidad.
6. ¿Qué implica el principio de beneficencia en la IA?
- A. Que la IA debe ser utilizada para promover el bienestar y minimizar los riesgos.
 - B. Que los sistemas de IA deben ser siempre abiertos y accesibles al público.
 - C. Que los desarrolladores no deben rendir cuentas por sus acciones.
 - D. Que la IA no debe involucrar ninguna forma de automatización.
7. ¿Cuál de las siguientes estrategias se utiliza para mitigar los sesgos en los modelos de IA?
- A. Uso exclusivo de datos históricos.
 - B. Entrenamiento con datos desbalanceados.
 - C. Auditoría de algoritmos.
 - D. Eliminación de regularizaciones en el modelo.
8. ¿Qué ocurrió con el sistema de contratación basado en IA de Amazon en 2018?
- A. Fue exitoso y ampliamente adoptado.
 - B. Mostró un sesgo en contra de candidatas mujeres.
 - C. Mejoró la diversidad en la contratación.
 - D. Fue hackeado y sus datos comprometidos.

9. ¿Qué principio se viola cuando un sistema de IA trata de manera diferente a varios grupos de personas en función de características protegidas, como raza o género?
- A. Privacidad.
 - B. Transparencia.
 - C. Discriminación algorítmica.
 - D. Beneficencia.
10. ¿Cuál es una de las técnicas mencionadas para balancear los datos en el entrenamiento de modelos de IA?
- A. Submuestreo.
 - B. Regresión logística.
 - C. Redes neuronales.
 - D. Eliminación de características.
11. ¿Cuál es uno de los principales riesgos de la falta de transparencia en los algoritmos de IA?
- A. Aumento de la eficiencia del sistema.
 - B. Menor costo de desarrollo.
 - C. Desconfianza pública y falta de rendición de cuentas.
 - D. Incremento en la capacidad de predicción.
12. ¿Qué objetivo persigue la implementación de principios éticos en la IA?
- A. Maximizar los beneficios comerciales.
 - B. Asegurar que la IA se desarrolle de manera justa y segura.
 - C. Acelerar la adopción de la IA en todas las industrias.
 - D. Reducir la cantidad de datos necesarios para entrenar modelos.

13. ¿Qué estrategia se recomienda para mejorar la equidad en los sistemas de IA?
 - A. Utilizar solo datos recientes.
 - B. Entrenar el modelo con datos variados y representativos.
 - C. Evitar el uso de datos sensibles.
 - D. Aumentar la complejidad del modelo.

14. ¿Qué principio ético en la IA está directamente relacionado con la protección de los datos personales?
 - A. Equidad.
 - B. Beneficencia.
 - C. Privacidad.
 - D. Transparencia.

15. ¿Cuál es el rol de la explicabilidad en la ética de la IA?
 - A. Hacer que la IA sea más eficiente.
 - B. Permitir que los usuarios entiendan y confíen en las decisiones de la IA.
 - C. Aumentar la velocidad de procesamiento.
 - D. Reducir los costos de implementación.

16. ¿Cuál es un desafío común en la implementación de principios éticos en la IA?
 - A. Falta de datos.
 - B. Dificultad para definir y medir principios, como la equidad y la transparencia.
 - C. Alta velocidad de procesamiento.
 - D. Uso de algoritmos complejos.

17. ¿Cuál de los siguientes es un enfoque para mitigar el sesgo en modelos de IA?
- A. Aumentar la cantidad de datos sesgados.
 - B. Eliminar todas las variables sensibles del modelo.
 - C. Incorporar técnicas de *fairness* durante el entrenamiento del modelo.
 - D. Evitar la supervisión humana en el entrenamiento.
18. ¿Qué es un modelo de caja negra en el contexto de la IA?
- A. Un modelo cuya estructura y funcionamiento es completamente transparente.
 - B. Un sistema de IA cuya toma de decisiones es opaca y difícil de interpretar.
 - C. Un algoritmo que es más rápido, pero menos preciso.
 - D. Un sistema de IA utilizado exclusivamente para propósitos de investigación.
19. ¿Cuál es uno de los objetivos principales de la ética en la IA?
- A. Maximizar el rendimiento técnico.
 - B. Asegurar que los sistemas de IA respeten los derechos humanos.
 - C. Incrementar la cantidad de datos recolectados.
 - D. Reducir los costos de desarrollo de la IA.
20. ¿Cuál de las siguientes opciones es un reto para la ética de la IA en aplicaciones militares?
- A. La mejora de la precisión en el campo de batalla.
 - B. La autonomía de las decisiones tomadas por sistemas de armas basados en IA.
 - C. El aumento de la velocidad en la toma de decisiones.
 - D. El uso exclusivo de IA para misiones de reconocimiento.

Fundamentos de Inteligencia Artificial para Ingenieros de
Software

Tema 5. Python para desarrollo de IA. Uso de Tensorflow y PyTorch

Índice

Esquema

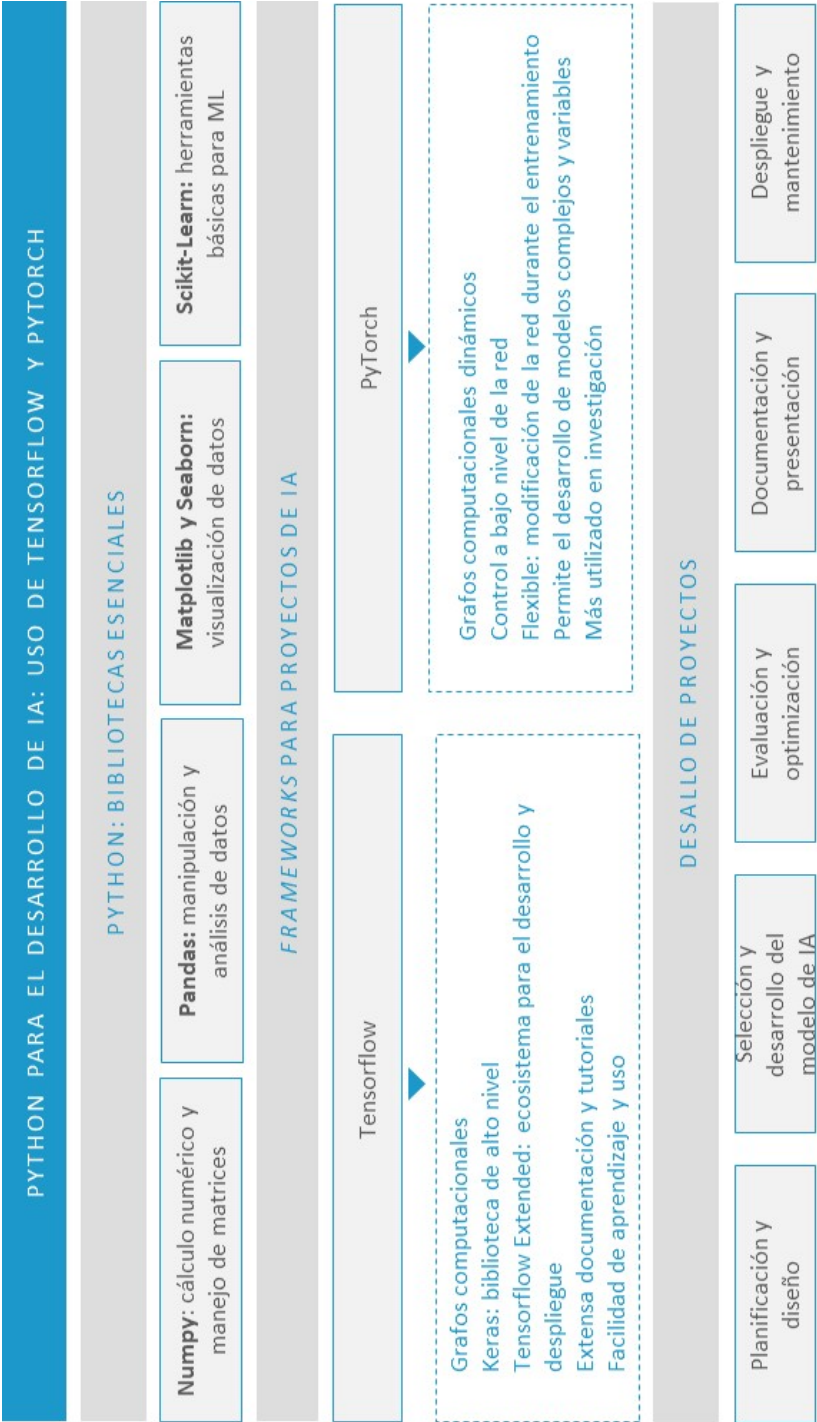
Ideas clave

- 5.1. Introducción y objetivos
- 5.2. Fundamentos de Python para el desarrollo
- 5.3. Uso de TensorFlow y PyTorch en proyectos de IA
- 5.4. Manipulación de datos con NumPy y Pandas
- 5.5. Integración de Python en entornos de desarrollo
- 5.6. Desarrollo de proyectos prácticos en Python
- 5.7. Referencias bibliográficas

A fondo

- Programación con Numpy
- Cómo manejar datos faltantes con Pandas
- Introducción al deep learning con Tensorflow y Keras
- Curso de PyTorch

Test



5.1. Introducción y objetivos

El desarrollo de la inteligencia artificial (IA) ha experimentado un crecimiento significativo en las últimas décadas, y Python ha emergido como uno de los lenguajes de programación más relevantes en este campo. Su popularidad se debe a su sintaxis clara y concisa, así como a la extensa gama de bibliotecas especializadas que permiten abordar todas las fases de un proyecto de IA, desde la manipulación de datos hasta la implementación y despliegue de modelos de aprendizaje profundo.

En este tema se proporciona una guía integral sobre el uso de Python para el desarrollo de proyectos de IA haciendo énfasis en las bibliotecas TensorFlow y PyTorch, que son ampliamente utilizadas para la creación y entrenamiento de modelos de aprendizaje automático y aprendizaje profundo. Se abordarán los fundamentos de Python necesarios para entender y aplicar estas tecnologías, así como las técnicas de manipulación de datos con bibliotecas, como NumPy y Pandas, y la integración de Python en diversos entornos de desarrollo.

Los objetivos del tema son:

- ▶ Comprender el uso de Python en la IA. Esto significa profundizar en los fundamentos de Python que son esenciales para el desarrollo de proyectos de IA, como la manipulación de datos, las estructuras de control y la programación orientada a objetos.
- ▶ Explorar TensorFlow y PyTorch para entender su uso. Estas son dos de las bibliotecas más potentes para el desarrollo de modelos de aprendizaje profundo y debemos saber cómo se integran en los proyectos de IA.
- ▶ Debe haber una aplicación práctica para desarrollar la capacidad para implementar proyectos prácticos de IA utilizando Python y asegurar la adquisición de las competencias necesarias para abordar problemas reales en el campo.

5.2. Fundamentos de Python para el desarrollo

Python es un lenguaje de programación que ha ganado una adopción significativa en el campo de la inteligencia artificial (IA) debido a su **sintaxis simple**, su extensa **comunidad** y la amplia gama de **bibliotecas especializadas** que soportan todo el ciclo de vida de un proyecto de IA, desde la manipulación de datos hasta el despliegue de modelos. Este apartado cubrirá los fundamentos de Python que son esenciales para el desarrollo de IA, incluyendo estructuras de datos, la programación orientada a objeto y el uso de bibliotecas clave.

Sintaxis y estructuras de control

Python es conocido por su sintaxis clara y legible. Entender las **estructuras de control** básicas es fundamental para cualquier tipo de desarrollo en Python y especialmente relevante en la IA, donde se necesitan bucles, condicionales y funciones para manipular datos y entrenar modelos.

A continuación, se muestra un ejemplo básico de estructura de control:

```
# Condicionales

x = 10

if x > 5:

    print("x es mayor que 5")

else:

    print("x es menor o igual a 5")


# Bucle for
```

```
for i in range(5):  
  
    print(f"Iteración {i+1}")  
  
# Funciones  
  
def cuadrado(n):  
  
    return n * n  
  
print(cuadrado(4)) # Output: 16
```

Estas estructuras son la base de las **implementaciones** más **complejas** que se encuentran en proyectos de IA, como la iteración sobre *datasets* y la ejecución condicional de algoritmos.

Estructuras de datos

Las estructuras de datos en Python, como listas, tuplas, conjuntos y diccionarios, son fundamentales para manejar y organizar los datos de manera eficiente. En el contexto de la IA se utilizan frecuentemente para almacenar y procesar grandes volúmenes de datos.

A continuación, se presenta un ejemplo de uso de estructuras de datos:

```
# Lista de números  
  
numeros = [1, 2, 3, 4, 5]  
  
# Diccionario de datos  
  
estudiantes = {"Ana": 22, "Luis": 21, "Maria": 23}
```

```
# Acceso a elementos

print(numeros[2]) # Output: 3

print(estudiantes["Luis"]) # Output: 21

# Añadir y remover elementos

numeros.append(6)

del estudiantes["Ana"]
```

Estas estructuras permiten manejar de manera eficiente los datos que serán utilizados en tareas, como el **preprocesamiento de datos**, la construcción de **conjuntos de entrenamiento** y la **manipulación de resultados**.

Programación orientada a objetos (POO)

La programación orientada a objetos (POO) es un paradigma esencial en Python que permite organizar el código de manera **modular** y **reutilizable**. En el desarrollo de IA, la POO es útil para estructurar componentes, como modelos de *machine learning*, pipelines de procesamiento de datos y sistemas completos de IA.

Ejemplo de Clase en Python:

```
class Perceptron:

    def __init__(self, pesos, bias):

        self.pesos = pesos

        self.bias = bias

    def predecir(self, entrada):
```

```
suma = sum(p * e for p, e in zip(self.pesos, entrada)) + self.bias

return 1 if suma >= 0 else 0
```

```
# Uso de la clase Perceptron
```

```
modelo = Perceptron(pesos=[0.5, -1.5], bias=0.5)
```

```
print(modelo.predecir([1, 0])) # Output: 1
```

En este ejemplo se define una clase `Perceptron`. Un modelo simple de *machine learning* que ilustra cómo encapsular la lógica del modelo dentro de una clase para facilitar la **reutilización** y la **extensión** del código.

Bibliotecas esenciales para la IA

Python cuenta con varias bibliotecas especializadas que son cruciales para el desarrollo de proyectos de IA. Las más importantes incluyen:

- ▶ **NumPy**: biblioteca para cálculos numéricos y manejo de *arrays* multidimensionales.
- ▶ **Pandas**: herramienta poderosa para la manipulación y análisis de datos.
- ▶ **Matplotlib y Seaborn**: bibliotecas para la visualización de datos.
- ▶ **Scikit-learn**: conjunto de herramientas para el aprendizaje automático.
- ▶ **TensorFlow y PyTorch**: *frameworks* para el desarrollo de modelos de aprendizaje profundo.

Ejemplo de uso de NumPy y Pandas:

```
import numpy as np
```

```
import pandas as pd
```

```
# Creación de un array de NumPy
```

```
array = np.array([1, 2, 3, 4, 5])
```

```
# Operación básica
```

```
print(array * 2) # Output: [2 4 6 8 10]
```

```
# Creación de un DataFrame de Pandas
```

```
datos = {'Nombre': ['Ana', 'Luis', 'Maria'], 'Edad': [22, 21, 23]}
```

```
df = pd.DataFrame(datos)
```

```
# Acceso a columnas
```

```
print(df['Nombre']) # Output: ['Ana', 'Luis', 'Maria']
```

Estas bibliotecas son fundamentales para cualquier tipo de desarrollo en IA, desde el preprocesamiento de datos hasta la implementación de complejos modelos de aprendizaje profundo.

Control de flujo y manejo de excepciones

El manejo adecuado de excepciones y el control de flujo son esenciales para construir aplicaciones **robustas** y **seguras**. En la IA es común tener que gestionar errores que pueden surgir durante el procesamiento de datos o el entrenamiento de modelos.

Ejemplo de manejo de excepciones:

```
try:

    resultado = 10 / 0

except ZeroDivisionError:

    print("Error: División por cero.")

finally:

    print("Operación completada.")
```

El uso de bloques `try-except` asegura que los errores se manejen de manera **controlada** mientras permiten que el programa continúe funcionando o proporcione **mensajes de error** claros.

Automatización y scripts

Python es ideal para escribir *scripts* que automatizan **tareas repetitivas**, como la preparación de datos o el entrenamiento de modelos en *batch*. Esta capacidad es fundamental para escalar proyectos de IA y mejorar la **eficiencia** del **desarrollo**.

Ejemplo de *script* para automatizar el **preprocesamiento de datos**:

```
import pandas as pd

# Cargar datos

df = pd.read_csv('datos.csv')

# Limpiar datos
```



```
df = df.dropna()
```

```
# Guardar datos procesados
```

```
df.to_csv('datos_procesados.csv', index=False)
```

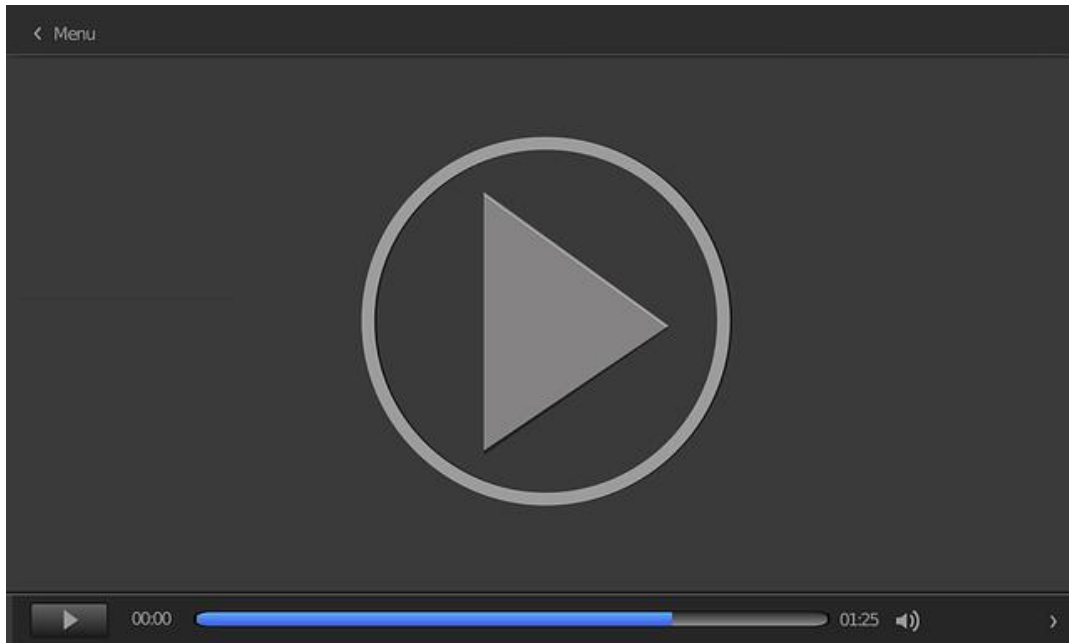
El método `dropna()` de Pandas elimina los **valores nulos** de un conjunto de datos. Esto es útil en las tareas de **preprocesamiento** y **limpieza** de datos

Este *script* sencillo automatiza el proceso de **cargar**, **limpiar** y **guardar** datos, que es un flujo de trabajo común en la preparación de datos para proyectos de IA.

5.3. Uso de TensorFlow y PyTorch en proyectos de IA

TensorFlow y PyTorch son dos de los *frameworks* más populares y potentes para el desarrollo de proyectos de inteligencia artificial (IA) y aprendizaje profundo. Ambos ofrecen una amplia gama de herramientas y bibliotecas para **construir, entrenar y desplegar** modelos de IA, pero presentan diferencias en cuanto a su enfoque y características que pueden influir en la elección de uno u otro dependiendo del proyecto.

El vídeo *Configuración del entorno Phyton para Tensorflow* muestra cómo descargar e instalar los *frameworks* de Tensorflow y Pytorch para el desarrollo de herramientas de IA, así como las librerías necesarias para la computación mediante GPU y TPU.



Configuración del entorno Phyton para Tensorflow

Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=c27ca564-bf55-4677-a83d-b1de0098531d>

TensorFlow

TensorFlow, desarrollado por Google Brain, es un *framework* de código abierto ampliamente utilizado en la investigación y en la industria para la implementación de redes neuronales profundas. Se caracteriza por su **flexibilidad**, **escalabilidad** y por un amplio **soporte** en producción. Sus características principales son:

- ▶ **Grafos computacionales:** TensorFlow permite la construcción de modelos a través de grafos computacionales, lo que facilita la optimización y el despliegue en múltiples plataformas, desde CPU y GPU hasta dispositivos móviles.
- ▶ **TensorFlow Extended (TFX):** ofrece un ecosistema completo para el desarrollo y despliegue de modelos de *machine learning* (ML). Incluyendo herramientas para la preparación de datos, el entrenamiento de modelos y el monitoreo.
- ▶ **Keras:** integrado en TensorFlow, Keras es una API de alto nivel que simplifica la construcción y el entrenamiento de redes neuronales.

Ejemplo de código en TensorFlow:

```
import tensorflow as tf

from tensorflow.keras import layers, models

# Definir un modelo simple de red neuronal

model = models.Sequential([

    layers.Dense(64, activation='relu', input_shape=(784,)),

    layers.Dense(64, activation='relu'),

    layers.Dense(10, activation='softmax')

])
```

```
# Compilar el modelo

model.compile(optimizer='adam',

              loss='sparse_categorical_crossentropy',

              metrics=['accuracy'])
```

```
# Entrenar el modelo

model.fit(train_data, train_labels, epochs=5)
```

En este ejemplo se define y entrena una **red neuronal simple** para la clasificación utilizando la API de Keras dentro de TensorFlow. Se utiliza `Dense` para crear capas completamente conectadas y se **compila** el modelo con el optimizador Adam y la función de **pérdida de entropía** cruzada categórica.

PyTorch

PyTorch, desarrollado por Facebook AI Research lab (FAIR), ha ganado popularidad en la comunidad de investigación debido a su **flexibilidad** y **facilidad** de uso. A diferencia de TensorFlow, PyTorch utiliza **grafos computacionales dinámicos**, lo que significa que los grafos se construyen sobre la marcha durante la ejecución. Esto facilita la depuración y el desarrollo de modelos más complejos. Entre sus características principales destacan:

- ▶ **Autograd:** PyTorch tiene un sistema de diferenciación automática llamado Autograd que permite calcular gradientes automáticamente, lo que facilita la implementación de algoritmos de entrenamiento.
- ▶ **Flexibilidad:** es particularmente apreciado en entornos de investigación debido a su enfoque más «pythonico» y su capacidad de integración con otras bibliotecas de Python.
- ▶ **TorchScript:** permite la conversión de modelos de PyTorch a un formato que se puede optimizar y ejecutar en producción.

Ejemplo de código en PyTorch:

```
import torch

import torch.nn as nn

import torch.optim as optim

# Definir un modelo simple de red neuronal

class SimpleNN(nn.Module):

    def __init__(self):
```

```
super(SimpleNN, self).__init__()

self.fc1 = nn.Linear(784, 64)

self.fc2 = nn.Linear(64, 64)

self.fc3 = nn.Linear(64, 10)


def forward(self, x):

    x = torch.relu(self.fc1(x))

    x = torch.relu(self.fc2(x))

    x = self.fc3(x)

    return x


model = SimpleNN()


# Definir el optimizador y la función de pérdida

optimizer = optim.Adam(model.parameters(), lr=0.001)

criterion = nn.CrossEntropyLoss()


# Entrenar el modelo

for epoch in range(5):

    for data, target in train_loader:
```

```
optimizer.zero_grad()

output = model(data)

loss = criterion(output, target)

loss.backward()

optimizer.step()
```

En este ejemplo se define un modelo simple de red neuronal en PyTorch. Se utilizan capas lineales (`Linear`), se define un optimizador Adam y se entrena el modelo utilizando un ciclo de entrenamiento explícito.

Comparación y selección en proyectos de IA

Ambos *frameworks* han sido ampliamente aceptados y utilizados dentro de la comunidad científica y desarrolladora en IA. Sin embargo, existen ciertas **diferencias** que cabe mencionar y que permiten al desarrollador escoger el *framework* más adecuado para su proyecto:

- ▶ **Investigación vs. producción:** PyTorch es frecuentemente preferido en entornos de investigación debido a su flexibilidad y facilidad de uso, mientras que TensorFlow es más comúnmente utilizado en entornos de producción, especialmente debido a su robusto soporte para el despliegue en diversas plataformas.
- ▶ **Curva de aprendizaje:** PyTorch suele ser más intuitivo para los programadores de Python gracias a su sintaxis más natural. TensorFlow, aunque más complejo, ofrece un ecosistema más completo y una mayor cantidad de recursos y herramientas.

- ▶ **Compatibilidad y ecosistema:** TensorFlow con su ecosistema TFX es ideal para proyectos que requieren un *pipeline* completo de *machine learning*, desde el preprocesamiento de datos hasta el despliegue. PyTorch con TorchScript también ha mejorado su capacidad para el despliegue en producción, pero aún es más utilizado en investigación.

5.4. Manipulación de datos con NumPy y Pandas

La manipulación de datos es una parte fundamental en el desarrollo de proyectos de inteligencia artificial (IA). Antes de entrenar modelos es necesario **preprocesar** y **analizar** los datos para asegurar que estén en un **formato** adecuado y que se hayan eliminado o corregido posibles **inconsistencias**. NumPy y Pandas son dos bibliotecas esenciales en Python que facilitan estas tareas.

NumPy: manipulación de datos numéricos

NumPy (Numerical Python) es una biblioteca que proporciona **soporte** para matrices y *arrays* **multidimensionales** junto con una colección de **funciones matemáticas** de alto rendimiento. Es la base sobre la cual se construyen muchas otras bibliotecas en Python, incluyendo Pandas, TensorFlow y PyTorch.

Las principales características de NumPy son:

- ▶ **Arrays multidimensionales:** NumPy permite la creación y manipulación eficiente de *arrays* de grandes volúmenes de datos numéricos.
- ▶ **Operaciones vectorizadas:** las operaciones sobre *arrays* en NumPy están vectorizadas, lo que significa que se aplican a todo el *array* de una sola vez sin necesidad de escribir bucles explícitos, lo que mejora el rendimiento.
- ▶ **Funcionalidades matemáticas:** incluye funciones para álgebra lineal, transformadas de Fourier, generación de números aleatorios, entre otras.

Ejemplo de código en NumPy:

```
import numpy as np
```

```
# Crear un array de NumPy
```

```
data = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Operaciones básicas

sum_data = np.sum(data) # Sumar todos los elementos

mean_data = np.mean(data) # Calcular la media

transposed_data = np.transpose(data) # Transponer la matriz

print("Suma de los elementos:", sum_data)

print("Media de los elementos:", mean_data)

print("Matriz transpuesta:\n", transposed_data)
```

En este ejemplo se crea una matriz 3x3 utilizando NumPy. Se calcula la suma y la media de todos los elementos y se obtiene la **transposición** de la matriz. Estas operaciones muestran cómo NumPy facilita la **manipulación** de datos numéricos de manera eficiente.

Pandas: manipulación de datos tabulares

Pandas es una biblioteca construida sobre NumPy que proporciona estructuras de datos y herramientas de alto nivel diseñadas para facilitar el **análisis** y la **manipulación** de datos tabulares, como tablas de bases de datos o hojas de cálculo.

Sus principales características son:

- **Series y dataframes:** las estructuras de datos principales en Pandas son las **Series** (una columna) y los **DataFrames** (una tabla de múltiples columnas), que permiten manejar datos heterogéneos de forma similar a las tablas de bases de datos.

- ▶ **Manejo de datos faltantes:** Pandas ofrece potentes herramientas para la detección, manipulación y corrección de datos faltantes o nulos.
- ▶ **Operaciones de filtrado y agrupación:** facilita la realización de operaciones de filtrado, agrupación, agregación y transformación de datos de forma intuitiva.

Ejemplo de código en Pandas:

```
import pandas as pd

# Crear un DataFrame en Pandas

data = {'Name': ['Alice', 'Bob', 'Charlie', 'David'],
        'Age': [24, 27, 22, 32],
        'Score': [85, 62, 95, 70]}

df = pd.DataFrame(data)

# Operaciones básicas

mean_age = df['Age'].mean() # Calcular la media de la edad

filtered_df = df[df['Score'] > 70] # Filtrar filas con puntuaciones superiores a 70

print("Media de la edad:", mean_age)

print("Filtrado de puntuaciones mayores a 70:\n", filtered_df)
```

En este ejemplo se crea un `DataFrame` en Pandas con los nombres, edades y puntuaciones de un grupo de personas. Luego, se calcula la media de las edades y se filtran las filas donde las puntuaciones son mayores a 70.

Comparación y uso combinado

NumPy es ideal para realizar **operaciones numéricas intensivas** en grandes conjuntos de datos, mientras que Pandas es más adecuado para **manipular** y **analizar** datos tabulares, que pueden tener diferentes tipos (numéricos, categóricos, etc.). En muchos proyectos de IA ambas bibliotecas se utilizan conjuntamente:

- ▶ **Preprocesamiento de datos:** se puede utilizar NumPy para realizar operaciones matemáticas y transformaciones en matrices numéricas grandes y luego Pandas para organizar esos datos en un *dataframe* para su análisis o limpieza.
- ▶ **Conversión entre NumPy y Pandas:** es común convertir entre *arrays* de NumPy y *dataframes* de Pandas para aprovechar las fortalezas de cada biblioteca.

Ejemplo de Conversión entre NumPy y Pandas:

```
# De NumPy a Pandas
```

```
np_array = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
df_from_np = pd.DataFrame(np_array, columns=['A', 'B', 'C'])
```

```
# De Pandas a NumPy
```

```
np_from_df = df_from_np.to_numpy()
```

```
print("DataFrame creado desde NumPy:\n", df_from_np)\n\nprint("Array NumPy creado desde DataFrame:\n", np_from_df)
```

Este código demuestra cómo convertir un *array* de NumPy en un *dataframe* de Pandas y viceversa al integrar las capacidades de ambas bibliotecas para el **manejo y análisis de datos**.

NumPy y Pandas son esenciales en cualquier **flujo de trabajo** de IA debido a su capacidad para manejar y manipular datos de manera eficiente y flexible. Los ejemplos presentados ilustran cómo estas bibliotecas pueden ser utilizadas para preparar y analizar datos antes de la etapa de **modelado** en proyectos de **inteligencia artificial**.

5.5. Integración de Python en entornos de desarrollo

Python se ha consolidado como un lenguaje fundamental en el desarrollo de proyectos de inteligencia artificial (IA) debido a su **simplicidad**, **versatilidad** y la vasta colección de **bibliotecas especializadas**. Sin embargo, para maximizar su potencial es crucial integrar Python de manera eficiente en **diversos entornos** de desarrollo que pueden incluir: entornos locales, servidores remotos, sistemas de nube y plataformas de despliegue. Esta integración abarca desde la **configuración inicial** del entorno hasta el **despliegue** y **mantenimiento** continuo de las aplicaciones de IA.

Entornos virtuales y gestión de dependencias

Uno de los primeros pasos en la integración de Python en cualquier entorno de desarrollo es la gestión de **dependencias** y la creación de **entornos virtuales**. Esto es fundamental para evitar conflictos entre diferentes proyectos que puedan requerir versiones distintas de las mismas bibliotecas.

Creación de un **entorno virtual** con `venv`:

```
# Crear un entorno virtual
```

```
python3 -m venv mi_entorno
```

```
# Activar el entorno virtual (Linux/macOS)
```

```
source mi_entorno/bin/activate
```

```
# Activar el entorno virtual (Windows)
```

```
mi_entorno\Scripts\activate
```

```
# Instalar dependencias
```

```
pip install tensorflow numpy pandas
```

En este ejemplo se crea un entorno virtual llamado `mi\_entorno` que luego se activa para instalar dependencias específicas. Esto permite aislar el entorno de desarrollo de Python y asegura que las bibliotecas instaladas no afecten a otros proyectos.

Entornos de desarrollo integrados (IDE)

Los entornos de desarrollo integrados (IDE) son herramientas que facilitan la **escritura, depuración y gestión** de código en Python. Algunos de los IDE más populares para el desarrollo de proyectos de IA incluyen PyCharm, Visual Studio Code y Jupyter Notebook.

- ▶ **PyCharm:** es un IDE robusto que ofrece soporte avanzado para Python e incluye la integración con sistemas de control de versiones (Git), herramientas de depuración, gestión de entornos virtuales y soporte para *frameworks* de IA, como TensorFlow y PyTorch.
- ▶ **Visual Studio Code (VS Code):** es un editor de código ligero, pero potente, que puede ser ampliado con extensiones para Python, Jupyter Notebooks y otros lenguajes o herramientas. Esto lo convierte en un entorno de desarrollo completo.
- ▶ **Jupyter Notebook:** aunque no es un IDE tradicional, es extremadamente popular en el desarrollo de proyectos de IA debido a su capacidad para combinar código, visualizaciones y texto en un solo documento interactivo.

Ejemplo de configuración en VS Code:

- Instalar la extensión de Python.

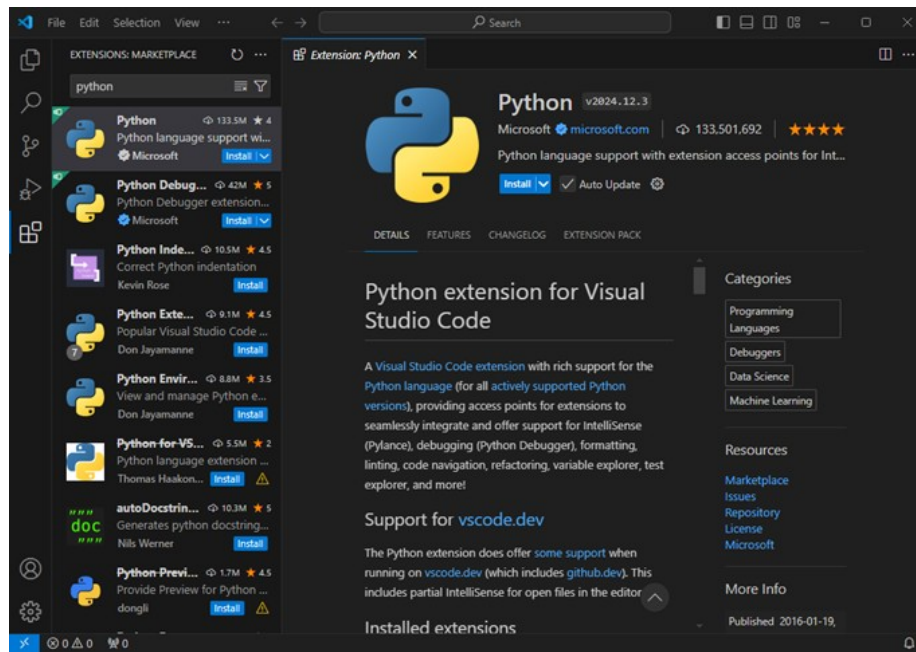


Figura 1. Extensiones básicas de Python para VS Code. Fuente: elaboración propia.

- Configurar el entorno Python (por ejemplo, un entorno virtual).

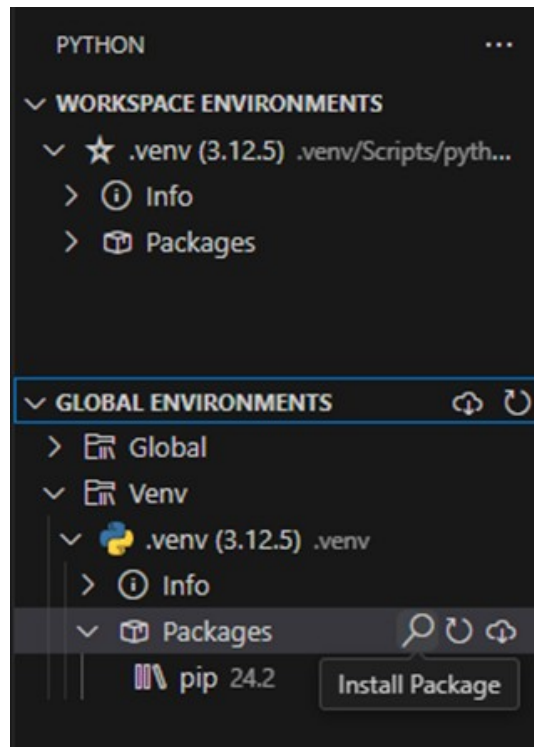


Figura 2. Configuración de entorno e instalación de paquetes. Fuente: elaboración propia.

- Utilizar el terminal integrado para ejecutar *scripts* de Python.

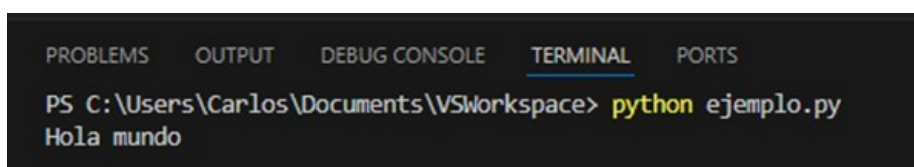


Figura 3. Ejemplo de uso del terminal integrado en VS Code. Fuente: elaboración propia.

- ▶ Aprovechar las características de IntelliSense para autocompletar el código y la depuración de errores.

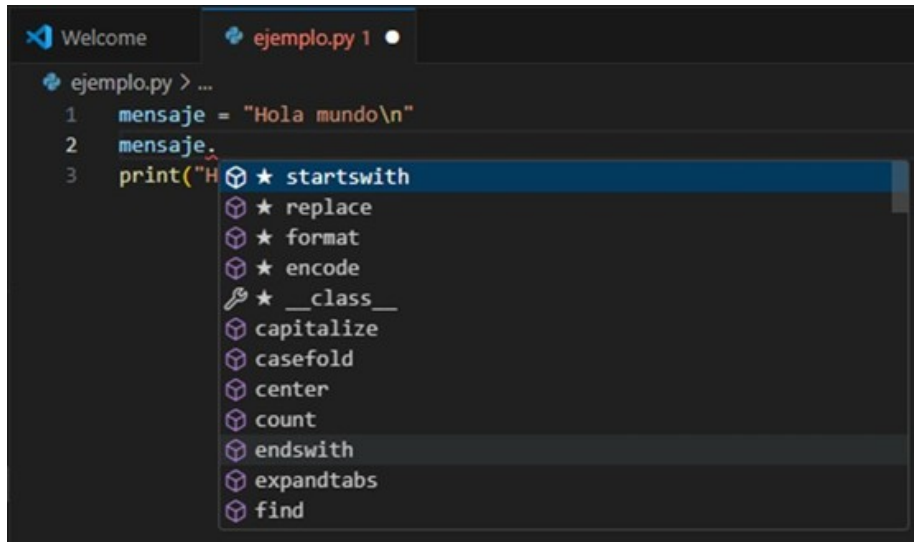


Figura 4. Uso de IntelliSense con Python para sugerencias y autocompletado de código. Fuente: elaboración propia.

Despliegue en la nube y automatización

Con la creciente demanda de **escalabilidad** y **disponibilidad**, desplegar aplicaciones de IA en la **nube** se ha vuelto común. Plataformas como AWS, Google Cloud Platform (GCP) y Microsoft Azure ofrecen servicios para entrenar modelos de IA y desplegar aplicaciones basadas en Python a gran escala.

- ▶ **AWS SageMaker:** proporciona un entorno completo para el desarrollo y despliegue de modelos de *machine learning*. Incluye herramientas para la preparación de datos, la experimentación con modelos y su implementación en producción.
- ▶ **Google AI Platform:** ofrece servicios similares con una integración nativa con TensorFlow y otras herramientas de Google, lo que permite un flujo de trabajo continuo desde el entrenamiento hasta el despliegue.

- **Azure Machine Learning:** proporciona una plataforma completa para desarrollar, entrenar y desplegar modelos de IA con soporte para una amplia gama de *frameworks*, incluyendo PyTorch y TensorFlow.

Ejemplo de despliegue automático con CI/CD

Integrar Python en un *pipeline* de CI/CD (integración continua/despliegue continuo) permite **automatizar** el proceso de **pruebas**, **integración** y **despliegue** de aplicaciones de IA. Algunas herramientas, como Jenkins, Travis CI o GitHub Actions, pueden ser configuradas para ejecutar test automáticamente cada vez que se realiza un *commit* y desplegar la aplicación en un entorno de producción si las pruebas son exitosas. A continuación, se muestra cómo se podría realizar dicha integración en GitHub Actions:

Ejemplo de pipeline de CI/CD en GitHub Actions

```
name: CI/CD Pipeline
```

```
on: [push]
```

```
jobs:
```

```
  build:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v2
```

```
      - name: Set up Python
```

```
        uses: actions/setup-python@v2
```

```
        with:
```

```
python-version: 3.8

- name: Install dependencies

  run: |

    python -m pip install --upgrade pip

    pip install -r requirements.txt

- name: Run tests

  run: |

    pytest
```

Este *pipeline* simple en GitHub Actions automatiza la instalación de **dependencias** y la ejecución de **pruebas** cada vez que se realiza un *push* al repositorio. Esto facilita el despliegue continuo.

Consideraciones de seguridad y buenas prácticas

Al integrar Python en entornos de desarrollo es crucial considerar aspectos de **seguridad**, como la gestión de credenciales, la configuración de permisos en la nube y la implementación de prácticas de **codificación** seguras para prevenir vulnerabilidades.

- ▶ **Almacenamiento seguro de credenciales:** evitar incluir credenciales en el código fuente. Usar servicios, como AWS Secrets Manager o Azure Key Vault, para gestionar credenciales y secretos de forma segura.

- ▶ **Control de acceso y permisos:** configurar roles y permisos adecuados en plataformas de nube para limitar el acceso a recursos críticos.
- ▶ **Pruebas de seguridad:** incluir pruebas de seguridad automatizadas en el *pipeline* de CI/CD para detectar vulnerabilidades en el código.

5.6. Desarrollo de proyectos prácticos en Python

El desarrollo de proyectos prácticos en Python es una manera efectiva de consolidar **conocimientos teóricos** y adquirir **habilidades** aplicadas en la programación, especialmente en el contexto de la inteligencia artificial (IA). Python, debido a su simplicidad y amplia gama de bibliotecas, es ideal para implementar una variedad de proyectos prácticos que abarcan desde **análisis de datos** hasta el desarrollo de **modelos de aprendizaje** profundo.

Selección y planificación del proyecto

El primer paso en el desarrollo de un proyecto práctico es seleccionar una **idea** que esté alineada con los **objetivos de aprendizaje** y los intereses del desarrollador. Los proyectos pueden variar desde simples análisis de datos hasta complejas implementaciones de modelos de IA. La planificación adecuada incluye la **definición** de los objetivos, la recopilación de **requisitos**, la elección de las **herramientas** y tecnologías necesarias y la creación de un **cronograma** de trabajo.

Ejemplos de proyectos:

- ▶ **Análisis exploratorio de datos (EDA):** utilizar *datasets* públicos, como el conjunto de datos de viviendas de Boston, para realizar un análisis exploratorio que identifique patrones y relaciones en los datos.
- ▶ **Clasificación de imágenes con redes neuronales convolucionales (CNN):** implementar un modelo de CNN utilizando TensorFlow o PyTorch para clasificar imágenes de un *dataset*, como CIFAR-10 o MNIST.
- ▶ **Sistema de recomendación:** crear un sistema de recomendación simple utilizando técnicas de filtrado colaborativo o basado en contenido.

Configuración del entorno de desarrollo

Una vez que el proyecto está planificado, es esencial configurar el **entorno de desarrollo**. Esto incluye la instalación de Python, la configuración de un entorno virtual y la instalación de las bibliotecas necesarias.

Instalación y configuración básica:

```
# Crear un entorno virtual
```

```
python3 -m venv proyecto_env
```

```
# Activar el entorno virtual
```

```
source proyecto_env/bin/activate # Linux/MacOS
```

```
proyecto_env\Scripts\activate # Windows
```

```
# Instalar las bibliotecas necesarias
```

```
pip install numpy pandas matplotlib scikit-learn tensorflow
```

Este entorno básico incluye herramientas esenciales para el **análisis** de datos, **visualización** y **desarrollo** de modelos de IA.

Desarrollo y prototipado

Durante esta etapa se realiza la implementación del **código base** del proyecto. Para un flujo de trabajo eficiente es recomendable seguir un **enfoque iterativo** donde se desarrollan pequeños prototipos que se prueban y mejoran continuamente.

A continuación, se presenta un ejemplo de desarrollo de un clasificador de imágenes con TensorFlow:

```
import tensorflow as tf

from tensorflow.keras import layers, models

import matplotlib.pyplot as plt

# Cargar y preprocesar datos (usando CIFAR-10 como ejemplo)

(train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.cifar10.load_data()

train_images, test_images = train_images / 255.0, test_images / 255.0

# Definir el modelo

model = models.Sequential([

    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),

    layers.Flatten(),

    layers.Dense(64, activation='relu'),

    layers.Dense(10)
```

```
1)

# Compilar el modelo

model.compile(optimizer='adam',

loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),

               metrics=['accuracy'])

# Entrenar el modelo

history = model.fit(train_images, train_labels, epochs=10,

                    validation_data=(test_images, test_labels))

# Evaluar y visualizar resultados

plt.plot(history.history['accuracy'], label='accuracy')

plt.plot(history.history['val_accuracy'], label = 'val_accuracy')

plt.xlabel('Epoch')

plt.ylabel('Accuracy')

plt.ylim([0, 1])

plt.legend(loc='lower right')

plt.show()
```

En este ejemplo se entrena un clasificador de imágenes utilizando TensorFlow. El modelo se entrena en el conjunto de datos CIFAR-10 y se visualiza la precisión del **entrenamiento** y la **validación** a lo largo de las épocas.

Evaluación y optimización

Una vez que el prototipo del proyecto está funcional, es necesario evaluar su desempeño y realizar **optimizaciones**. Esto puede incluir la mejora de la **precisión** del modelo, la reducción del tiempo de **ejecución** o la **simplificación** del código para facilitar su mantenimiento.

Las técnicas de optimización más habituales son:

- ▶ **Ajuste de hiperparámetros:** modificar parámetros, como la tasa de aprendizaje, la estructura del modelo o el tamaño del lote, para mejorar el rendimiento del modelo.
- ▶ **Regularización:** aplicar técnicas, como Dropout o L2 Regularization, para prevenir el sobreajuste.
- ▶ **Optimización de código:** identificar y mejorar las partes del código que consumen más recursos utilizando técnicas como la vectorización o la paralelización.

A continuación, se presenta un ejemplo de ajuste de hiperparámetros con Keras Tuner:

```
import keras_tuner as kt

def build_model(hp):

    model = models.Sequential()

    model.add(layers.Conv2D(hp.Int('conv_units', min_value=32,
max_value=128, step=32),
```

```

        (3, 3), activation='relu', input_shape=(32, 32,
3)))

    model.add(layers.MaxPooling2D((2, 2)))

    model.add(layers.Flatten())

    model.add(layers.Dense(hp.Int('dense_units', min_value=32,
max_value=128, step=32),

        activation='relu'))

    model.add(layers.Dense(10))

    model.compile(optimizer='adam',

loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),

        metrics=['accuracy'])

    return model

tuner = kt.Hyperband(build_model, objective='val_accuracy', max_epochs=10)

tuner.search(train_images, train_labels, epochs=10, validation_data=
(test_images, test_labels))

```

Este ejemplo utiliza Keras Tuner para realizar un **ajuste de hiperparámetros** en un modelo de CNN. Se busca la mejor configuración para maximizar la precisión en el conjunto de validación.

Documentación y presentación

Un aspecto crucial de cualquier proyecto es la **documentación adecuada**, que debe incluir:

- ▶ **Descripción del proyecto:** objetivos, motivación y resumen de la solución implementada.
- ▶ **Instrucciones de uso:** cómo configurar el entorno, ejecutar el código y reproducir los resultados.
- ▶ **Análisis de resultados:** gráficos, tablas y explicaciones de los resultados obtenidos y las decisiones tomadas durante el desarrollo.

Los Jupyter Notebooks son ideales para proyectos de IA, ya que permiten combinar **código, visualizaciones y texto** en un solo documento interactivo. Esto facilita la documentación en tiempo real a medida que se desarrolla el proyecto.

Despliegue y mantenimiento

El despliegue de un proyecto de IA puede implicar llevar el modelo a la producción, que es donde será utilizado en un **entorno real**. Dependiendo del proyecto, esto puede involucrar la creación de una **API** para acceder al modelo, la integración en una aplicación web o móvil o su despliegue en la nube.

A continuación, se presenta un ejemplo de despliegue de un modelo como API con Flask:

```
from flask import Flask, request, jsonify

import tensorflow as tf

app = Flask(__name__)
```

```
# Cargar el modelo preentrenado

model = tf.keras.models.load_model('model.h5')

@app.route('/predict', methods=['POST'])

def predict():

    data = request.json

    predictions = model.predict(data['input'])

    return jsonify(predictions.tolist())

if __name__ == '__main__':

    app.run(debug=True)
```

Este código muestra cómo desplegar un modelo de IA como una API REST utilizando Flask, lo que permite a otros servicios o aplicaciones enviar datos al modelo y recibir **predicciones en tiempo real**.

El desarrollo de proyectos prácticos en Python no solo refuerza la comprensión teórica, sino que también proporciona una **experiencia** valiosa en la implementación de **soluciones reales**, la **optimización** de modelos y el despliegue de **aplicaciones** en entornos de producción. Cada paso desde la planificación hasta el mantenimiento contribuye a la formación de un conjunto integral de **habilidades** en el desarrollo del *software* y modelos de inteligencia artificial.

5.7. Referencias bibliográficas

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J. y Zheng, X. (2016). *TensorFlow: A system for large-scale machine learning* (pp. 265-283). 12th USENIX symposium on operating systems design and implementation (OSDI 16), Georgia, Estados Unidos.

Downey, A. B. (2009). *Python for software design: how to think like a computer scientist*. Cambridge University Press Textbooks.

Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media, Inc.

Harris, C. R., Millman, K. J., Van Der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D. y Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362.

Lutz, M. (2013). *Learning python: powerful object-oriented programming*. O'Reilly Media, Inc.

McKinney, W. (2010). Data structures for statistical computing in Python. *SciPy*, 445(1), 51-56.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G. y Chintala, S. (2019). Pytorch: an imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.

Sweigart, A. (2019). *Automate the boring stuff with Python: practical programming for total beginners*. No Starch Press.

VanderPlas, J. (2016). *Python data science handbook: essential tools for working with data*. O'Reilly Media, Inc.

Programación con Numpy

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H. y Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>

Este artículo científico detalla las capacidades de NumPy como herramienta fundamental para la manipulación de *arrays* multidimensionales en Python. Ofrece una visión profunda sobre cómo NumPy facilita operaciones matemáticas complejas y optimizadas y es un recurso clave para cualquier proyecto de inteligencia artificial.

Cómo manejar datos faltantes con Pandas

Brownlee, J. (2023). *How to handle missing data in Python*. Machine Learning Mastery. <https://machinelearningmastery.com/handle-missing-data-python/>

Este artículo de Machine Learning Mastery explica diversas técnicas para manejar los datos faltantes en Python utilizando la biblioteca Pandas. Es un complemento ideal para aquellos que buscan profundizar en la manipulación y preprocesamiento de datos antes de aplicarlos en modelos de IA.

Introducción al deep learning con Tensorflow y Keras

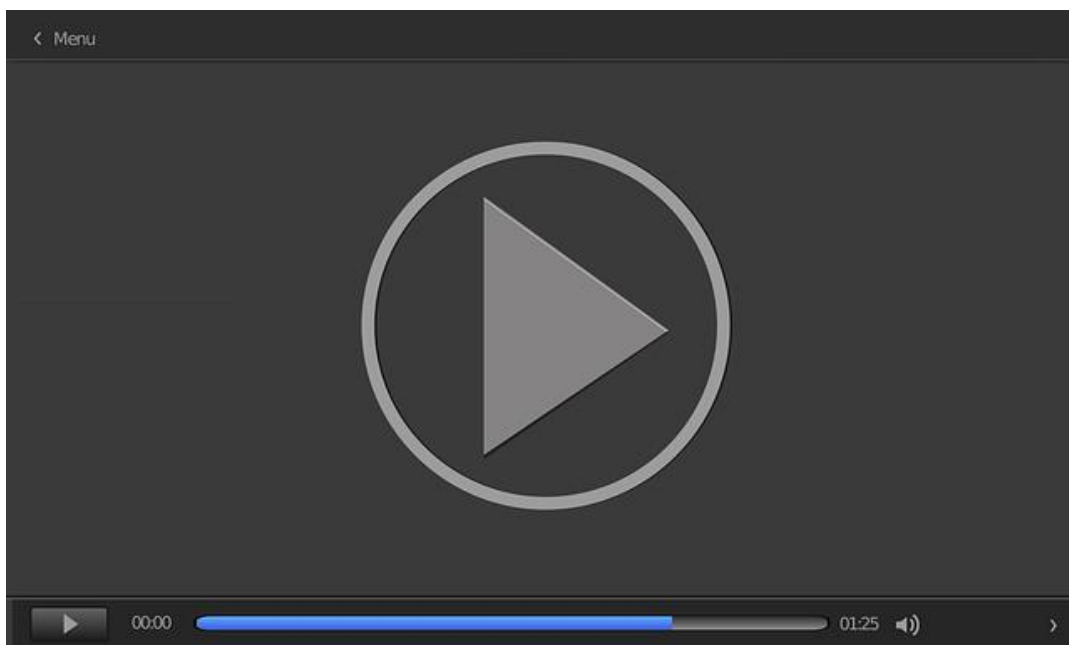
Chollet, F. (2018). *Deep learning with Python*. Simon and Schuster.
<https://www.mangoud.com/wp-content/uploads/2020/11/Francois-Chollet-Deep-Learning-with-Python-2018-Manning.pdf>

Este libro escrito por el creador de Keras ofrece una introducción práctica al *deep learning* utilizando TensorFlow y Keras. Es ideal para aquellos que desean profundizar en la construcción de modelos de aprendizaje profundo y complementar los conceptos básicos con ejemplos avanzados y casos de uso.

Curso de PyTorch

Sensio, J. (2020, agosto 17). *Pytorch – introducción* [vídeo]. YouTube. <https://youtu.be/WL50sQVdQFg?si=IntXdGZC6VEfliea>

En este conjunto de vídeos se ofrece un tutorial completo sobre el uso de PyTorch para el desarrollo de modelos de aprendizaje profundo.



Accede al vídeo:

<https://www.youtube.com/embed/WL50sQVdQFg>

1. ¿Cuál es una de las principales razones por las que Python es popular en el desarrollo de IA?
 - A. Es el lenguaje más rápido.
 - B. Tiene una sintaxis compleja.
 - C. Posee una amplia gama de bibliotecas especializadas.
 - D. Es un lenguaje orientado a la web.

2. ¿Qué estructura de datos en Python se utiliza para almacenar datos en pares clave-valor?
 - A. Lista.
 - B. Tupla.
 - C. Conjunto.
 - D. Diccionario.

3. ¿Cuál es la función de la biblioteca NumPy en proyectos de IA?
 - A. Manejar datos en formato JSON.
 - B. Manipular *arrays* multidimensionales.
 - C. Crear aplicaciones web.
 - D. Realizar pruebas unitarias.

4. ¿Cuál de las siguientes es una biblioteca para la visualización de datos en Python?
 - A. TensorFlow.
 - B. PyTorch.
 - C. Matplotlib.
 - D. Scikit-learn.

5. ¿Qué paradigma de programación es esencial en Python para la modularidad y reutilización del código en IA?
- A. Programación funcional.
 - B. Programación estructurada.
 - C. Programación orientada a objetos.
 - D. Programación procedural.
6. ¿Qué hace el método `forward` en una clase de PyTorch?
- A. Inicializa los parámetros del modelo.
 - B. Ejecuta la pasada hacia adelante en la red neuronal.
 - C. Optimiza los hiperparámetros.
 - D. Calcula la función de pérdida.
7. ¿Cuál de los siguientes bloques de código en Python maneja excepciones?
- A. `if-else` .
 - B. `for-loop` .
 - C. `try-except` .
 - D. `while-loop` .
8. ¿Cuál es una ventaja de usar PyTorch sobre TensorFlow?
- A. Mejor soporte en producción.
 - B. Uso de grafos computacionales estáticos.
 - C. Facilidad de depuración con grafos dinámicos.
 - D. Mayor soporte de dispositivos móviles.

9. ¿Qué biblioteca se utiliza para manipular y analizar datos en Python?
- A. NumPy.
 - B. Pandas.
 - C. Matplotlib.
 - D. TensorFlow.
10. ¿Cuál es una característica principal de TensorFlow?
- A. Autograd.
 - B. Grafos computacionales.
 - C. TorchScript.
 - D. Optimización de *hardware*.
11. ¿Cuál es la función de la biblioteca Scikit-learn?
- A. Visualizar datos.
 - B. Manejar *arrays* multidimensionales.
 - C. Automatizar tareas repetitivas.
 - D. Proveer herramientas de aprendizaje automático.
12. ¿Qué función de TensorFlow facilita la construcción de redes neuronales de manera más simple?
- A. TFX.
 - B. Keras.
 - C. PyTorch.
 - D. Matplotlib.

13. ¿Qué estructura de control en Python permite iterar sobre elementos de una lista?
- A. if-else .
 - B. for-loop .
 - C. while-loop .
 - D. switch-case .
14. ¿Qué paradigma de programación utiliza PyTorch para construir modelos de IA?
- A. Programación estructurada.
 - B. Programación orientada a objetos.
 - C. Programación procedural.
 - D. Programación funcional.
15. ¿Qué librería de Python es especialmente útil para cálculos numéricos?
- A. Pandas.
 - B. NumPy.
 - C. Seaborn.
 - D. Keras.
16. ¿Qué función tiene `optimizer.step()` en PyTorch?
- A. Define la arquitectura del modelo.
 - B. Calcula el gradiente.
 - C. Actualiza los parámetros del modelo.
 - D. Inicializa el modelo.

17. ¿Qué herramienta de TensorFlow es parte de su ecosistema completo para *machine learning*?
 - A. TorchScript.
 - B. Autograd.
 - C. TFX.
 - D. PyTorch.

18. ¿Qué hace el método ``sum`` en Python cuando se aplica a una lista?
 - A. Suma todos los elementos de la lista.
 - B. Concatena los elementos de la lista.
 - C. Ordena los elementos de la lista.
 - D. Elimina duplicados de la lista.

19. ¿Qué biblioteca de Python proporciona soporte para *arrays* multidimensionales?
 - A. Seaborn.
 - B. Pandas.
 - C. Matplotlib.
 - D. NumPy.

20. ¿Qué hace el método ``dropna()`` en Pandas?
 - A. Añade valores nulos a un *dataframe*.
 - B. Elimina filas con valores nulos en un *dataframe*.
 - C. Llena valores nulos en un *dataframe*.
 - D. Duplica un *dataframe*.

Fundamentos de Inteligencia Artificial para Ingenieros de
Software

Tema 6. Proyectos prácticos. Aplicación de IA en problemas de software

Índice

Esquema

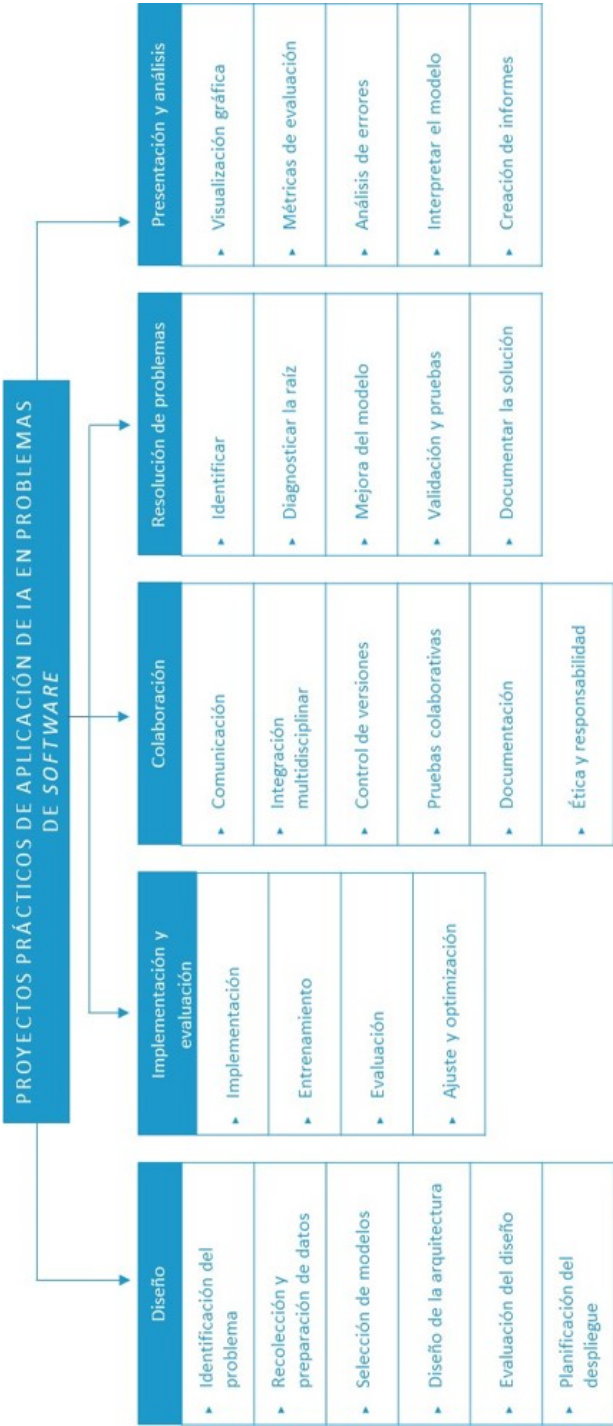
Ideas clave

- 6.1. Introducción y objetivos
- 6.2. Diseño de proyectos prácticos
- 6.3. Implementación y evaluación de modelos de IA
- 6.4. Colaboración en equipos de desarrollo de software
- 6.5. Resolución de problemas prácticos con IA
- 6.6. Presentación y análisis de resultados
- 6.7. Referencias bibliográficas

A fondo

- Despliegue de modelos de aprendizaje automático
- Como construir una infraestructura de aprendizaje automático en producción
- Gestión del ciclo de vida de modelos de IA en proyectos software

Test



6.1. Introducción y objetivos

La inteligencia artificial (IA) ha emergido como una disciplina clave en la resolución de problemas complejos dentro del ámbito del desarrollo de *software*. La integración de la IA en proyectos de *software* no solo mejora las capacidades técnicas de las aplicaciones, sino que también transforma la forma en que se abordan los problemas.

A medida que los modelos de aprendizaje automático y el procesamiento de grandes volúmenes de datos se vuelven más accesibles, los equipos de desarrollo pueden automatizar tareas complejas, optimizar procesos y predecir resultados con mayor exactitud. Esto es particularmente relevante en contextos donde la toma de decisiones rápida y basada en datos es crucial, como en la personalización de experiencias de usuario, la detección de fraudes o el mantenimiento predictivo.

Este tema se enfoca en la aplicación práctica de técnicas de IA en el desarrollo de *software* con un énfasis en cómo diseñar, implementar y evaluar proyectos de IA. Esto implica no solo aprender a utilizar las herramientas y técnicas más avanzadas, sino también entender cómo integrar estas soluciones en entornos de desarrollo existentes mientras se colabora eficazmente en equipos multidisciplinarios y se asegura que los resultados sean comprensibles y accionables.

Los objetivos de este tema son:

- ▶ Analizar las metodologías y herramientas más eficaces para la planificación y estructuración de proyectos de IA aplicados a problemas de *software*.
- ▶ Profundizar en los procesos de implementación y evaluación de modelos de IA
- ▶ Identificar las mejores prácticas para la colaboración en equipos multidisciplinarios.

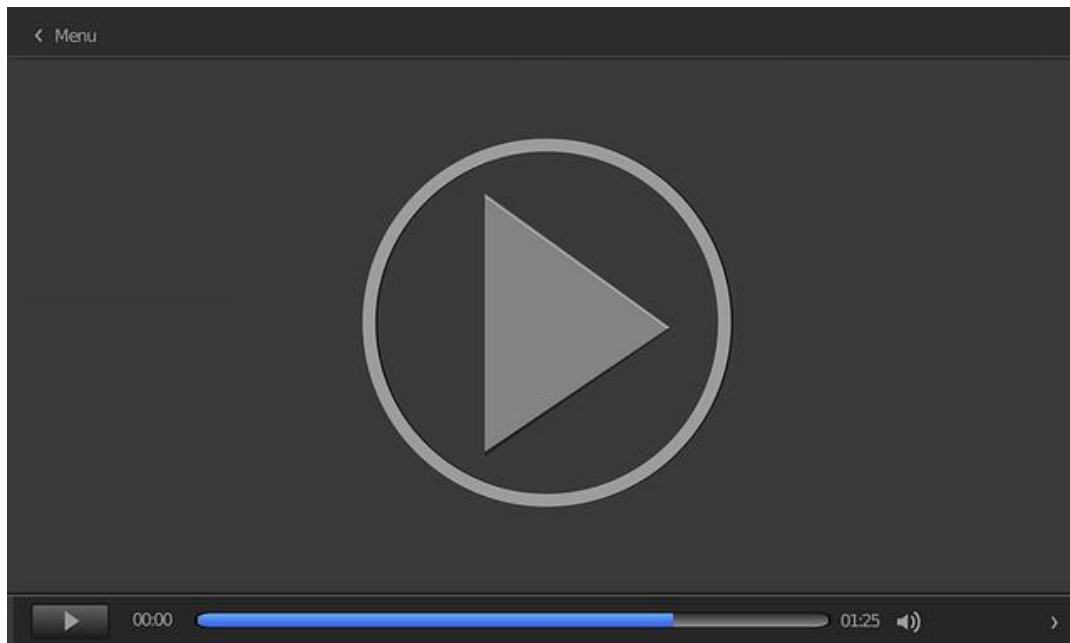
- ▶ Desarrollar habilidades para la aplicación de técnicas de IA en la resolución de problemas concretos en el desarrollo de *software*.
- ▶ Capacitar en la presentación clara y rigurosa de los resultados obtenidos.

6.2. Diseño de proyectos prácticos

El diseño de proyectos prácticos en inteligencia artificial (IA) implica una planificación cuidadosa que abarca desde la identificación del **problema** hasta la implementación de la **solución**. Este proceso requiere no solo de **conocimientos técnicos** en programación y uso de *frameworks*, como TensorFlow y PyTorch, sino también una comprensión clara del problema que se desea resolver y de los **objetivos** del proyecto.

El diseño de proyectos prácticos asegura que las soluciones de IA desarrolladas sean robustas, escalables y alineadas con los objetivos del negocio o del proyecto específico.

El vídeo *Clasificador de sentimientos con TensorFlow* nos muestra cómo implementar un modelo de clasificación de textos en positivo/negativo pasando por las fases de recolección y preprocesamiento de datos, diseño e implementación del modelo, entrenamiento y evaluación del modelo.



Clasificador de sentimientos con TensorFlow

Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=73f71c16-0746-4f63-8453-b21200a255d1>

A continuación, se exploran los pasos clave en el **diseño** de proyectos de IA aplicados a problemas de *software*:

- ▶ **Identificación del problema:** el primer paso en el diseño de un proyecto práctico de IA es identificar un problema específico que pueda ser resuelto mediante técnicas de IA. Este problema debe estar bien definido y ser relevante dentro del contexto de aplicación. Por ejemplo, un problema común en el desarrollo de *software* es la clasificación automática de errores en *logs* de sistemas, que puede ser abordado mediante modelos de aprendizaje supervisado. Por ejemplo, un pequeño comercio con tienda *online* podría registrar una alta actividad de los usuarios en el portal web, pero que no se materializa finalmente en compras. Esto podría deberse a que solo comparan precios, tienen problemas con la gestión del carro de la compra o problemas de conectividad desde sus dispositivos.
- **Definición del problema:** identificar y definir claramente el problema asegurando que sea relevante y abordable mediante técnicas de IA.
- **Recolección de requisitos:** colaborar con expertos en el dominio y partes interesadas para comprender los requisitos funcionales y no funcionales del proyecto.
- **Objetivos y métricas de éxito:** establecer objetivos claros y definibles, así como las métricas que se utilizarán para medir el éxito del proyecto.
- ▶ **Recolección y preparación de datos:** una vez identificado el problema, el siguiente paso es la recolección y preparación de los datos necesarios para entrenar y evaluar el modelo de IA. Los datos deben ser de alta calidad y representativos del problema. En el caso de la clasificación de errores en *logs*, esto podría implicar la recopilación de *logs* de sistemas de *software* en operación seguido de un preprocesamiento para limpiar y estructurar la información. En esta etapa se incluyen:
- ▶ **Recolección de datos:** obtener datos relevantes de diversas fuentes, como bases de datos internas, API o *datasets* públicos.

- ▶ **Limpieza de datos:** identificar y corregir errores en los datos, eliminar duplicados y manejar valores faltantes para asegurar que los datos sean consistentes y utilizables.
- ▶ **Transformación de datos:** normalizar, escalar y transformar los datos según sea necesario para que sean adecuados para el modelo. Esto puede incluir la codificación de variables categóricas o la creación de nuevas características a partir de los datos existentes.
- ▶ **Segmentación de datos:** dividir los datos en conjuntos de entrenamiento, validación y prueba para asegurar una evaluación justa del modelo.

Ejemplo en Python usando Pandas para la preparación de datos:

```
import pandas as pd

# Cargar datos de logs

logs = pd.read_csv('logs_sistema.csv')

# Preprocesamiento: Limpieza de datos nulos

logs = logs.dropna()

# Preprocesamiento: Tokenización y eliminación de caracteres especiales

logs['log_message'] = logs['log_message'].str.replace('[^a-zA-Z]', ' ')
logs['log_message'] = logs['log_message'].str.lower()

# Visualización de los primeros registros

print(logs.head())
```

- ▶ **Selección de modelos y algoritmos:** el siguiente paso es seleccionar los modelos y algoritmos más adecuados para resolver el problema. La selección del modelo depende de varios factores, incluido el tipo de problema (clasificación, regresión, *clustering*, etc.), la naturaleza de los datos y los recursos disponibles. En el contexto

de problemas de *software*, los modelos de clasificación y detección de anomalías son comunes. Por ejemplo, para la clasificación de errores en *logs* se podría utilizar una red neuronal simple implementada en TensorFlow o PyTorch. En un proyecto de clasificación de imágenes se podría evaluar el rendimiento de diferentes arquitecturas de redes neuronales convolucionales (CNN) y seleccionar la que ofrezca el mejor rendimiento en un conjunto de validación. En esta etapa se debe abordar:

- **Exploración de algoritmos:** evaluar diferentes algoritmos que podrían ser adecuados para el problema, como redes neuronales, árboles de decisión, SVM, entre otros.
- **Prototipado de modelos:** crear prototipos rápidos de diferentes modelos para evaluar su rendimiento inicial en los datos.
- **Selección del modelo:** elegir el modelo que ofrezca el mejor equilibrio entre precisión, interpretabilidad y eficiencia.
- ▶ **Diseño de la arquitectura del sistema:** el diseño de la arquitectura del sistema es una etapa crítica que implica planificar cómo se integrará el modelo de IA en el sistema de *software* más amplio. Esto incluye considerar la escalabilidad, la seguridad y la interoperabilidad del sistema. En esta etapa se debe considerar:
 - **Definición de la arquitectura:** diseñar la arquitectura del sistema especificando cómo interactuarán los diferentes componentes (modelos de IA, bases de datos, interfaces de usuario, etc.).
 - **Integración de IA:** planificar cómo el modelo de IA se integrará con otros sistemas existentes, como aplicaciones web, sistemas de bases de datos o API.
 - **Escalabilidad:** considerar cómo se escalará el sistema para manejar un mayor volumen de datos o usuarios utilizando soluciones como la computación en la nube o microservicios.

En un proyecto de **recomendación de productos** en un *e-commerce* se diseñaría una arquitectura que integre el modelo de IA con el sistema de gestión de la tienda en línea que asegure que las recomendaciones se generen en **tiempo real** y se presenten de manera **efectiva** al usuario final.

- ▶ **Validación y evaluación del diseño:** antes de pasar a la implementación completa, es crucial validar el diseño propuesto mediante revisiones y pruebas iniciales. Esto ayuda a identificar y corregir posibles problemas antes de que se conviertan en costosos errores en etapas posteriores. Para llevar a cabo esta tarea se deben realizar los siguientes pasos:
 - **Revisión del diseño:** involucrar a expertos en diferentes áreas (como ingenieros de *software*, científicos de datos y expertos en el dominio) para revisar y criticar el diseño.
 - **Pruebas piloto:** implementar un piloto o una versión mínima del sistema para evaluar su funcionamiento en condiciones reales.
 - **Evaluación de riesgos:** identificar posibles riesgos técnicos y de negocio y planificar cómo mitigarlos.

En un sistema de detección de fraudes, se podría desplegar un piloto que monitoree un pequeño *subset* de transacciones para evaluar la efectividad del modelo en la identificación de transacciones fraudulentas.

- ▶ **Planificación del despliegue y monitorización:** finalmente, es importante planificar cómo se desplegará el sistema en producción y cómo se monitorizará su desempeño a lo largo del tiempo. Esto incluye considerar aspectos, como la actualización del modelo, la respuesta ante fallos y la recopilación continua de datos para mejorar el sistema. Esta etapa incluye:
 - **Plan de despliegue:** definir cómo y cuándo se desplegará el sistema en el entorno de producción, incluyendo pruebas finales y validación en producción.

- **Monitorización continua:** implementar herramientas para monitorizar el desempeño del sistema y del modelo de IA en tiempo real y que permita la detección de problemas y la realización de ajustes.
- **Mantenimiento y mejora continua:** planificar cómo se mantendrá y actualizará el sistema a lo largo del tiempo, incluyendo el reentrenamiento de modelos con nuevos datos y la adaptación a cambios en el entorno.

En un sistema de análisis de sentimientos para redes sociales se podría planificar el despliegue en una infraestructura en la nube que permita la **monitorización** en tiempo real de las **predicciones** del modelo y la **actualización** regular de los datos de entrenamiento.

6.3. Implementación y evaluación de modelos de IA

La implementación y evaluación de modelos de inteligencia artificial (IA) son etapas críticas en el desarrollo de cualquier proyecto de IA. Después de diseñar y seleccionar los modelos adecuados, es esencial **implementar** estos modelos utilizando herramientas, como TensorFlow o PyTorch, y luego **evaluarlos** para asegurarse de que cumplen con los requisitos establecidos.

Un enfoque meticuloso en estas etapas asegura que los modelos desarrollados no solo sean **precisos**, sino también **escalables** y **eficientes**, lo cual es crucial para su éxito al aplicarlo al mundo real.

A continuación, se detallan los pasos clave en este proceso:

- **Implementación del modelo:** la implementación del modelo implica traducir el diseño conceptual del modelo a un código utilizando un *framework* de IA. TensorFlow y PyTorch son dos de las bibliotecas más utilizadas para este propósito, ya que permiten la construcción de redes neuronales desde simples perceptrones hasta modelos complejos de *deep learning*.

A continuación, se muestra un ejemplo de implementación en TensorFlow:

```
import tensorflow as tf

from tensorflow.keras import layers

# Definición del modelo

model = tf.keras.Sequential([

    layers.Dense(128, activation='relu', input_shape=(input_shape,)),

    layers.Dropout(0.2),
```

```
layers.Dense(64, activation='relu'),

layers.Dense(num_classes, activation='softmax')

])

# Compilación del modelo

model.compile(optimizer='adam', loss='categorical_crossentropy',

              metrics=['accuracy'])

# Resumen del modelo

model.summary()
```

En este ejemplo se implementa una **red neuronal multicapa** con dos capas densas y una capa de salida con activación `softmax` comúnmente utilizada para tareas de **clasificación** multiclase. La capa `Dropout` se usa para reducir el **sobreajuste**, un problema común en el entrenamiento de modelos de *deep learning*.

La capa de *dropout* asigna aleatoriamente el valor 0 a algunas entradas de la red. Esto ayuda a prevenir el sobreajuste o *overfitting*, que son problemas que pueden derivar en una **memorización de datos** de entrenamiento e impiden la generalización del modelo a **nuevos datos**.

A continuación, se muestra un ejemplo de implementación en PyTorch:

```
import torch

import torch.nn as nn

import torch.optim as optim
```

```
# Definición del modelo en PyTorch

class NeuralNet(nn.Module):

    def __init__(self, input_size, num_classes):

        super(NeuralNet, self).__init__()

        self.fc1 = nn.Linear(input_size, 128)

        self.relu = nn.ReLU()

        self.fc2 = nn.Linear(128, 64)

        self.fc3 = nn.Linear(64, num_classes)

    def forward(self, x):

        out = self.fc1(x)

        out = self.relu(out)

        out = self.fc2(out)

        out = self.relu(out)

        out = self.fc3(out)

        return out

# Instanciación del modelo y definición del optimizador

model = NeuralNet(input_size=input_shape, num_classes=num_classes)
```

```
criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(model.parameters(), lr=0.001)

# Resumen del modelo

print(model)
```

Aquí, se implementa una red neuronal similar en PyTorch. La clase `NeuralNet` define la arquitectura del modelo y el optimizador Adam se utiliza para entrenar el modelo. PyTorch ofrece **flexibilidad** al permitir un control más **detallado** sobre el proceso de entrenamiento, lo que supone una ventaja para desarrolladores que requieren personalización avanzada.

- **Entrenamiento del modelo:** una vez que el modelo ha sido implementado, el siguiente paso es entrenarlo utilizando los datos de entrenamiento. Este proceso se realiza automáticamente mediante los *frameworks* de IA.

Entrenamiento en TensorFlow:

```
# Entrenamiento del modelo

history = model.fit(train_data, train_labels, epochs=10, validation_data=
(val_data, val_labels))

# Gráfico de la precisión durante el entrenamiento

import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'], label='Precisión de entrenamiento')

plt.plot(history.history['val_accuracy'], label='Precisión de validación')

plt.legend()
```



```
plt.show()
```

Entrenamiento en PyTorch:

```
# Entrenamiento del modelo en PyTorch
```

```
num_epochs = 10
```

```
for epoch in range(num_epochs):
```

```
    for i, (inputs, labels) in enumerate(train_loader):
```

```
        # Forward pass
```

```
        outputs = model(inputs)
```

```
        loss = criterion(outputs, labels)
```

```
        # Backward pass y optimización
```

```
        optimizer.zero_grad()
```

```
        loss.backward()
```

```
        optimizer.step()
```

```
    print(f'Epoch [{epoch+1}/{num_epochs}], Pérdida: {loss.item():.4f}')
```

En estos ejemplos se muestra cómo entrenar un modelo en TensorFlow y PyTorch. En TensorFlow, la función `fit` simplifica el proceso, mientras que en PyTorch el proceso es más explícito al ofrecer un control más granular.

Además de realizar el entrenamiento, existen herramientas, como Tensorboard, que permiten la **visualización gráfica** de este proceso. De esta forma, se puede ver la **evolución** del modelo y detectar **puntos de mejora**, anomalías, *overfitting*, etc.

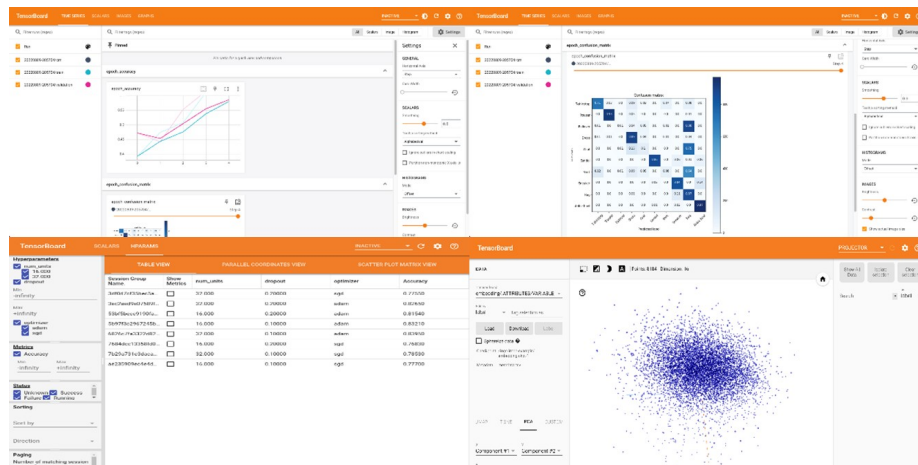


Figura 1. Algunos ejemplos de gráficas y tablas mostradas por Tensorboard: Fuente: Empiece a utilizar TensorBoard, s. f.

- **Evaluación del modelo:** después de entrenar el modelo es fundamental evaluarlo para asegurarse de que funciona correctamente con datos no vistos. Esto se hace, generalmente, usando un conjunto de datos de prueba o de validación y calculando métricas de rendimiento, como la precisión, la pérdida, la *F1-score*, entre otras.

Evaluación en TensorFlow:

```
# Evaluación del modelo en los datos de prueba
```

```
test_loss, test_acc = model.evaluate(test_data, test_labels)
```

```
print(f'Precisión en los datos de prueba: {test_acc:.4f}')
```

Evaluación en PyTorch:

```
# Evaluación del modelo en PyTorch
```

```
model.eval() # Modo de evaluación
```

```
with torch.no_grad():
```

```
    correct = 0
```

```
total = 0

for inputs, labels in test_loader:

    outputs = model(inputs)

    _, predicted = torch.max(outputs.data, 1)

    total += labels.size(0)

    correct += (predicted == labels).sum().item()

print(f'Precisión en los datos de prueba: {100 * correct /
total:.4f}%')
```

En la evaluación se compara el **rendimiento del modelo** en los datos de prueba con las **métricas esperadas**. Este paso es crucial para verificar si el modelo generaliza bien y si es adecuado para su implementación en un entorno de producción.

- **Ajuste y optimización:** después de la evaluación inicial es posible que el modelo requiera ajustes adicionales para mejorar su rendimiento. Esto puede implicar la modificación de hiperparámetros, el uso de técnicas de regularización o la recolección de más datos para mejorar la robustez del modelo.

Optimización en TensorFlow:

```
from tensorflow.keras.optimizers import SGD

# Reentrenar con un optimizador diferente

model.compile(optimizer=SGD(learning_rate=0.01, momentum=0.9),
loss='categorical_crossentropy',

metrics=['accuracy'])

model.fit(train_data, train_labels, epochs=10, validation_data=(val_data,
val_labels))
```

Optimización en PyTorch:

```
# Cambiar el optimizador para realizar ajustes adicionales
```

```
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

```
# Reentrenar el modelo con nuevos parámetros
```

```
for epoch in range(num_epochs):
```

```
    # (código de entrenamiento)
```

6.4. Colaboración en equipos de desarrollo de software

La colaboración efectiva en equipos de desarrollo de *software* es crucial para el éxito de cualquier proyecto de inteligencia artificial (IA). Los proyectos de IA suelen ser **complejos y multidisciplinarios**, lo que requiere una cooperación estrecha entre desarrolladores, científicos de datos, ingenieros de *software*, expertos en dominios específicos y, en algunos casos, especialistas en ética y legislación.

La colaboración requiere una combinación de comunicación efectiva, integración de conocimientos multidisciplinarios, uso de herramientas de control de versiones, desarrollo y pruebas colaborativas, documentación rigurosa y gestión ética.

A continuación, se analizan las mejores prácticas para fomentar una **colaboración efectiva** en equipos de desarrollo de *software* enfocados en proyectos de IA:

- **Comunicación clara y continua:** la comunicación efectiva es la base de la colaboración satisfactoria en cualquier equipo. En proyectos de IA esto implica asegurarse de que todos los miembros del equipo comprendan claramente los objetivos del proyecto, los requisitos, las expectativas y los plazos. Es recomendable utilizar herramientas de comunicación y gestión de proyectos, como Slack, Jira o Asana, para facilitar el seguimiento del progreso y la resolución de problemas en tiempo real.

Un equipo de desarrollo que trabaja en la implementación de un sistema de recomendación basado en IA podría utilizar reuniones diarias rápidas (*stand-ups*) para discutir el **progreso**, identificar **obstáculos** y coordinar **tareas**. Además, pueden utilizar Jira para gestionar tareas y asegurarse de que todos los miembros del equipo estén al tanto de sus responsabilidades.

- **Integración de conocimientos multidisciplinarios:** los proyectos de IA requieren la integración de conocimientos de diferentes disciplinas. Por ejemplo, los científicos de datos aportan conocimientos en modelos de aprendizaje automático, mientras que los ingenieros de *software* se centran en la arquitectura del sistema y el desarrollo de código escalable. Es fundamental fomentar un entorno en el que se valoren y aprovechen las habilidades de cada miembro del equipo.

Durante el desarrollo de un sistema de detección de fraudes, un experto en finanzas podría trabajar estrechamente con científicos de datos para definir **características relevantes** (*features*) basadas en el **comportamiento** financiero, mientras que los desarrolladores implementan estas características en un modelo de IA.

- **Uso de herramientas de control de versiones:** el uso de herramientas de control de versiones, como Git, es esencial en proyectos colaborativos, ya que permite a los miembros del equipo trabajar en paralelo sin interferir en el trabajo de los demás. Además, estas herramientas facilitan la revisión del código, el seguimiento de cambios y la integración continua.

Ejemplo en Git:

```
# Clonar el repositorio del proyecto
```

```
git clone https://github.com/usuario/proyecto-ia.git
```

```
# Crear una nueva rama para trabajar en una función específica
```

```
git checkout -b nueva-funcion
```

```
# Realizar cambios y commit

git add .

git commit -m "Implementación de nueva función de preprocesamiento"

# Subir los cambios al repositorio remoto

git push origin nueva-funcion
```

En este ejemplo un miembro del equipo podría estar trabajando en una nueva función de preprocesamiento de datos sin afectar el **código principal** del proyecto. Una vez completada, la función se puede revisar e integrar en la **rama principal**.

- **Desarrollo y pruebas colaborativas:** el desarrollo de *software* en proyectos de IA debe incluir la implementación de pruebas continuas para asegurar la calidad del código y la precisión del modelo. Las pruebas unitarias, la validación cruzada y las pruebas A/B son prácticas comunes que deben ser realizadas y revisadas por todo el equipo.

Ejemplo en TensorFlow para pruebas A/B:

```
# Supongamos que tenemos dos versiones del modelo para pruebas A/B

model_a = crear_modelo(version='a')

model_b = crear_modelo(version='b')

# Entrenar ambos modelos

model_a.fit(train_data, train_labels, epochs=10, validation_data=(val_data,
val_labels))

model_b.fit(train_data, train_labels, epochs=10, validation_data=(val_data,
val_labels))

# Comparar el rendimiento de ambos modelos
```

```
loss_a, acc_a = model_a.evaluate(test_data, test_labels)

loss_b, acc_b = model_b.evaluate(test_data, test_labels)

print(f'Rendimiento del modelo A: {acc_a:.4f}')

print(f'Rendimiento del modelo B: {acc_b:.4f}')
```

Este tipo de pruebas permite al equipo evaluar qué versión del modelo de IA tiene un mejor rendimiento, lo que facilita la toma de decisiones informadas.

- **Documentación y transferencia de conocimiento:** la documentación es vital para la colaboración efectiva, especialmente en proyectos de IA donde el código, los datos y los modelos deben ser comprensibles para todos los miembros del equipo y para futuros desarrolladores. La documentación debe incluir la descripción del modelo, las decisiones de diseño, las pruebas realizadas y los resultados obtenidos.

Un equipo de desarrollo podría utilizar herramientas, como Sphinx o Jupyter Notebooks, para **documentar** el proceso de desarrollo de un modelo de IA que incluya ejemplos de código, resultados de pruebas y gráficos que muestren el rendimiento del modelo.

- **Gestión de la ética y la responsabilidad:** es crucial que los equipos de desarrollo de IA consideren los aspectos éticos y la responsabilidad social del uso de IA. Esto incluye la transparencia en el proceso de toma de decisiones del modelo, la equidad en los resultados y la protección de los datos de los usuarios. Los equipos deben incluir especialistas en ética o consultores legales cuando sea necesario para asegurar que el proyecto cumple con los estándares legales y éticos.

6.5. Resolución de problemas prácticos con IA

En un proyecto de *software* que incluye modelos de inteligencia artificial (IA), el análisis y resolución de problemas es un proceso **continuo** y **crítico** para asegurar el éxito del proyecto. Los problemas que afectan a los modelos de IA pueden surgir en diferentes etapas del ciclo de vida del desarrollo, desde la fase de datos y entrenamiento hasta la implementación y monitorización en producción. A continuación, se describen las **estrategias** y enfoques para abordar y solucionar estos problemas, centrándose en cómo impactan específicamente a los modelos de IA.

La documentación y comunicación efectiva durante todo el proceso de desarrollo garantiza que el equipo pueda aprender de los **problemas resueltos** y mejorar continuamente sus prácticas de desarrollo.

A continuación, se describen los pasos clave para resolver **problemas prácticos** con IA:

- ▶ **Identificación de problemas en el modelo de IA:** la primera etapa en la resolución de problemas es la identificación precisa de los problemas que afectan al modelo de IA. Estos problemas pueden manifestarse de diversas maneras, como un rendimiento bajo, problemas de sobreajuste (*overfitting*), falta de generalización o errores en la predicción:
- **Análisis de desempeño:** utilizar métricas clave como la precisión, *recall*, *F1-score*, error cuadrático medio (MSE), entre otras, para evaluar el rendimiento del modelo y detectar desviaciones respecto a las expectativas.
- **Detección de sobreajuste:** observar la discrepancia entre el rendimiento en los datos de entrenamiento y los datos de validación para identificar si el modelo está sobreajustado a los datos de entrenamiento.

- **Modelos de baja vs alta escala (*low-scale vs. large-scale modelos*):** analizar las diferencias entre los conjuntos de datos de entrenamiento/validación/test y los utilizados como entrada en el modelo desplegado.
- **Errores en predicciones:** revisar ejemplos específicos donde el modelo comete errores significativos para identificar patrones comunes o características que puedan estar causando problemas.

Un **bajo rendimiento** puede ser un indicio de datos insuficientes, mala elección del modelo o problemas en el preprocesamiento de datos.

- ▶ **Diagnóstico y análisis de causas raíz:** una vez identificado un problema, el siguiente paso es diagnosticar la causa raíz. Este proceso puede involucrar la revisión del *pipeline* completo de desarrollo del modelo, desde la recolección de datos hasta la implementación del modelo.
- **Revisión de los datos:** verificar si los datos utilizados para entrenar el modelo son representativos, equilibrados y libres de sesgos. Los problemas de calidad de datos, como datos ruidosos, valores faltantes o sesgados, son una causa común de bajo rendimiento en los modelos de IA.
- **Evaluación del preprocesamiento:** asegurarse de que las etapas de preprocesamiento, como la normalización, escalado y selección de características, se hayan realizado correctamente. Los problemas en esta fase pueden llevar a que el modelo no aprenda adecuadamente. Si un modelo de clasificación de imágenes muestra un **rendimiento deficiente**, un análisis de las imágenes mal clasificadas podría revelar que el modelo tiene dificultades con imágenes de baja calidad o con ruido. En este caso, la causa raíz podría ser la falta de **preprocesamiento** adecuado para manejar estas variaciones en las imágenes.
- **Análisis de la complejidad del modelo:** evaluar si el modelo es demasiado simple (subajuste) o demasiado complejo (sobreajuste) para los datos disponibles. Esto puede incluir la revisión de la arquitectura de la red neuronal, la cantidad de capas,

el número de neuronas, etc.

Una red neuronal aprende el **modelo matemático** subyacente que representa un **conjunto de datos**. Un modelo muy simple podría no ser capaz de captar toda esa complejidad, mientras que un modelo demasiado complejo podría aprender o memorizar esos datos y no funcionar correctamente con nueva información

- ▶ **Ajuste y mejora del modelo:** después de diagnosticar el problema, se deben realizar ajustes y mejoras al modelo para resolver los problemas identificados. Este proceso puede involucrar modificaciones en los datos, el modelo o las técnicas de entrenamiento.
- **Recolección y ampliación de datos:** si se identifican problemas relacionados con la falta de datos representativos o sesgos en los datos, se puede considerar la recolección de más datos, la generación de datos sintéticos o la aplicación de técnicas de aumento de datos (*data augmentation*). Las técnicas de *data augmentation* permiten el incremento artificial del conjunto de datos mediante la manipulación. Por ejemplo, en los modelos que analizan imágenes, estas se pueden rotar, voltear o aplicar ejes de simetría a las imágenes para incrementar el conjunto de datos.
- **Ajuste de hiperparámetros:** modificar hiperparámetros, como la tasa de aprendizaje, el tamaño de los bloques de entrenamiento (*batch size*) o el número de etapas (*epochs*), para mejorar el proceso de entrenamiento y la capacidad del modelo para generalizar mejor.
- **Regularización:** aplicar técnicas de regularización, como *L2 regularization* o *dropout*, para reducir el sobreajuste del modelo.
- **Cambios en la arquitectura del modelo:** si el modelo es demasiado complejo o no lo suficientemente complejo, se pueden realizar cambios en la arquitectura del modelo. Esto puede incluir la adición o eliminación de capas, el cambio de funciones de activación o la introducción de nuevas capas, como capas convolucionales en

redes neuronales.

En este ejemplo se ajusta un modelo en TensorFlow utilizando una regularización L2 y una capa de *dropout* para prevenir el sobreajuste. Se ajusta la **tasa de aprendizaje** para optimizar el entrenamiento.

```
from tensorflow.keras import layers, regularizers

# Modificación de la arquitectura del modelo con regularización L2

model = tf.keras.Sequential([

    layers.Dense(128, activation='relu',

                 kernel_regularizer=regularizers.l2(0.001)),

    layers.Dropout(0.5),

    layers.Dense(64, activation='relu',

                 kernel_regularizer=regularizers.l2(0.001)),

    layers.Dense(10, activation='softmax')

])

# Ajuste del modelo con una tasa de aprendizaje diferente

model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),

              loss='sparse_categorical_crossentropy',

              metrics=['accuracy'])
```

- ▶ **Validación y pruebas adicionales:** después de realizar ajustes en el modelo, es fundamental validar nuevamente el modelo para asegurarse de que los problemas hayan sido resueltos y que el rendimiento del modelo haya mejorado. Esto puede incluir la repetición de pruebas con conjuntos de datos de validación y prueba, así como la implementación de pruebas adicionales, como la validación cruzada (Hastie, Tibshirani y Friedman, 2017).
- **Validación cruzada:** utilizar técnicas de validación cruzada para evaluar la robustez del modelo en diferentes particiones del conjunto de datos.
- **Pruebas A/B:** implementar pruebas A/B si el modelo ya está en producción para comparar el rendimiento del modelo ajustado con el modelo anterior en condiciones reales.
- **Monitoreo de métricas de rendimiento:** establecer un sistema de monitoreo continuo para seguir el rendimiento del modelo en producción y detectar cualquier degradación en tiempo real.

La validación cruzada y las pruebas adicionales son esenciales para asegurar la **fiabilidad** y la **robustez** del modelo.

- ▶ **Documentación y comunicación de resultados:** finalmente, es crucial documentar todo el proceso de resolución de problemas y comunicar los resultados a las partes interesadas. La documentación debe incluir los problemas identificados, las causas raíz diagnosticadas, las soluciones implementadas y los resultados obtenidos. Esto no solo facilita la transparencia, sino que también proporciona una base para futuras mejoras y la resolución de problemas similares.
- **Documentación:** crear un informe detallado que describa los problemas encontrados, las decisiones tomadas y los resultados alcanzados.
- **Comunicación con el equipo:** compartir los hallazgos y las mejoras con todo el equipo, incluyendo científicos de datos, desarrolladores y gestores, para asegurar

una comprensión común y para planificar los próximos pasos.

- **Lecciones aprendidas:** reflexionar sobre el proceso y documentar las lecciones aprendidas para mejorar los procesos futuros y evitar problemas similares.

Una buena documentación ayuda a evitar la **repetición de errores** y facilita la **colaboración en equipo** (Sommerville, 2015).

6.6. Presentación y análisis de resultados

La presentación y el análisis de los resultados son etapas críticas en cualquier proyecto de inteligencia artificial (IA). Estas etapas permiten no solo evaluar el **rendimiento** del modelo, sino también comunicar los **hallazgos** de manera efectiva a las partes interesadas. Un análisis riguroso y una presentación clara ayudan a validar la utilidad del modelo, identificar posibles mejoras y tomar decisiones informadas sobre la implementación del sistema de IA.

Este proceso incluye la visualización de **resultados**, el uso de **métricas** adecuadas, el análisis de **errores**, la **interpretabilidad** del modelo y la **validación** de su desempeño.

A continuación, se describen las mejores prácticas para la **presentación** y **análisis** de resultados en proyectos de IA:

- ▶ **Visualización de resultados:** una de las formas más efectivas de presentar los resultados de un modelo de IA es mediante las visualizaciones gráficas. Las visualizaciones permiten a las partes interesadas comprender rápidamente el rendimiento del modelo y las relaciones clave en los datos. Algunos gráficos, como las curvas ROC, las matrices de confusión y las gráficas de dispersión, son particularmente útiles.

Ejemplo en Python utilizando Matplotlib:

```
import matplotlib.pyplot as plt

from sklearn.metrics import roc_curve, confusion_matrix,
ConfusionMatrixDisplay

# Supongamos que tenemos las etiquetas verdaderas y las predicciones del
modelo
```

```

y_true = [0, 1, 0, 1, 0, 1, 0, 1]

y_pred = [0, 1, 0, 1, 0, 0, 1, 1]

# Matriz de confusión

cm = confusion_matrix(y_true, y_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)

disp.plot()

plt.show()

# Curva ROC

fpr, tpr, _ = roc_curve(y_true, y_pred)

plt.plot(fpr, tpr, marker='.')

plt.xlabel('Tasa de falsos positivos')

plt.ylabel('Tasa de verdaderos positivos')

plt.show()

```

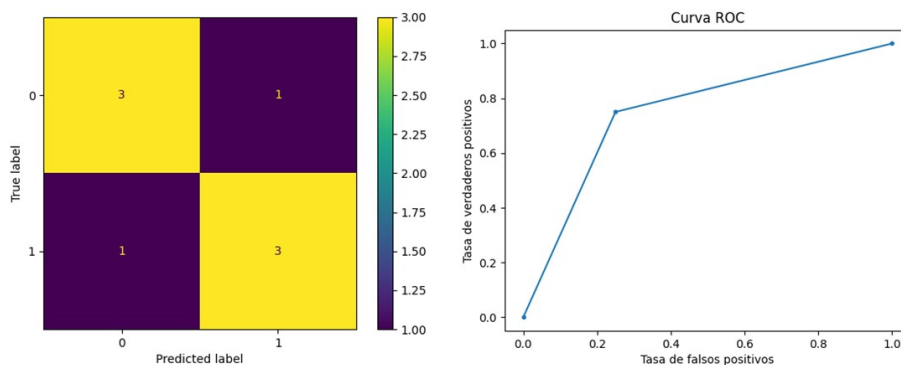


Figura 1. Matriz de confusión y curva ROC mediante Matplotlib. Fuente: elaboración propia.

En este ejemplo, la matriz de confusión y la curva ROC se utilizan para **evaluar** y **visualizar** el **rendimiento** de un modelo de clasificación. Estas herramientas gráficas son fundamentales para comprender cómo el modelo maneja las **clases positivas** y **negativas**.

Las gráficas de entrenamiento generadas por Tensorboard nos muestran la **evolución** y **progreso** del modelo entrenado.

- **Métricas de evaluación:** las métricas son esenciales para cuantificar el rendimiento de un modelo. Dependiendo del tipo de problema (clasificación, regresión, etc.) se seleccionan diferentes métricas.

Ejemplo en Python para calcular métricas de evaluación:

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score

# Calcular métricas

accuracy = accuracy_score(y_true, y_pred)

precision = precision_score(y_true, y_pred)

recall = recall_score(y_true, y_pred)

f1 = f1_score(y_true, y_pred)

print(f'Precisión: {accuracy:.4f}')

print(f'Precisión positiva (Precision): {precision:.4f}')

print(f'Exhaustividad (Recall): {recall:.4f}')
```

```
print(f'Puntuación F1: {f1:.4f}')
```

Este ejemplo muestra cómo calcular y presentar métricas clave que permiten evaluar la calidad de un modelo de clasificación. Estas métricas son cruciales para comunicar la **eficacia** del modelo a las partes interesadas.

- **Análisis de errores:** el análisis de errores es una parte fundamental del análisis de resultados. Consiste en identificar los casos en los que el modelo falla y entender por qué se producen estos errores. Este análisis puede revelar patrones importantes en los datos que no fueron capturados adecuadamente por el modelo y sugerir mejoras potenciales.

En un modelo de reconocimiento de imágenes, el análisis de los errores podría revelar que el modelo tiene dificultades para **distinguir** entre clases que son **visualmente similares** (por ejemplo, perros y lobos). Este descubrimiento podría llevar a ajustes en la **arquitectura** del modelo o a la incorporación de **más datos** de entrenamiento para estas clases específicas.

- **Interpretabilidad del modelo:** en muchos casos es esencial que los modelos de IA sean interpretables, especialmente cuando se utilizan en dominios críticos, como la medicina o las finanzas. La interpretabilidad permite a los usuarios entender cómo el modelo toma decisiones, lo que es crucial para generar confianza y asegurar el cumplimiento de regulaciones.

Ejemplo de interpretabilidad con LIME en Python:

```
import lime

import lime.lime_tabular

# Crear un explicador LIME para un modelo de clasificación

explainer = lime.lime_tabular.LimeTabularExplainer(X_train.values,
feature_names=X_train.columns, class_names=['No', 'Sí'],
mode='classification')
```

```
# Explicar una predicción específica

exp = explainer.explain_instance(X_test.iloc[0], model.predict_proba)

exp.show_in_notebook(show_table=True)
```

LIME (Local Interpretable Model-agnostic Explanations) es una herramienta que ayuda a **interpretar** las **predicciones** de los modelos de IA al mostrar qué características contribuyen más a una predicción específica. Esto es especialmente útil en modelos complejos, como las redes neuronales.

- **Comunicación de resultados a partes interesadas:** la comunicación efectiva de los resultados es crucial para asegurar que todos los miembros del equipo y las partes interesadas comprendan los hallazgos y las implicaciones del modelo. Esto puede implicar la creación de informes detallados, presentaciones o *dashboards* interactivos que resuman los resultados de manera accesible. Un equipo de desarrollo podría crear un *dashboard* utilizando herramientas, como Tableau o Dash de Plotly, que permitan a las partes interesadas interactuar con los resultados del modelo y explorar diferentes escenarios. Esto facilita la toma de decisiones basada en datos y asegura que los resultados del modelo se utilicen de manera efectiva en la organización.

6.7. Referencias bibliográficas

Empiece a utilizar TensorBoard. (S. f.). https://www.tensorflow.org/tensorboard/get_started?hl=es-419

Hastie, T., Tibshirani, R., Friedman, J. H. y Friedman, J. H. (2017). *The elements of statistical learning: data mining, inference, and prediction* (2ª ed., pp. 1-758). Springer.

Sommerville, I. (2011). *Software engineering*. Pearson Education Inc.

Despliegue de modelos de aprendizaje automático

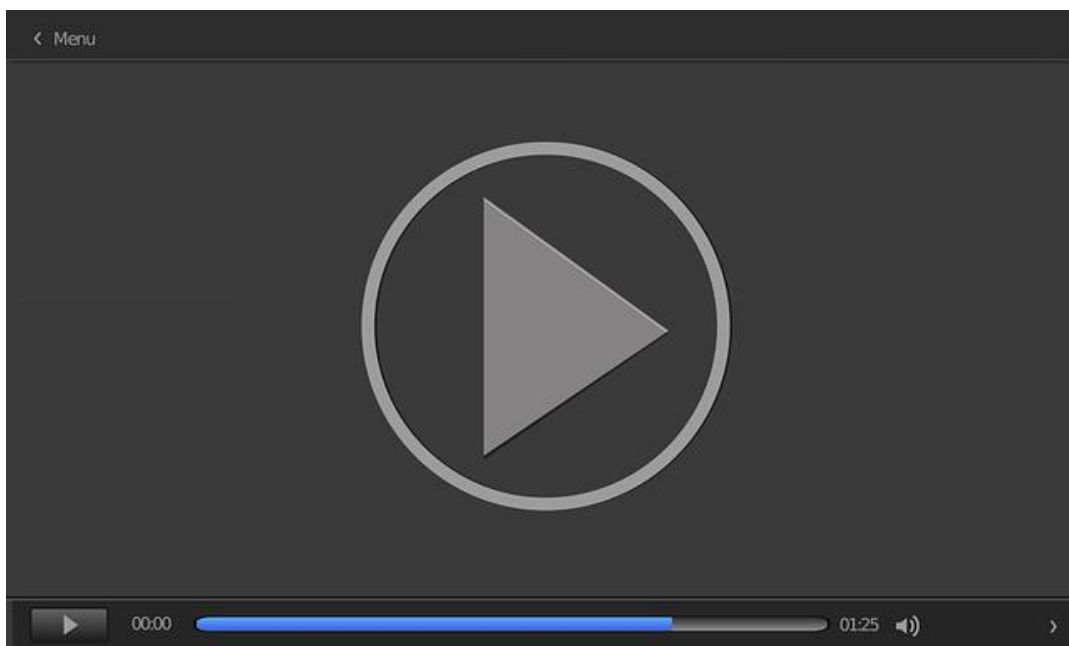
Brownlee, J. (2021). *Deploy your predictive model to production*. Machine Learning Mastery. <https://machinelearningmastery.com/deploy-machine-learning-model-to-production/>

En este artículo se dan una serie de consejos y pasos para incorporar un modelo predictivo de aprendizaje automático en un entorno de producción: análisis de recursos necesarios para desplegar el modelo y puntos por tener en cuenta para el mantenimiento y mejora.

Como construir una infraestructura de aprendizaje automático en producción

Wills, J. (2014, agosto 1). *Midwest.io 2014 - Building a Production Machine Learning Infrastructure* [vídeo]. YouTube. https://youtu.be/lgfRdDjLxe0?si=CGL_IMmJ6P9RR2Ji

En esta ponencia se explica cómo desplegar una infraestructura de aprendizaje automático para la producción.



Accede al vídeo:

<https://www.youtube.com/embed/lgfRdDjLxe0>

Gestión del ciclo de vida de modelos de IA en proyectos software

Shafiq, S., Mashkoor, A., Mayr-Dorn, C., & Egyed, A. (2021). A literature review of using machine learning in software development life cycle stages. *IEEE Access*, 9, 140896- 140920. https://www.researchgate.net/publication/355227662_A_Literature_Review_of_Machine_Learning_and_Software_Development_Life_cycle_Stages

En este artículo se describe cómo se está integrando y aplicando el aprendizaje automático en el ciclo de vida de los proyectos de *software*. Se realiza una revisión de las herramientas y modelos aplicados en diversas etapas del ciclo de vida del desarrollo *software* y cuáles pueden ser más útiles en cada fase.

1. ¿Cuál es el primer paso en el diseño de un proyecto práctico de IA?
 - A. Recolección de datos.
 - B. Identificación del problema.
 - C. Selección de algoritmos.
 - D. Implementación del modelo.

2. ¿Qué herramienta se menciona para el preprocesamiento de datos en Python?
 - A. Scikit-learn.
 - B. Numpy.
 - C. Pandas.
 - D. TensorFlow.

3. ¿Qué técnica se menciona para reducir el sobreajuste en redes neuronales?
 - A. *Batch normalization*.
 - B. *Dropout*.
 - C. *Data augmentation*.
 - D. *Cross-validation*.

4. ¿Cuál es una práctica recomendada para la colaboración efectiva en equipos de desarrollo de *software*?
 - A. Evitar el uso de herramientas de gestión.
 - B. Comunicación clara y continua.
 - C. Trabajar de manera independiente.
 - D. Ignorar la documentación.

5. ¿Qué *framework* se menciona como herramienta para la implementación de modelos de *deep learning*?
- A. Hadoop.
 - B. PyTorch.
 - C. Spark.
 - D. H2O.
6. ¿Qué se debe hacer después de entrenar un modelo de IA para asegurarse de su correcto funcionamiento?
- A. Desplegarlo directamente en producción.
 - B. Evaluarlo con datos de prueba.
 - C. Reiniciar el entrenamiento.
 - D. Ignorar los datos de prueba.
7. ¿Cuál es un ejemplo de problema práctico en desarrollo de software que se puede resolver con IA?
- A. Optimización de recursos en la nube.
 - B. Clasificación automática de errores en *logs*.
 - C. Generación automática de código.
 - D. Mejora del rendimiento de bases de datos.
8. ¿Qué es lo que asegura un diseño adecuado de proyectos prácticos de IA?
- A. Que sean escalables y estén alineados con los objetivos del negocio.
 - B. Que sean fáciles de entender por todos.
 - C. Que utilicen siempre TensorFlow.
 - D. Que se hagan rápidamente.

9. ¿Qué implica la recolección y preparación de datos en un proyecto de IA?
- A. Recopilación de cualquier tipo de datos disponibles.
 - B. Uso exclusivo de datos ya preprocesados.
 - C. Recolección y estructuración de datos relevantes para el problema.
 - D. Uso de datos generados sintéticamente.
10. ¿Qué técnica se menciona para evaluar la generalización de un modelo de IA?
- A. *Data augmentation*.
 - B. Regularización L2.
 - C. Validación cruzada.
 - D. Normalización de datos.
11. ¿Qué herramienta se menciona para la gestión de proyectos y tareas en un equipo de desarrollo?
- A. GitHub.
 - B. Trello.
 - C. Jira.
 - D. Confluence.
12. ¿Cuál es el objetivo de la evaluación del modelo en un proyecto de IA?
- A. Verificar la exactitud del modelo en datos de prueba.
 - B. Probar todos los posibles modelos existentes.
 - C. Implementar el modelo en producción.
 - D. Recolectar más datos para mejorar el modelo.

13. ¿Qué opción describe una práctica incorrecta en la implementación de modelos de IA?
- A. Evaluar el modelo en datos no vistos.
 - B. Usar *frameworks*, como TensorFlow o PyTorch.
 - C. Entrenar el modelo sin pruebas de evaluación.
 - D. Implementar redes neuronales simples para tareas de clasificación.
14. ¿Qué es esencial para el éxito en la implementación de modelos de IA en aplicaciones del mundo real?
- A. Precisión y eficiencia del modelo.
 - B. Rapidez en el desarrollo del código.
 - C. Uso de librerías exclusivas de Python.
 - D. Evitar la colaboración en equipo.
15. ¿Qué se recomienda hacer en caso de que un modelo de IA no funcione correctamente con nuevos datos?
- A. Desplegar el modelo tal como está.
 - B. Realizar ajustes y optimización.
 - C. Cambiar el lenguaje de programación.
 - D. Ignorar los nuevos datos.
16. ¿Qué técnica se menciona como parte del proceso de ajuste y optimización de un modelo?
- A. Selección de algoritmos.
 - B. *Data augmentation*.
 - C. Uso de técnicas de regularización.
 - D. Diseño de interfaces gráficas.

17. ¿Qué problema común en modelos de deep learning puede ser mitigado por la capa dropout ?
- A. Subentrenamiento.
 - B. *Overfitting*.
 - C. Subestimación de parámetros.
 - D. Ruido en los datos.
18. ¿Qué fase sigue al entrenamiento del modelo en el desarrollo de proyectos de IA?
- A. Recolección de más datos.
 - B. Evaluación del modelo.
 - C. Despliegue del modelo.
 - D. Documentación del proceso.
19. ¿Cuál es una característica clave de la implementación de modelos en PyTorch?
- A. Simplicidad del proceso.
 - B. Control detallado sobre el entrenamiento.
 - C. Uso limitado en la industria.
 - D. Requiere menos datos que TensorFlow.
20. ¿Qué se debe hacer para mejorar la robustez de un modelo de IA?
- A. Cambiar el *framework*.
 - B. Utilizar un optimizador con menor tasa de aprendizaje.
 - C. Recolectar más datos.
 - D. Usar un solo algoritmo en todo el proyecto.